



## Original article

# Comparison of autoencoder architectures for fault detection in industrial processes

Deris Eduardo Spina<sup>a,\*</sup>, Luiz Felipe de O. Campos<sup>a</sup>, Wallthynay F. de Arruda<sup>a</sup>, Afrânio Melo<sup>a,c</sup>, Marcelo F. de S. Alves<sup>b</sup>, Gildeir Lima Rabello<sup>b</sup>, Thiago K. Anzai<sup>c</sup>, José Carlos Pinto<sup>a</sup>

<sup>a</sup> Programa de Engenharia Química, COPPE, Universidade Federal do Rio de Janeiro, Rio de Janeiro, RJ, CEP 21941-972, Brazil

<sup>b</sup> Programa de Pós-Graduação em Engenharia de Processos Químicos e Bioquímicos, EPQB, Universidade Federal do Rio de Janeiro, Rio de Janeiro, RJ, CEP 21941-909, Brazil

<sup>c</sup> Centro de Pesquisas Leopoldo Américo Miguez de Mello - CENPES, Petrobras - Petróleo Brasileiro SA, Rio de Janeiro, RJ, CEP 21941-915, Brazil

## ARTICLE INFO

## Keywords:

Autoencoder

Denosing

Variational

Fault detection

## ABSTRACT

Fault detection constitutes a fundamental task for predictive maintenance, requiring mathematical models that can be conveniently provided by data-driven techniques. Autoencoders are a particular type of unsupervised Artificial Neural Networks that can be suitable for fault detection applications. Diverse architectures might be used for autoencoders, resulting in different fault detection performances, which are usually compared by means of Fault Detection Rates for a fixed threshold of the False Alarm Rate, limiting the conclusions to particular cases. To improve the comparability, the present work uses the area under the receiver operating characteristic curve to compare autoencoder architectures for a range of false alarm rates using the Tennessee Eastman Process benchmark. Performances obtained for shallow and deep autoencoders were compared with those of the denoising and variational autoencoders for undercomplete and sparse structures. Overall, the results indicate better performances for sparse structures, especially for the variational autoencoder and the deep denoising autoencoder, with area under the curve of 98.35%.

## 1. Introduction

Industrial processes have become complex with numerous and integrated types of equipment and systems, giving rise to a wider range of possible undesirable events. In this context, more effective reliability assessment procedures and safety measures are needed in order to minimize the risk of accidents and losses (Zhang et al., 2023). On the other hand, an increasing amount of data is generated that may provide useful information about the process (Hozdić, 2015; Divya et al., 2023).

A popular framework used for assessing the reliability of a system is referred to as Process Monitoring. It is mainly constituted by a task known as Fault Detection, for which the main objective is to detect the occurrence of a fault as early as possible, making it feasible to apply appropriate maintenance procedures in a predictive manner (Ran et al., 2019; Zonta et al., 2020). Process Monitoring has been frequently divided into three categories referred to as (i) Knowledge-based, when experienced professionals define a set of process patterns; (ii) Model-based, when a model of the process is developed based on first-principles equations; and (iii) Data-driven, when models are developed based on statistical or machine learning techniques and process data (Miljković, 2011; Mansouri et al., 2020). Since the latter

does not rely on *a priori* knowledge about the system and given the increasing process data availability, it has become an appealing choice in the industry.

Principal Component Analysis (PCA) and Partial Least Squares (PLS) are the most popular and mature statistical techniques explored in process systems monitoring. However, PCA and PLS cannot handle problems involving nonlinearities, non-Gaussian distributions, or dynamics (Alauddin et al., 2018; Melo et al., 2024; Ávila Okada et al., 2021). On the other hand, Machine Learning (ML) methods are capable of extracting patterns hidden within the data in complex and even dynamic conditions. One of the most popular ML algorithms is the Artificial Neural Network (ANN), which essentially integrates several simple calculation units, called neurons, grouped in various layers (Carvalho et al., 2019). The interactions among the neurons allow the ANN to capture complex nonlinear behaviors within the data. A self-supervised ANN of particular interest in process monitoring is the Autoencoder (AE), for which the main objective is to reconstruct its inputs at the output layer (Carvalho et al., 2019; Géron, 2019; Chollet and Allaire, 2018). Both the nonlinear and the self-supervised characteristics of the AE turn this technique and its variations into an

\* Corresponding author.

E-mail address: [deris@peq.coppe.ufrj.br](mailto:deris@peq.coppe.ufrj.br) (D.E. Spina).

<https://doi.org/10.1016/j.dche.2024.100162>

Received 5 March 2024; Received in revised form 11 May 2024; Accepted 30 May 2024

Available online 31 May 2024

2772-5081/© 2024 The Authors. Published by Elsevier Ltd on behalf of Institution of Chemical Engineers (IChemE). This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

### List of Abbreviations

|          |                                      |
|----------|--------------------------------------|
| AE       | Autoencoders                         |
| ANN      | Artificial Neural Network            |
| AUC      | Area Under the Curve                 |
| DAE      | Denosing Autoencoder                 |
| Deep AE  | Deep Autoencoder                     |
| Deep DAE | Deep Denosing Autoencoder            |
| DL       | Deep Learning                        |
| DPCA     | Dynamic Principal Component Analysis |
| FAR      | False Alarm Rate                     |
| FDR      | Fault Detection Rate                 |
| KL       | Kullback-Leibler                     |
| ML       | Machine Learning                     |
| MSE      | Mean Squared Error                   |
| PCA      | Principal Component Analysis         |
| PLS      | Partial Least Squares                |
| ROC      | Receiver Operating Characteristic    |
| SAP      | Subspace Aided Approach              |
| SPE      | Squared Prediction Error             |
| TEP      | Tennessee Eastman Process            |
| TPLS     | Total Projection to Latent Structure |
| VAE      | Variational Autoencoder              |

attractive candidate for fault detection applications, as the currently available methods present poor performance when dealing with non-linear data (Zhang et al., 2018). For this reason, AE and some of its variants constitute the scope of the present work.

The most common architectures or variants of AE are stacked or deep AE, denoising AE, sparse AE, contractive AE, and variational AE (Qian et al., 2022a; Yang et al., 2022). Each architecture was developed to improve the performance of traditional AE using different strategies, generally involving changes in the structure of the network, altering the training procedure, or modifying the loss function. Recently, some other variants like adversarial AE, which combines variational AE and generative adversarial networks for data enrichment, and temporal deep learning-based AE, which combines AE with networks having dynamic characteristics like temporal convolutional networks to deal with long sequence memorization problems, have gained attention in the context of process monitoring (Jang et al., 2022; Lomov et al., 2021). A detailed description of the architectures considered in the present work and a literature review for each of them is given in Section 2.

A general procedure utilized to detect faults in industrial processes consists of extracting features from data representing normal operation conditions and using these features to construct an index related to the state of the process. Moreover, a threshold value is defined to be compared with the index in order to enable the classification of a sample as normal if the index does not surpass the threshold or as faulty otherwise. When autoencoder or any of its variants is applied to extract features in a fault detection task, the most commonly used index is the reconstruction error (Qian et al., 2022a).

Despite the wide range of AE variants, few works compared the performances of different Autoencoders (AE) architectures in fault detection applications for industrial contexts. Chadha et al. (2019) conducted a work of this type by comparing the results of the traditional AE architecture with those obtained by using the Variational Autoencoder (VAE) and the Denoising Autoencoder (DAE) architectures in terms of accuracy for the Tennessee Eastman Process (TEP) benchmark. They concluded that VAE systematically performed better than the other architectures that were tested in the conditions adopted in their work. The methodology used by Chadha et al. (2019) was

similar to the one proposed by Yin et al. (2012), although, as pointed out by Rieth et al. (2018), conclusions based on comparisons made by considering only one threshold for fault detection purposes are not generalizable, frequently masking eventual performance crossovers. For this reason, Rieth et al. (2018) proposed some alternatives to address this issue.

Based on the previous paragraphs, in the present paper, traditional AE architecture is compared with the VAE and DAE architectures to perform Fault Detection tasks based on the TEP benchmark and considering a range of thresholds. A Receiver Operating Characteristic (ROC) curve is constructed for each architecture in each faulty scenario, by changing the thresholds and determining the associated Fault Detection Rate (FDR) and False Alarm Rate (FAR). This procedure allows one to compare the performances of the different architectures for each threshold, showing any eventual performance crossover. The Area Under the Curve (AUC) indicates the best performance considering the entire range of thresholds evaluated in each case, which enables one to select the architecture that can provide the most satisfactory result, pointed out by the largest AUC value. This procedure can produce detailed and generalizable conclusions for the problem studied in the present work, providing a more reliable decision-making support. Therefore, the main contributions of this work are the proposition of a procedure that enables a detailed comparison among different techniques applied in a fault detection task independent of the fault detection threshold and the presentation of more general criteria based on the AUC for choosing among the techniques being compared.

In the following Section 2, the mentioned AE architectures are further explained. A brief description of the TEP benchmark is presented in Section 3. Section 4 presents a detailed description of the methodology proposed in the present work. In Section 5 the results obtained for each case are displayed and a detailed discussion is developed. Finally, the conclusions are summarized in Acknowledgments section.

## 2. Autoencoders

An Artificial Neural Network (ANN) consists of a mathematical model built from multiple simple functions (neurons) that process information as per Eq. (1) (Baughman and Liu, 1995). Neurons are interconnected generating a composite function that can capture non-linear and complex behavior on the data, especially in the context of Deep Learning (DL), where several layers of neurons are present (Ávila Okada et al., 2021; Carvalho et al., 2019), once its parameters are optimized in a step usually referred to as “training” of the model (Baughman and Liu, 1995) based on the instances fed into the network (Baughman and Liu, 1995).

$$o_i = f\left(\sum_{j=1}^n w_{ij} a_j + b_i\right) \quad (1)$$

where  $a_j$  represents as an input vector,  $o_i$  represents the output of neuron  $i$ ,  $f$  represents the activation function of the neuron,  $w_{ij}$  are the weights applied to the inputs  $ij$  of node  $i$  and  $b_i$  is the bias of neuron  $i$ .

ANN can be trained in a supervised manner, being necessary to provide the algorithm with information both about the input variables and their results. They can also be applied to capture representations of data in an unsupervised manner, as in autoencoders (Géron, 2019).

Autoencoders form a special class of self-supervised feedforward ANN where the main objective is to reconstruct the input data at the output layer. Therefore, for autoencoders, the true output is the same as the inputs,  $Y_i = X_i$ . A typical architecture of such a network is basically composed of two structures, an encoder and a decoder, separated by a bottleneck layer, also known as the code. This arrangement is illustrated in Fig. 1.

The encoder is responsible for extracting features from the original space of the variables by projecting the data nonlinearly into a subspace, giving rise to a latent representation illustrated by the code object in Fig. 1 (Melo et al., 2024). The decoder, in turn, maps the

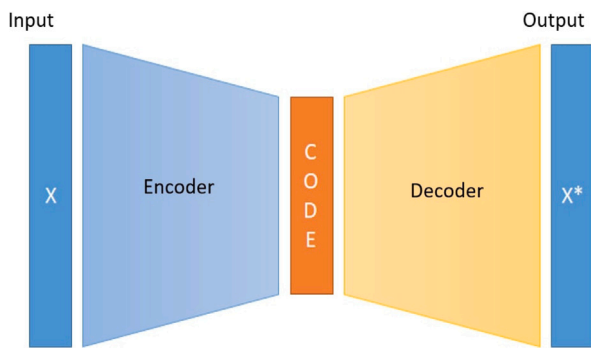


Fig. 1. Classical architecture of autoencoders.

latent structures back to the original space producing the reconstructed data (Géron, 2019; Qian et al., 2022b). Then, if  $p$  is the number of inputs and  $q$  is the number of neurons in the code layer, the encoder can be interpreted as a mathematical transformation from the space  $R^p$  to  $R^q$ . Similarly, the decoder can be understood as a mathematical transformation from the space  $R^q$  to  $R^p$ . It is evident that the dimension of the space  $R^p$  is not limited to the dimension of the space  $R^q$ .

Depending on the size of the code layer, the autoencoder can be classified as undercomplete or sparse. Undercomplete autoencoders have code dimensions that are smaller than the input dimension in order to force the model to learn a small set of useful features to describe the data. On the other side, sparse autoencoders can have code dimensions that are larger than the input dimension, although a regularization technique is needed to avoid overfitting.

In the present paper, an analysis will be performed on different types of autoencoders: simple AE, Deep Autoencoder (Deep AE), VAE, DAE and Deep Denoising Autoencoder (Deep DAE). For each type of autoencoder, an undercomplete and a sparse version were constructed to compare the effect of the code dimension on the obtained results.

### 2.1. Deep autoencoders

An autoencoder that contains multiple hidden layers (intermediate layers) is called a deep autoencoder. The large number of layers in the network is responsible for increasing the potential of abstraction of the technique, allowing it to learn more complex relationships among the variables, although it can become necessary to prevent overfitting in order to guarantee a good generalization of the reconstructions (Géron, 2019; Liu et al., 2021; Yu et al., 2019). It is common to construct symmetric autoencoders, with the same number of layers in the encoder and the decoder (Fig. 2).

In the context of fault detection, several authors employ deep autoencoders as a tool. Kathlyn et al. (2023) used the deep autoencoder approach to detect faults in a natural gas processing process. This choice is justified by the prevalence of such faults and the scarcity of in-depth investigations on the subject in the literature. Additionally, Ganesan and Lavanya (2023) chose to combine deep autoencoders with the Random Forest algorithm for the detection and diagnosis of faults in satellite operations. These approaches are important due to the need for effective methods for early detection and accurate diagnosis of faults in complex systems, such as those mentioned.

### 2.2. Denoising autoencoders

Besides limiting the dimension of the latent space, forcing autoencoders to learn from noisy data also helps them to identify and learn the most important features of the data for the most appropriate representation of the available information, as in the case of Denoising Autoencoders (Géron, 2019; Vincent et al., 2010).

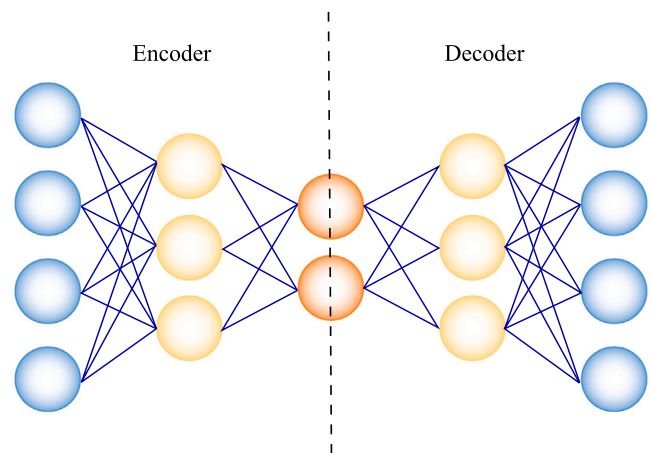


Fig. 2. Illustrative description of a deep autoencoder architecture.

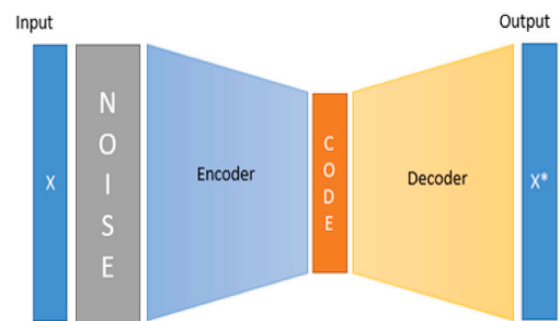


Fig. 3. Illustrative description of a denoising autoencoder architecture.

According to the DAE structure, as represented in Fig. 3, noise is added after the input layer using one of two strategies. The first option is to add noise (usually gaussian noise) to the inputs and the second one is to add noise through a dropout layer that feeds only part of the information on each instance of the inputs of the encoder (Géron, 2019). In both cases, noise is only present during training, which is performed analogously to the training of regular autoencoders (Géron, 2019), although the reconstruction error must be evaluated by comparing the outputs to the original uncorrupted inputs, even though the encoder is actually fed with noisy versions of the inputs (Vincent et al., 2010).

Training of DAE can be schematically represented by Eq. (2). As the actual inputs of the encoder are different from the DAE's target outputs, the proposed training prevents the autoencoders from simply copying the inputs, a typical concern (Bank et al., 2020; Vincent et al., 2010; Bengio et al., 2013). Besides, it is assumed that a higher level representation of the instances of training can lead to robust performances against the noise. So, the corrupted input data can be sufficient to allow the proper learning of the analyzed features (Vincent et al., 2010).

$$\underset{A, B}{\operatorname{argmin}} E[\Delta(x, B(A(x')))] \quad (2)$$

in which  $x$  represents the corrupted inputs that are fed to the encoder  $A$  and  $B(A(x'))$  represents the output  $X^*$  of the decoder  $B$  in a DAE.

The applications of denoising autoencoders are prevalent across various research domains. Zhang et al. (2018) combined stacked denoising autoencoders with kNN (k Nearest Neighbors) for fault detection in the Tennessee Eastman Process. They monitored the distances between samples in the autoencoder's feature and residual space. Another effective combination is denoising autoencoders with extreme gradient boosting (XGBoost). Zhang et al. (2022) applied this method to detect and diagnose faults in wind turbines, enabling the identification of

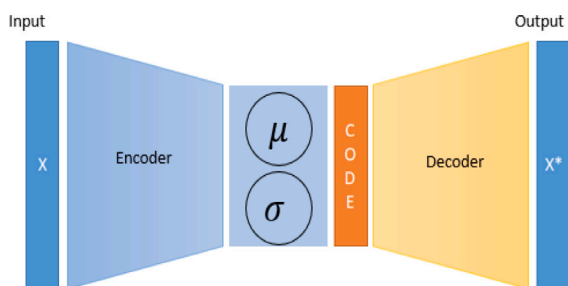


Fig. 4. Illustrative description of a Variational Autoencoder architecture.

the temporal behavior of variables under normal conditions. In the context of wind turbines, [Chen et al. \(2020\)](#) proposed using denoising autoencoders and the moving window approach to reconstruct data under normal conditions.

### 2.3. Variational autoencoders

The architecture of a typical VAE is intrinsically probabilistic, as it is based on a random component in both the training and prediction steps as opposed to the previously presented architectures. The use of probability for predictions can lead to generation of new data by the encoder, which also allows VAE to be used for generation of new data ([Géron, 2019](#)).

The differential attributes can be achieved with two main modifications of the VAE's structure when compared to traditional Autoencoders. First, as represented in [Fig. 4](#), between the “traditional” encoding and decoding parts of the AE, a sampling layer is added in the VAE. The encoding task can then be viewed as two separated parts. The first one is fitting the parameters of a probability distribution that describes the VAE's input instances; in the case of Gaussian distribution, the parameters are the mean ( $\mu$ ) and standard deviation ( $\sigma$ ). In that sense, the sampling layer will be, in a “bottleneck manner”, fed only with a mean coding and its standard deviation, as opposed to traditional direct generation of one coding per instance at the encoder. This fitted probability function is the latent representation of the data (the coding space) in a VAE and, thus, this becomes the main feature extracted from the data ([Géron, 2019](#)).

However, one coding per instance is also needed in VAE, so that generating codes constitutes the second encoding task, performed in the added sampling layer. In this layer, the fitted parameters for the probability distribution are used to create random samples from the coding space, which are fed to the decoder to generate the outputs. The same type of sampling from latent space is used after training and, because of the randomness, can also be used to generate new data.

The second modification in VAE's is the definition of the loss function  $L$  used to train these models, now comprising the two contributions shown in [Eq. \(3\)](#) ([Bank et al., 2020](#); [Chadha et al., 2019](#)). Summed to a reconstruction error (first term) that measures differences between inputs and outputs, thus forcing them to be similar, the Kullback-Leibler (KL) divergence term (second term) also forces the probability distributions of inputs and latent-space variables to be similar ([Géron, 2019](#); [Chadha et al., 2019](#)). This KL regularization strategy typically simplifies the probability distribution of data in latent space when compared to the distribution of the original data ([Kingma and Welling, 2019](#)), forcing two instances that are close to each other in the latent space to be similar in the real input (or output) domain. This proximity is what enables sampled codings to be meaningful once decoded ([Bank et al., 2020](#)).

$$L = -\mathbb{E}_{Z \sim Q} [\log P(X|Z)] + \mathbb{KL}(Q(Z|X) \parallel P(Z)) \quad (3)$$

in which  $X$  represents the Autoencoder's inputs,  $Z$  represents the latent space variables,  $Q(Z|X)$  is the function that generates codings,

$E$  denotes the expectation operator, and  $P$  denotes the considered probability distribution function.

The detection of faults using variational autoencoders has been a growing area of research. Several authors have explored the potential of VAE for monitoring industrial processes, highlighting their ability to identify anomalies and diagnose faults in complex systems. For instance, [Jakubowski et al. \(2022\)](#) investigated the use of VAE to monitor the process of a steel plant, demonstrating their effectiveness in detecting anomalies. Besides, [Zhu et al. \(2022\)](#) evaluated different variants of variational autoencoders for monitoring industrial processes, such as the Tennessee Eastman reference process, concluding that recurrent VAEs are recommended. Additionally, [Tang et al. \(2021\)](#) proposed an innovative approach by combining VAE with the deep variational information bottleneck (VIB) algorithm to extract latent variables related to quality in industrial processes, further improving fault detection capabilities. [Arias Chao et al. \(2021\)](#) addressed the challenge of lacking labeled data for fault diagnosis by proposing an adapted VAE that utilizes all available information in training, resulting in significant improvements in fault detection and segmentation. These studies highlight the potential of VAEs for fault detection in industrial systems, showcasing different approaches and enhancements for this application.

## 3. Methods

### 3.1. Tennessee Eastman process

The TEP is a process flowsheet proposed by Eastman Chemical Company as a benchmark for applications involving process simulation, control and monitoring ([Downs and Vogel, 1993](#)). The TEP process is depicted in [Fig. 5](#). Two products and one by-product are generated from four reactants with the presence of an inert. The reactions are assumed to be irreversible and exothermic. The process comprises five units: reactor, condenser, vapor-liquid separator, stripper and recycle centrifugal compressor.

The dataset used in the present work was provided by Prof. Richard D. Braatz through an adaptation of the original simulation scheme and can be downloaded in [the reference's link](#) ([Chiang et al., 2003](#)). The data files consist of 25 h of operation for training data, 48 extra hours of normal operation for validation data and test data files consisting of 48 h of operation for each fault, where the fault is introduced after 8 h. Of the total of 52 measured variables, 22 are process variables, 11 are manipulated variables and 19 are composition variables. Process and manipulated variables are sampled with sampling intervals of 3 min. Sampling intervals of composition variables are equal to 6 or 15 min. The adopted plant-wide control strategy was proposed by [Lyman and Georgakis \(1995\)](#). Available faults are shown in [Table 1](#) and include step disturbances, increased process variability, reaction kinetics drift, valve sticking, and other unknown disturbances. For specific details of this dataset, including an extensive exploratory data analysis, the reader is referred to [Melo et al. \(2022\)](#).

### 3.2. Fault detection method

The methodology proposed in the present study consists of using the normal training data without faults from the TEP benchmark, to perform the training of 5 different types of autoencoders: simple AE, Deep AE, DAE, Deep DAE and VAE. For each type of autoencoder, undercomplete and sparse versions are constructed and compared to the other autoencoder versions. Before training, the data are treated through a pre-processing step. Pre-processing is performed to normalize the data, according to [Eq. \(4\)](#). Additionally, a comparison will be conducted with the Principal Component Analysis (PCA) technique, aiming to achieve a comparative analysis with a commonly employed approach.

$$x^* = \frac{x - \mu}{\sigma} \quad (4)$$



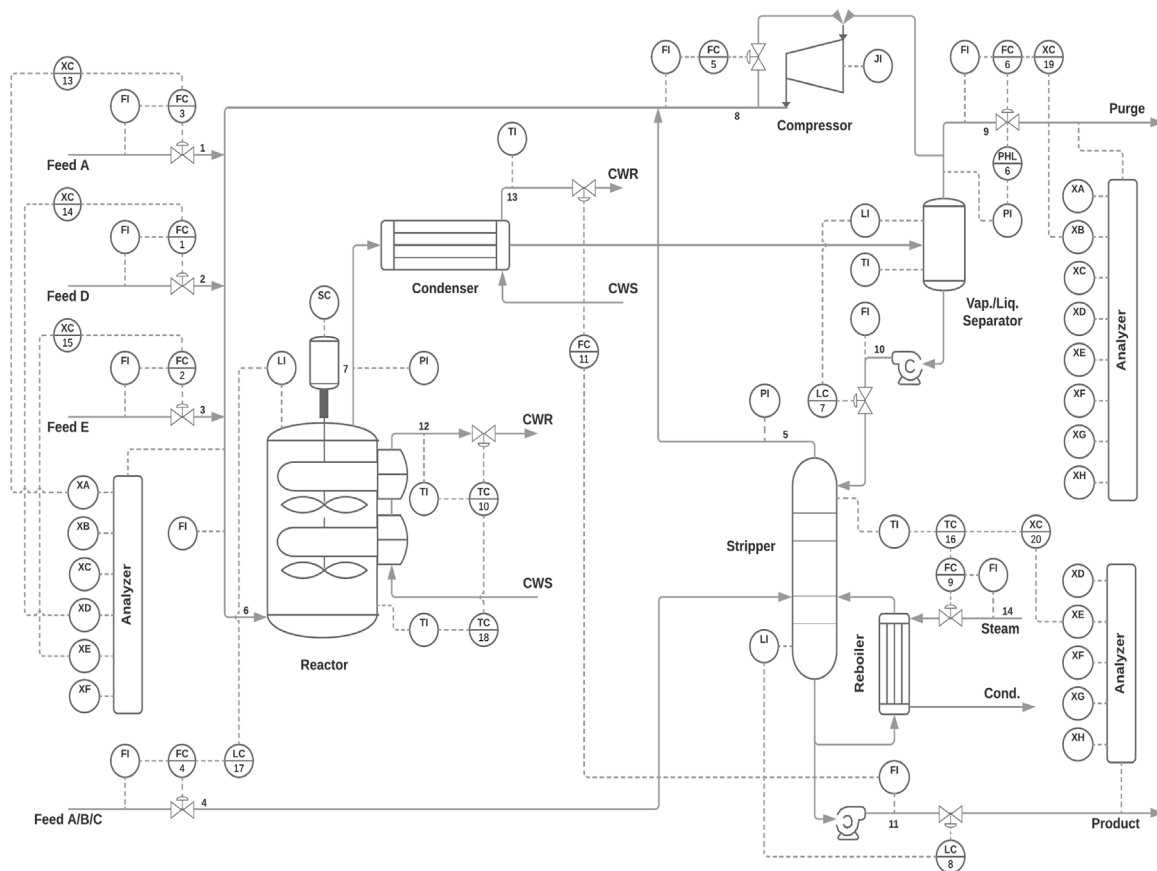


Fig. 5. Tennessee Eastman Process diagram (Xavier and de Seixas, 2018).

**Table 1**  
Tennessee Eastman Process faults.

| Fault   | Description                                                   |
|---------|---------------------------------------------------------------|
| IDV(1)  | A/C feed ratio, B composition constant (stream 4) step        |
| IDV(2)  | B composition, A/C ratio constant (stream 4) step             |
| IDV(3)  | D feed temperature (stream 2) step                            |
| IDV(4)  | Reactor cooling water inlet temperature step                  |
| IDV(5)  | Condenser cooling water inlet temperature step                |
| IDV(6)  | A feed loss (stream 1) step                                   |
| IDV(7)  | C header pressure loss - reduced availability (stream 4) step |
| IDV(8)  | A, B, C feed composition (stream 4) random variation          |
| IDV(9)  | D feed temperature (stream 2) random variation                |
| IDV(10) | C feed temperature (stream 4) random variation                |
| IDV(11) | Reactor cooling water inlet temperature random variation      |
| IDV(12) | Condenser cooling water inlet temperature random variation    |
| IDV(13) | Reaction kinetics slow drift                                  |
| IDV(14) | Reactor cooling water valve sticking                          |
| IDV(15) | Condenser cooling water valve sticking                        |
| IDV(16) | Unknown                                                       |
| IDV(17) | Unknown                                                       |
| IDV(18) | Unknown                                                       |
| IDV(19) | Unknown                                                       |
| IDV(20) | Unknown                                                       |

After performing the pre-processing and training steps, variable prediction is carried out for the validation data without faults and the mean squared error of the reconstruction is calculated. The fault detection is performed when the model predictions surpass a threshold value, which can be set as a percentile (e.g. 99%) of the empirical distribution of the Squared Prediction Error (SPE) observed in the validation data set.

Subsequently, variable predictions are calculated for the test data with faults. For each observation, the mean squared error is compared with the threshold value calculated earlier. If the threshold value is

surpassed, the observation is considered a fault. This procedure is repeated for several different threshold values, changing the percentile between 0 and 100% of the error distribution in the validation data set. For each threshold value, the FDR and the FAR are calculated and plotted to generate the ROC curve. An overall ROC curve was determined for all faults, and for each fault. Finally, the AUC metric is calculated through numerical integration of the area under the ROC curve.

As in each test simulation the fault starts only after 8 h, the initial observations are not considered for the calculation of the FDR. For the calculation of the FAR, both the observations in the validation data and the initial 8 h of simulation for each failure were considered.

### 3.3. Fault detection using a fixed threshold

As stated previously, using AUC as the metric for comparing fault detection models is a relatively new approach; thus, most studies in the literature only report the Fault Detection Rate and False Alarm Rate using a fixed threshold. To make a comparison between the trained AEs and previous literature works, a fixed threshold of 99% is set for the SPE distribution in the validation set, and the Fault Detection Rate was calculated for the different architectures.

Due to the large number of faults, the comparison is focused on three specific faults: Faults 1, 3 and 5. Fault 1 is considered easily detectable, and it is linked to a step change in the A/C feed ratio for stream 4. Fault 3 is a fault that is difficult to detect and is related to a step change in the supply temperature of D in stream 2. Fault 5 is associated with a step change in the condenser cooling water inlet temperature, and fault detection methods usually present intermediate results.

The Fault Detection Rate of the created AEs is compared to the results of different fault detection models from the literature. In addition, ROC curves are constructed for the three faults. This comparison promotes a discussion over the limitations of using a single threshold and the advantages of ROC curves for comparing different models. To complete the analysis, Square Prediction Error (SPE) Charts were created for the simple Autoencoder and Variational Autoencoder. The SPE Charts allow the visualization of the error for each measurement and show how the defined threshold distinguishes between normal and faulty points.

### 3.4. Hyperparameter tuning

The tuning of hyperparameters constitutes an important step for building autoencoders because the final performance can be highly dependent on the hyperparameters, such as the number of hidden layers, code size, number of neurons in the hidden layers, and activation functions. The objective of the hyperparameter tuning task is to find the values that provides the best performance in the validation dataset for a specific model. In the present work, the adam optimizer (Kingma and Ba, 2014) were used for all the autoencoders, but different strategies were used to tune the other hyperparameters for undercomplete and sparse autoencoders.

For the undercomplete autoencoders, there is a trade-off between the code size and the reconstruction error. To select the best code size, a graphical strategy, called elbow method (Humaira and Rasyidah, 2020), commonly used to determine the number of clusters in unsupervised clustering algorithms, was employed (Thorndike, 1953). This method consists of plotting the Mean Squared Error (MSE) as a function of the code size and selecting the value at which increasing the code size does not produce a significant reduction of the MSE. For Deep AE and Deep DAE, the number of neurons for the other hidden layers was fixed at 52 (equal to the number of inputs).

For the sparse autoencoders, hyperparameter tuning was performed to determine the code size, number of neurons in hidden layers and the regularization terms. Tuning was carried out with help of the Optuna library for Python, which implements a bayesian optimization algorithm (Preferred Networks Inc., 2023; Akiba et al., 2019). The final values for the hyperparameters are shown in Table 2.

It is worth noting that the upper limit for the code size and the neurons in the hidden layers during the optimization was 400 and that optimized values were close to this maximum value. For the activation function, the options were tanh and ReLU. In addition, three regularization techniques were combined for training the sparse autoencoders: lasso (L1) (Vidaurre et al., 2013), ridge (L2) (Hastie, 2020) and dropout (Hinton et al., 2012; Wager et al., 2013). The possible values during optimization for the L1 and L2 weights were between  $10^{-5}$  and 1, while the possible values for the dropout rate changed from 0 to 0.50 with 0.10 steps.

For the implementation of the principal component analysis, an examination of the number of principal directions was carried out to evaluate the impact of the variance explained by the PCA technique on the AUC value. Therefore, the PCA method selected for comparison with the autoencoder techniques was the one that maximizes the AUC value. This approach enables a more precise and robust comparative analysis between the techniques, considering the quality of data representation and its influence on performance metrics, such as AUC.

## 4. Results and discussion

### 4.1. Code size for undercomplete autoencoders

The code size for the undercomplete autoencoders was selected using the elbow method described in the methodology. Fig. 6 shows that the reduction in reconstruction error is negligible when the code size is about 40 for all the evaluated autoencoders. Therefore, the different types of undercomplete autoencoders were implemented considering 40 neurons in the code layer.

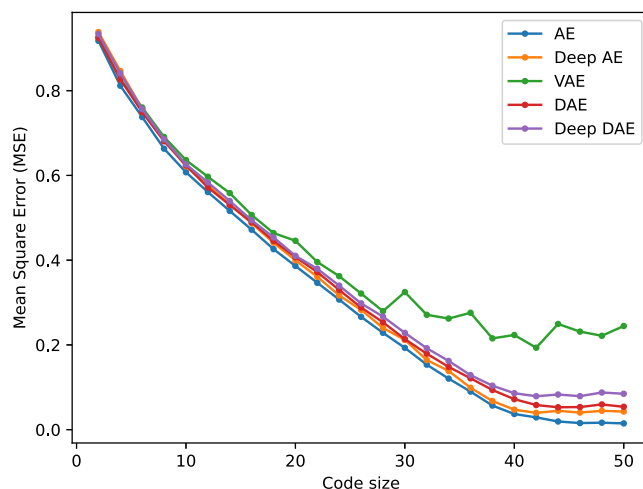


Fig. 6. Tuning of code sizes using the elbow method.

### 4.2. Reconstruction error

The MSE was the metric used here to evaluate the reconstruction error of the different types of autoencoders and for data validation. To compare the reconstruction capabilities of the analyzed autoencoders, the MSE was determined for both undercomplete and sparse autoencoders (see Table 3).

The sparse autoencoders were better at reconstructing the data, meaning that increasing the number of neurons caused the reduction of the reconstruction error when the regularization techniques were applied. In both scenarios (undercomplete and sparse), the traditional autoencoder performed better, as it provided the lowest MSE. One of the reasons for this is that denoising DAE and VAE make a trade-off between accuracy and robustness, which increases MSE and can be useful to detect anomalies. Besides, the VAE provided significantly lower performance than the others when the code size was limited, but allowed the appropriate reconstruction in the sparse scenario. Although the traditional autoencoder was able to better reconstruct the original space, it is necessary to investigate the use of the different types of autoencoders for fault detection.

### 4.3. Fault detection: overall ROC and AUC comparison

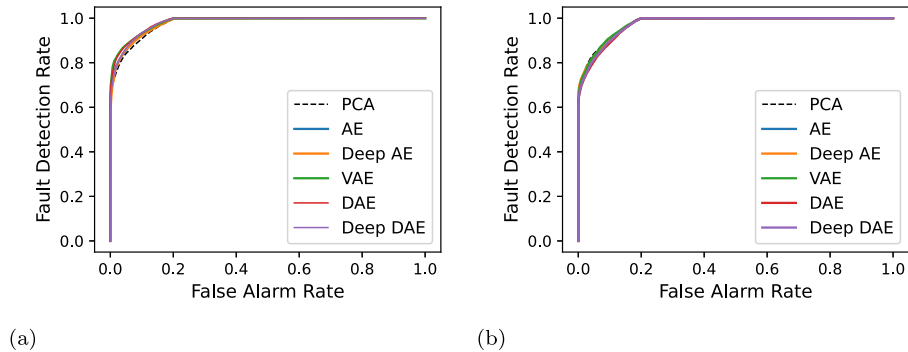
A first analysis of fault detection by the different types of autoencoders was performed by evaluating the ROC curves for the overall performance across the 21 labeled faults on the Tennessee Eastman Process. Fig. 7-a shows the performance for the sparse architectures, while Fig. 7-b shows the performances for the undercomplete architectures. Besides the ROC curves for the different autoencoders, both Figures show the PCA's ROC curve for comparison.

Based on Fig. 7, one can see that all tested architectures, including both sparse and undercomplete architectures, provided similar performances from an overall fault detection point of view. All the ROC curves presented similar shape and good fault detection rates, given that for most FAR values, the FDR was higher than 60%. Fig. 8 shows the magnification of the curves displayed in Fig. 7 in the region where the differences among the ROC curves were larger, with FDR values above 60% and FAR values under 15%.

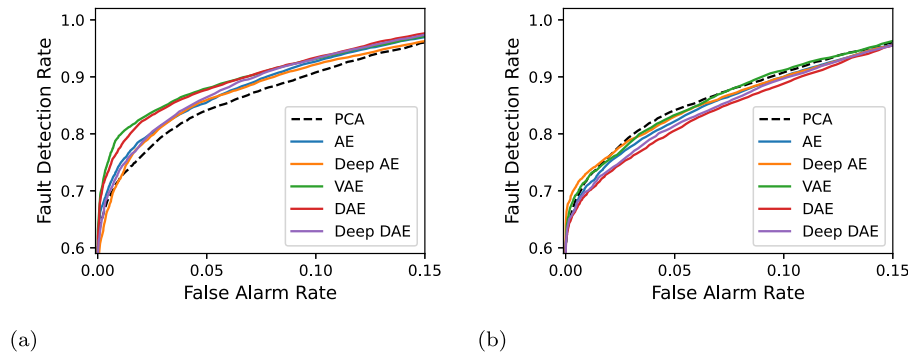
ROC curves that remain close to the y-axis up to high FDR rates can be regarded as better, as this means that a higher FDR can be achieved with low FAR. From this graphical point of view, Fig. 8 shows good performances of the sparse autoencoders, especially the VAE and DAE for FAR lower than 15%. In addition, when comparing the results obtained for autoencoders and PCA, all the sparse autoencoders

**Table 2**  
Optimized hyperparameters for sparse autoencoders.

|                                                 | AE      | Deep AE | VAE     | DAE     | Deep DAE |
|-------------------------------------------------|---------|---------|---------|---------|----------|
| N <sup>o</sup> hidden layers in encoder/decoder | –       | 1       | 1       | –       | 1        |
| N <sup>o</sup> neurons in hidden layers         | –       | 398     | 127     | –       | 384      |
| Code size                                       | 399     | 394     | 400     | 384     | 393      |
| Activation function                             | tanh    | tanh    | tanh    | tanh    | tanh     |
| L1 regularization factor                        | 1.77e–3 | 6.85e–4 | 3.34e–3 | 7.33e–6 | 2.71e–4  |
| L2 regularization factor                        | 5.43e–2 | 1.02e–6 | 2.96e–5 | 0.220   | 1.84e–5  |
| Dropout rate for hidden layers                  | 0       | 0.1     | –       | 0.1     | 0.1      |
| Dropout rate for input layer                    | –       | –       | –       | 0.1     | 0.1      |



**Fig. 7.** ROC curves for the overall fault detection evaluation: (a) sparse; (b) and undercomplete AEs.



**Fig. 8.** Magnification of ROC curves for the overall fault detection evaluation: (a) sparse; (b) and undercomplete AEs.

**Table 3**

Reconstruction error (MSE) for the validation data without faults using different types of autoencoders.

| Model type                 | Reconstruction Error (MSE) |               |
|----------------------------|----------------------------|---------------|
|                            | Sparse                     | Undercomplete |
| AutoEncoder                | 0.0039                     | 0.0370        |
| Deep AutoEncoder           | 0.0203                     | 0.0472        |
| Variational AutoEncoder    | 0.0261                     | 0.2209        |
| Denoising AutoEncoder      | 0.0274                     | 0.0720        |
| Deep Denoising AutoEncoder | 0.0578                     | 0.0860        |

**Table 4**

AUC values for the overall fault detection evaluation.

| Architecture               | Undercomplete | Sparse |
|----------------------------|---------------|--------|
| Autoencoder                | 0.9829        | 0.9809 |
| Deep Autoencoder           | 0.9832        | 0.9795 |
| Variational Autoencoder    | 0.9796        | 0.9835 |
| Denoising Autoencoder      | 0.9813        | 0.9835 |
| Deep Denoising Autoencoder | 0.9812        | 0.9815 |
| PCA                        | 0.9779        |        |

provided higher FDR values than PCA for most FAR values in the shown region. On the other hand, PCA provided better results than most of the undercomplete autoencoders in this region. Therefore, sparse autoencoders provided a better ROC curve for fault detection than undercomplete autoencoders for FAR lower than 15%.

For the numerical and objective evaluation of the performances, the AUC values for each curve are also presented in Table 4. In general, when the area under the ROC curve is higher, the overall model performance for fault detection is better. Unlike what was observed for FAR values smaller than 15%, the Table 4 indicates that the AUC values for sparse and undercomplete autoencoders were similar. This may be associated with a better FDR of the undercomplete models for

higher FAR values, which also explains why these autoencoders present a better AUC than PCA despite presenting a worse performance in the region shown in Fig. 8.

Table 4 shows an interesting behavior for VAE's performance. VAE provided the worst AUC among the undercomplete autoencoders, while the sparse VAE presented one of the best results. The opposite is true for Deep AE, with a good AUC for the undercomplete architecture but the worst result among the sparse autoencoders, indicating that the larger number of parameters in sparse Deep AE is more difficult to regularize.

The best AUC values were obtained by the sparse VAE and the sparse DAE. According to Fig. 8, when FAR values were below 7%, sparse VAE yielded higher FDR values than the sparse DAE. However, for higher values of FAR, sparse DAE provided the highest FDR values.

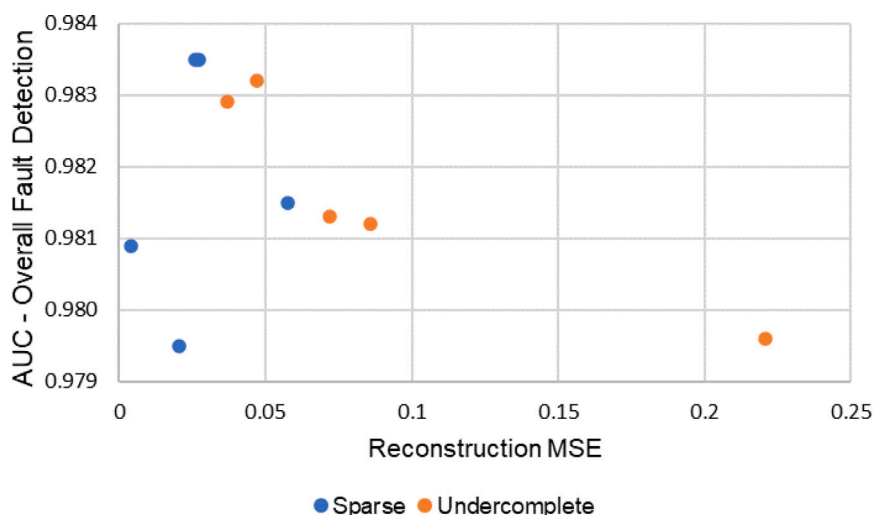


Fig. 9. AUC values as a function of MSE values.

Since, in practice, the FAR values should be relatively low, from an overall fault detection point of view, the sparse VAE provided the best performance among the evaluated models. These results agree with the findings of Chadha et al. (2019) and they indicate that the sparse DAE could be useful if higher FAR values are allowed.

Taking into consideration the performance of traditionally-applied PCA for the overall fault detection, its replacement for sparse autoencoders can be suggested when analyzing TEP since all studied architectures presented better AUC. In addition, although some AEs presented lower FDR than PCA for smaller FAR, PCA provided the lowest FAR values above 3%. In particular, the use of sparse VAE can be suggested, as it presented better performance both in terms of AUC and graphically.

Finally, considering the results displayed in Tables 3 and 4, it seems that there is no significant correlation between the reconstruction error and the fault detection performance over the whole data set period. When undercomplete and sparse architectures are segregated, however, one can observe a consistent trend of better performances for autoencoders that provided lower reconstruction errors on training, as shown in Fig. 9. Nevertheless, it must be highlighted that the number of datapoints is small, and, although the MSE values were subject to large variations, the AUC values were similar.

#### 4.4. ROC curves and AUC values for selected faults

The AUC values of all investigated autoencoders for each fault are shown in Table 5. As the number of faults is high, the fault detection and diagnosis results for Faults 1, 3 and 5 are used as examples to summarize the findings. For comprehensive insights into the detection performance across all faults, please refer to the supplementary material, which includes ROC curves and SPE Charts for each fault.

Fig. 10 shows the ROC curves for Fault 1 of the TEP dataset, comparing sparse and undercomplete autoencoders. Since Fault 1 is a relatively simple fault to detect, all autoencoders provided very similar ROC curves. The autoencoders presented high FDR rates and low FAR rates, achieving AUC values close to 1.

For Fault 3, which is considered difficult to detect, the autoencoders provided a bad performance. In Fig. 11-a it can be observed that the Deep AE for low FAR values behaved as the best sparse autoencoder, providing the highest FDR value up to 4% of FAR, when it was surpassed by the Denoising Autoencoder. For the undercomplete architecture, shown in Fig. 11-b, DAE provided the highest FDR value for this fault.

For Fault 5, as shown in Fig. 12-a, one can notice that the simple AE provided the highest FDR and the lowest FAR for the sparse architecture, so that this can be considered the architecture with the best performance for this fault, while the Deep DAE provided the worst performance. For the undercomplete architecture (see Fig. 12-b), the Deep AE produced the highest FDR and lowest FAR values. After 2% of FAR the VAE provided the worst performance for this fault as the FDR value was significantly low when compared to the values obtained with the other AEs. This behavior is similar to what was shown for the reconstruction error, where the undercomplete VAE had a poor performance.

#### 4.5. Fault detection using a fixed threshold

To compare the trained autoencoders with other results in the literature, the Fault Detection Rate was calculated for the Autoencoders using a fixed threshold of 99% for the SPE distribution in the validation dataset. Table 6 shows the resulting detection rates and reported False Alarm Rates for the sparse and undercomplete autoencoders.

For Fault 1, all methods achieve a great detection rate, close to 100%. As the fault is easily detectable, small changes in the threshold would not affect the Fault Detection Rate. This can be seen in the SPE Charts for Fault 1, shown in Fig. 13. The SPE values for the faulty observations are high, when compared to the faultless observations. Then, increasing or reducing the threshold would barely affect the number of faults detected.

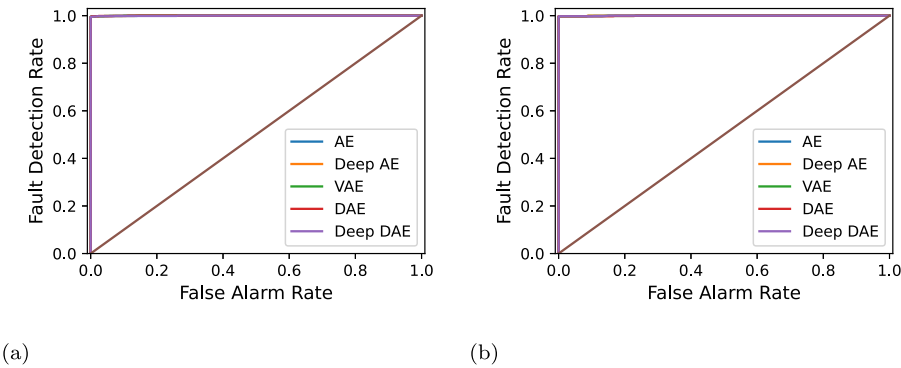
For Fault 3, the proposed Autoencoders presented difficulties in achieving a good Fault Detection Rate at this threshold level, registering rates between 0.75% and 2.63%. This is shown in the SPE Charts of Fig. 14, where the Autoencoder and Variational Autoencoder could not distinguish the fault from normal operation for most observations. This corroborates published studies that claim that this fault is hard to detect with other fault detection techniques, because the TEP control system can maintain the normal operation when this fault appears (Lau et al., 2013).

From the other techniques presented in Table 6, the Subspace Aided Approach (SAP) achieved 6.38% with a similar False Alarm Rate of 1.5%. The Stacked Denoising Autoencoder with KNN, proposed by Zhang et al. (2018) had the best results of 34.9%. The Dynamic PCA (DPCA) and Total Projection to Latent Structure (TPLS) approaches had better results (12.88% and 24.25%, respectively) than the autoencoders but at the cost of a much higher False Alarm Rate of 10.13% and 19.62%. This is one of the reasons why using a single threshold limits the comparability of the results, as different thresholds give distinct combinations of FAR and FDR, and it is not trivial to indicate which

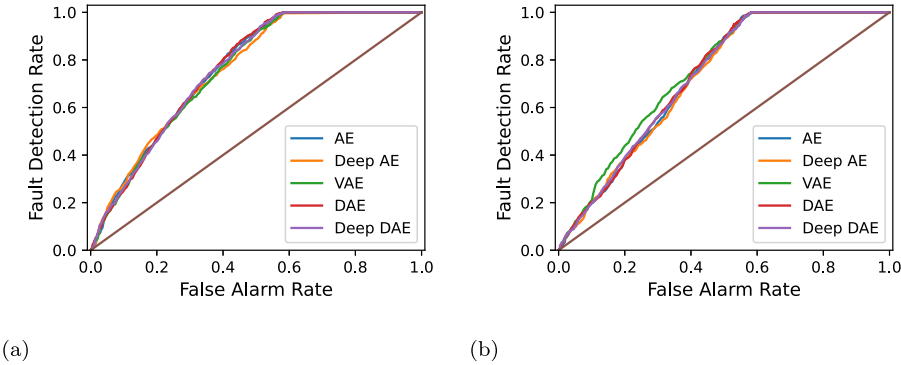


**Table 5**  
AUC values for each fault and autoencoder.

| Fault number | AUC           |         |       |       |          |        |         |       |       |          |       |
|--------------|---------------|---------|-------|-------|----------|--------|---------|-------|-------|----------|-------|
|              | Undercomplete |         |       |       |          | Sparse |         |       |       |          | PCA   |
|              | AE            | Deep AE | VAE   | DAE   | Deep DAE | AE     | Deep AE | VAE   | DAE   | Deep DAE | PCA   |
| 1            | 1.000         | 1.000   | 1.000 | 1.000 | 1.000    | 1.000  | 1.000   | 1.000 | 1.000 | 1.000    | 1.000 |
| 2            | 0.998         | 0.997   | 0.998 | 0.997 | 0.997    | 0.997  | 0.997   | 0.997 | 0.998 | 0.997    | 0.997 |
| 3            | 0.743         | 0.754   | 0.754 | 0.754 | 0.761    | 0.763  | 0.758   | 0.756 | 0.763 | 0.761    | 0.718 |
| 4            | 1.000         | 0.999   | 0.999 | 0.998 | 0.999    | 0.994  | 0.990   | 0.998 | 0.998 | 0.995    | 1.000 |
| 5            | 1.000         | 1.000   | 0.899 | 0.912 | 0.900    | 0.999  | 0.972   | 0.987 | 0.981 | 0.928    | 0.952 |
| 6            | 1.000         | 1.000   | 1.000 | 1.000 | 1.000    | 1.000  | 1.000   | 1.000 | 1.000 | 1.000    | 1.000 |
| 7            | 1.000         | 1.000   | 1.000 | 1.000 | 1.000    | 1.000  | 1.000   | 1.000 | 1.000 | 1.000    | 0.999 |
| 8            | 0.998         | 0.997   | 0.996 | 0.997 | 0.997    | 0.997  | 0.996   | 0.997 | 0.997 | 0.997    | 0.99  |
| 9            | 0.743         | 0.746   | 0.735 | 0.743 | 0.746    | 0.746  | 0.739   | 0.737 | 0.748 | 0.747    | 0.734 |
| 10           | 0.959         | 0.957   | 0.932 | 0.950 | 0.945    | 0.932  | 0.933   | 0.969 | 0.964 | 0.951    | 0.933 |
| 11           | 0.960         | 0.952   | 0.951 | 0.953 | 0.957    | 0.947  | 0.940   | 0.953 | 0.954 | 0.949    | 0.941 |
| 12           | 0.999         | 0.999   | 0.999 | 0.999 | 0.999    | 0.999  | 0.999   | 0.999 | 0.999 | 0.999    | 0.991 |
| 13           | 0.990         | 0.989   | 0.987 | 0.989 | 0.989    | 0.989  | 0.989   | 0.988 | 0.988 | 0.989    | 0.988 |
| 14           | 1.000         | 1.000   | 1.000 | 1.000 | 1.000    | 1.000  | 1.000   | 1.000 | 1.000 | 1.000    | 0.984 |
| 15           | 0.747         | 0.755   | 0.741 | 0.746 | 0.751    | 0.749  | 0.744   | 0.749 | 0.750 | 0.748    | 0.731 |
| 16           | 0.958         | 0.959   | 0.934 | 0.945 | 0.943    | 0.925  | 0.923   | 0.970 | 0.966 | 0.948    | 0.934 |
| 17           | 0.992         | 0.991   | 0.989 | 0.990 | 0.990    | 0.984  | 0.981   | 0.989 | 0.989 | 0.986    | 0.992 |
| 18           | 0.976         | 0.975   | 0.978 | 0.979 | 0.977    | 0.978  | 0.977   | 0.976 | 0.978 | 0.978    | 0.973 |
| 19           | 0.937         | 0.940   | 0.884 | 0.918 | 0.911    | 0.866  | 0.854   | 0.957 | 0.943 | 0.905    | 0.924 |
| 20           | 0.944         | 0.948   | 0.933 | 0.940 | 0.941    | 0.933  | 0.927   | 0.949 | 0.943 | 0.936    | 0.931 |
| 21           | 0.891         | 0.885   | 0.863 | 0.875 | 0.867    | 0.855  | 0.854   | 0.867 | 0.864 | 0.852    | 0.898 |



**Fig. 10.** ROC curves for fault 1: (a) sparse; (b) and undercomplete autoencoders.



**Fig. 11.** ROC curves for fault 3: (a) sparse; (b) and undercomplete autoencoders.

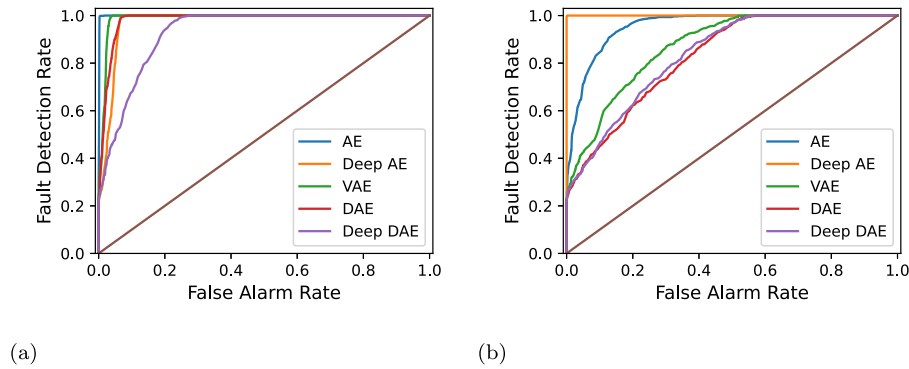


Fig. 12. ROC curves for fault 5: (a) sparse; (b) and undercomplete autoencoders.

Table 6

Comparison of Fault Detection Rate and False Alarm Rate obtained using different Autoencoder architectures with a fixed 99% threshold against models from the literature for faults 1, 3 and 5.

| Model                                             | Type                   | Fault detection rate % |         |         | False alarm rate %       | Source              |
|---------------------------------------------------|------------------------|------------------------|---------|---------|--------------------------|---------------------|
|                                                   |                        | Fault 1                | Fault 3 | Fault 5 |                          |                     |
| Autoencoder                                       | Undercomplete          | 99.75                  | 1.63    | 36.25   | 1.0                      | Yin et al. (2012)   |
|                                                   | Sparse                 | 99.62                  | 0.75    | 99.75   | 1.0                      |                     |
| Deep autoencoder                                  | Undercomplete          | 99.75                  | 1.75    | 98.88   | 1.0                      |                     |
|                                                   | Sparse                 | 99.5                   | 1.00    | 25.9    | 1.0                      |                     |
| Variational autoencoder                           | Undercomplete          | 99.75                  | 1.12    | 26.88   | 1.0                      | Zhang et al. (2018) |
|                                                   | Sparse                 | 99.63                  | 1.50    | 34.50   | 1.0                      |                     |
| Denoising autoencoder                             | Undercomplete          | 99.75                  | 2.25    | 25.75   | 1.0                      |                     |
|                                                   | Sparse                 | 99.75                  | 1.63    | 32.88   | 1.0                      |                     |
| Deep denoising autoencoder                        | Undercomplete          | 99.75                  | 2.63    | 26.88   | 1.0                      | Xiao et al. (2023)  |
|                                                   | Sparse                 | 99.50                  | 0.75    | 0.25    | 1.0                      |                     |
| PCA                                               |                        | 99.75                  | 1.13    | 26.63   | 1.0                      |                     |
| Subspace Aided Approach (SAP)                     | s = 13                 | 99.63                  | 6.38    | 100.0   | 1.5                      | Yin et al. (2012)   |
| Dynamic PCA (DPCA)                                | 17 components          | 99.88                  | 12.88   | 43.25   | 10.13                    |                     |
| Total Projection to Latent Structure (TPLS)       | 6 latent variables     | 99.88                  | 24.25   | 100.0   | 19.62                    |                     |
| Stacked Denoising Autoencoder with KNN (SDAE-kNN) |                        | 100.0                  | 34.9    | 56.9    | <sup>a</sup>             | Zhang et al. (2018) |
| Deep Autoencoder                                  | 5 layers, 3 input lags | 100.0                  | 3.6     | 100.0   | 2.5/1.3/3.1 <sup>b</sup> | Xiao et al. (2023)  |

<sup>a</sup> False alarm rate not reported.

<sup>b</sup> False alarm rate reported for each fault.

one is better. One of the advantages of using AUC as the comparison metric is that it inherently balances the trade-off between FDR and FAR, providing a more balanced evaluation of the models.

For Fault 5, the results for the proposed autoencoders presented more variability. Most of them had detection rates between 25.9% and 36.25%, while the sparse Autoencoder and the Undercomplete Deep Autoencoder presented really good Fault Detection Rates of 99.75% and 98.88%. The SPE Charts shown in Fig. 15 help to explain this difference. At the start of the fault, the SPE values present a large deviation from normal operation. However, when the controller starts to act, the process returns to a condition that is close to normality, with intermediate SPE values. As a result, the sparse Autoencoder was able to detect the fault for most observations, but the undercomplete Autoencoder and both Variational Autoencoders were unable to detect the fault correctly after 18 h of simulation. In addition, the SPE Charts show that, for this fault, small changes in the threshold would change the Fault Detection Rate significantly, as many of the SPE values are close to the threshold. This reveals one of the drawbacks of using a single threshold, as choosing a slightly different threshold could potentially produce large changes in Fault Detection Rates, biasing the comparison between different techniques.

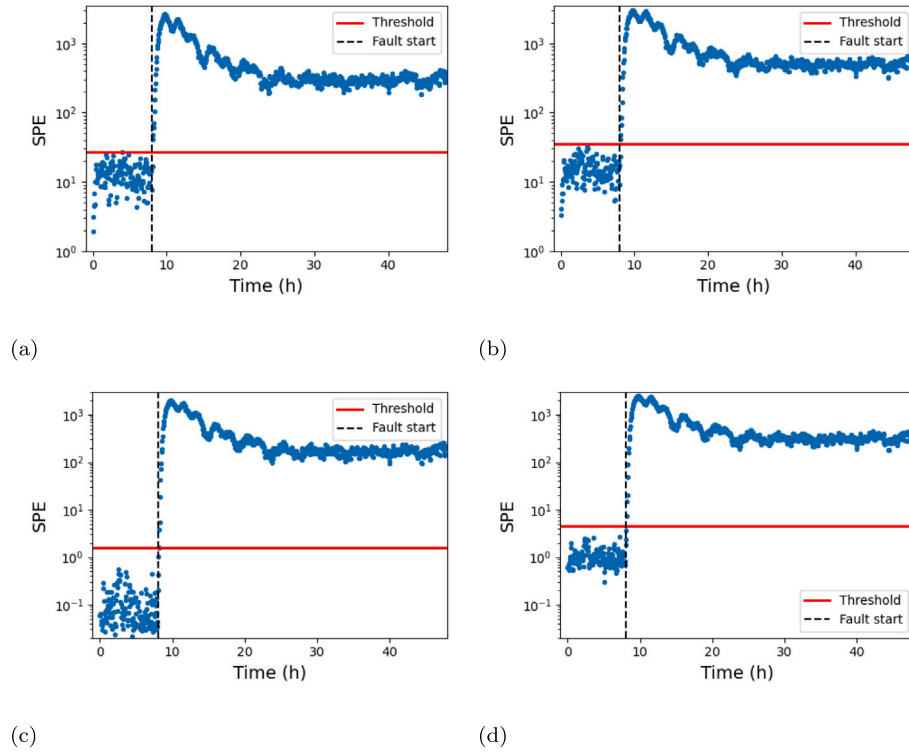
When looking at the results for Fault 5 of other techniques presented in the literature, there are two techniques (SAP and TPLS) with a 100% Fault Detection Rate and two techniques with intermediate detection: the Stacked Denoising Autoencoder with KNN with a FDR of 56.9% and

the DPCA with a FDR of 43.25%, indicating that these techniques suffer some difficulties to detect this fault.

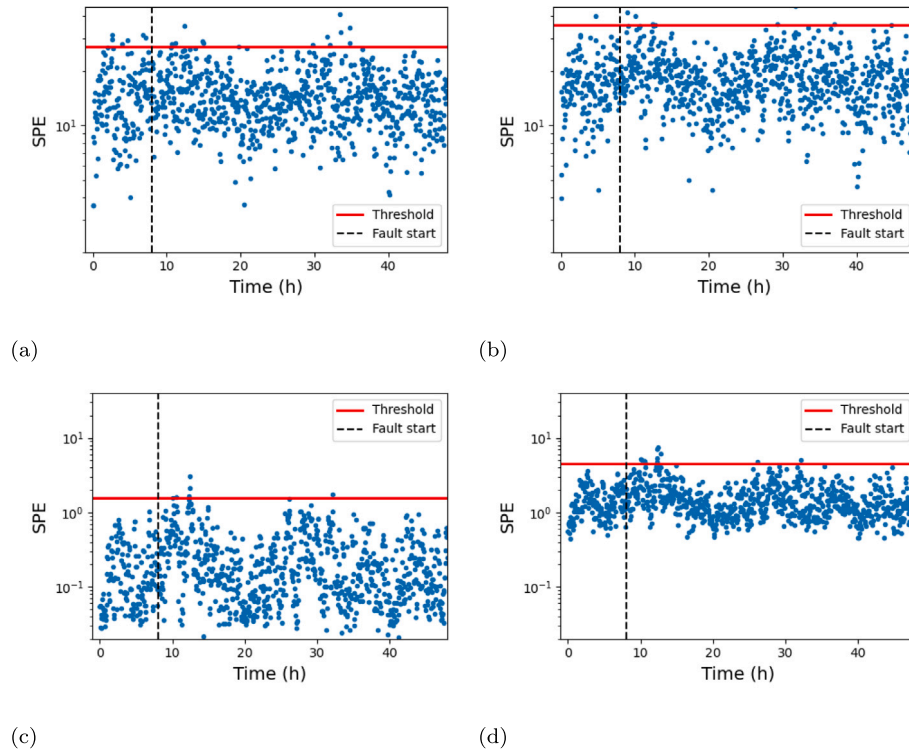
## 5. Conclusion

In the present article, the performances of different types of Autoencoders were evaluated and compared for failure detection problems using the Tennessee Eastman Process benchmark. Moreover, ROC curves and AUC were selected as performance measures to avoid the issues associated with establishing a single fixed threshold. The results were compared directly with PCA, a traditional technique widely employed in the industry. Another comparison was made against other models in the literature by setting a fixed threshold for the SPE.

All the tested autoencoders provided good performances for most faults, as their characteristic ROC curves were similar and presented higher AUC values than the PCA. Overall, the sparse VAE and sparse Deep DAE delivered the best results. The ROC curves indicated a performance crossover for these two models, as the sparse VAE had the best performance for low FAR values while the Deep DAE performed best at large FARs. Another insightful finding was that applying a sparse architecture with a large number of neurons was better than using a low code size for the denoising autoencoders and VAE, while the use of an undercomplete architecture was better for the simple AE and deep AE. In conclusion, autoencoders showed promising results for fault detection in the TEP benchmark, particularly the sparse VAE and Deep DAE architectures.



**Fig. 13.** SPE Charts - fault 1: (a) undercomplete AE; (b) undercomplete VAE; (c) sparse AE; (d) sparse VAE.



**Fig. 14.** SPE Charts - fault 3: (a) Undercomplete AE; (b) undercomplete VAE; (c) sparse AE; (d) sparse VAE.

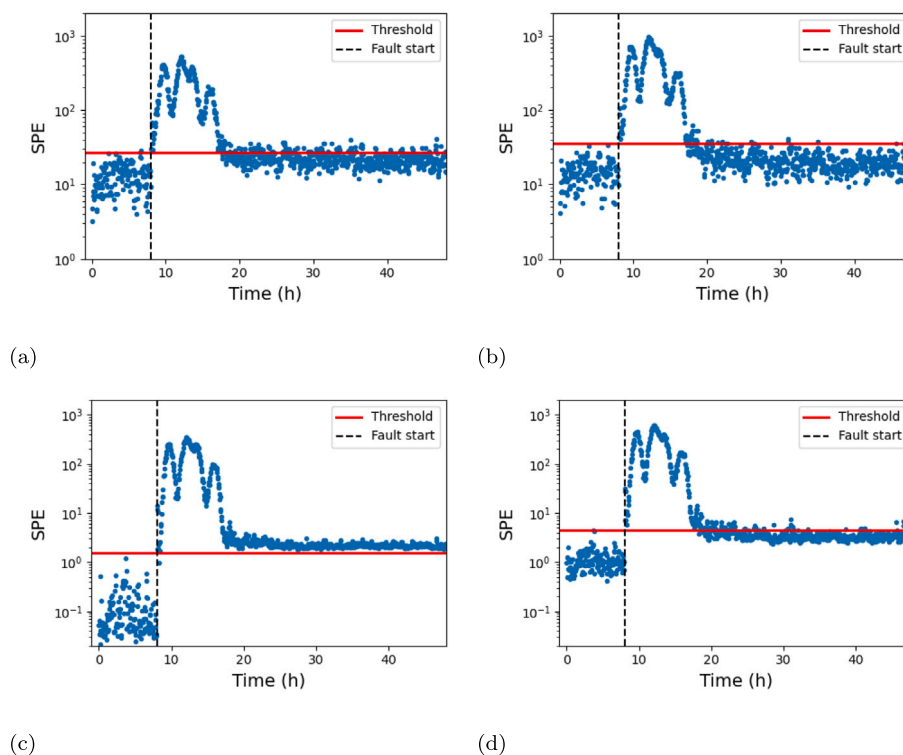


Fig. 15. SPE Charts - fault 5: (a) undercomplete AE; (b) undercomplete VAE; (c) sparse AE; (d) sparse VAE.

#### CRediT authorship contribution statement

**Deris Eduardo Spina:** Conceptualization, Formal analysis, Methodology, Software, Writing – original draft, Writing – review & editing. **Luiz Felipe de O. Campos:** Conceptualization, Formal analysis, Methodology, Writing – original draft, Writing – review & editing. **Wallthynay F. de Arruda:** Conceptualization, Writing – original draft, Methodology. **Afrânio Melo:** Conceptualization, Methodology, Writing – original draft, Writing – review & editing. **Marcelo F. de S. Alves:** Formal analysis, Methodology, Writing – original draft, Writing – review & editing. **Gildeir Lima Rabello:** Formal analysis, Methodology, Writing – original draft, Writing – review & editing. **Thiago K. Anzai:** Funding acquisition, Project administration, Supervision, Writing – review & editing. **José Carlos Pinto:** Conceptualization, Funding acquisition, Project administration, Supervision, Writing – review & editing.

#### Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Deris Eduardo Spina, Luiz Felipe de O. Campos, Wallthynay F. de Arruda, Marcelo F. de S. Alves report financial support was provided by Petrobras. Gildeir Lima Rabello reports financial support was provided by Coordenação de aperfeiçoamento de Pessoal de Nível Superior. Jose Carlos Costa da Silva Pinto reports financial support was provided by Conselho Nacional de Desenvolvimento Científico e Tecnológico. Afrânio Melo and Thiago K. Anzai report administrative support was provided by Petrobras.

#### Acknowledgments

We thank CNPq (Conselho Nacional de Desenvolvimento Científico e Tecnológico, Brazil), FAPERJ (Fundação Carlos Chagas Filho de Amparo à Pesquisa do Estado do Rio de Janeiro, Brazil) and Petrobras SA for providing technical support and scholarships.

#### Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.dche.2024.100162>.

#### References

- Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M., 2019. Optuna: A next-generation hyperparameter optimization framework. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. pp. 2623–2631.
- Alauddin, M., Khan, F., Imtiaz, S., Ahmed, S., 2018. A bibliometric review and analysis of data-driven fault detection and diagnosis methods for process systems. *Ind. Eng. Chem. Res.* 57, 10719–10735. <http://dx.doi.org/10.1021/acs.iecr.8b00936>.
- Arias Chao, M., Adey, B.T., Fink, O., 2021. Implicit supervision for fault detection and segmentation of emerging fault types with deep variational autoencoders. *Neurocomputing* 454, 324–338. <http://dx.doi.org/10.1016/j.neucom.2021.04.122>, URL <https://www.sciencedirect.com/science/article/pii/S0925231221007001>.
- Ávila Okada, K.F., Silva de Moraes, A., Oliveira-Lopes, L.C., Ribeiro, L., 2021. A survey on fault detection and diagnosis methods. In: *2021 14th IEEE International Conference on Industry Applications. INDUSCON*, pp. 1422–1429. <http://dx.doi.org/10.1109/INDUSCON51756.2021.9529495>.
- Bank, D., Koenigstein, N., Giryès, R., 2020. Autoencoders. *CoRR* abs/2003.05991. [arXiv:2003.05991](https://arxiv.org/abs/2003.05991). URL <https://arxiv.org/abs/2003.05991>.
- Baughman, D.R., Liu, Y.A.Y.A., 1995. *Neural Networks in Bioprocessing and Chemical Engineering*. Academic Press, p. 488.
- Bengio, Y., Courville, A., Vincent, P., 2013. Representation learning: A review and new perspectives. *IEEE Trans. Pattern Anal. Mach. Intell.* 35 (8), 1798–1828. <http://dx.doi.org/10.1109/TPAMI.2013.50>.
- Carvalho, T.P., Soares, F.A., Vita, R., da P. Francisco, R., Basto, J.P., Alcalá, S.G., 2019. A systematic literature review of machine learning methods applied to predictive maintenance. *Comput. Ind. Eng.* 137, 106024. <http://dx.doi.org/10.1016/j.cie.2019.106024>.
- Chadha, G.S., Rabbani, A., Schwung, A., 2019. Comparison of semi-supervised deep neural networks for anomaly detection in industrial processes. In: *2019 IEEE 17th International Conference on Industrial Informatics. INDIN*, Vol. 1, pp. 214–219. <http://dx.doi.org/10.1109/INDIN41052.2019.8972172>.
- Chen, J., Li, J., Chen, W., Wang, Y., Jiang, T., 2020. Anomaly detection for wind turbines based on the reconstruction of condition parameters using stacked denoising autoencoders. *Renew. Energy* 147, 1469–1480. <http://dx.doi.org/10.1016/j.renene.2019.09.041>, URL <https://www.sciencedirect.com/science/article/pii/S0960148119313710>.



- Chiang, L.H., Russell, E.L., Braatz, R.D., 2003. Open-Loop and the Closed-Loop Simulations for the Tennessee Eastman Process. TEP, University of Illinois, URL [http://web.mit.edu/braatzgroup/TE\\_process.zip](http://web.mit.edu/braatzgroup/TE_process.zip).
- Chollet, F., Allaire, J.J., 2018. Deep Learning with R, first ed. Manning Publications Co., USA.
- Divya, D., Marath, B., Kumar, M.B.S., 2023. Review of fault detection techniques for predictive maintenance. <http://dx.doi.org/10.1108/JQME-10-2020-0107>.
- Downs, J., Vogel, E., 1993. A plant-wide industrial process control problem. *Comput. Chem. Eng.* 17 (3), 245–255. [http://dx.doi.org/10.1016/0098-1354\(93\)80018-1](http://dx.doi.org/10.1016/0098-1354(93)80018-1).
- Ganesan, M., Lavanya, R., 2023. Simultaneous fault detection in satellite power systems using deep autoencoders and classifier chain. *Telecommun. Syst.* <http://dx.doi.org/10.1007/s11235-023-00998-3>.
- Géron, A., 2019. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, second ed. O'Reilly Media, Inc..
- Hastie, T., 2020. Ridge regularization: An essential concept in data science. *Technometrics* 62 (4), 426–433. <http://dx.doi.org/10.1080/00401706.2020.1791959>, [arXiv:https://doi.org/10.1080/00401706.2020.1791959](https://arxiv.org/abs/https://doi.org/10.1080/00401706.2020.1791959).
- Hinton, G.E., Srivastava, N., Krizhevsky, A., Sutskever, I., Salakhutdinov, R., 2012. Improving neural networks by preventing co-adaptation of feature detectors. *CoRR* abs/1207.0580. [arXiv:1207.0580](https://arxiv.org/abs/1207.0580). URL <http://arxiv.org/abs/1207.0580>.
- Hozdici, E., 2015. Smart factory for industry 4.0: A review. *Int. J. Mod. Manuf. Technol.* VII, 2067–2604.
- Humaira, H., Rasyidah, R., 2020. Determining the appropriate cluster number using elbow method for K-means algorithm. In: Proceedings of the 2nd Workshop on Multidisciplinary and Applications (WMA) 2018, 24–25 January 2018, Padang, Indonesia. EAI, <http://dx.doi.org/10.4108/eai.24-1-2018.2292388>.
- Jakubowski, J., Stanisz, P., Bobek, S., Nalepa, G.J., 2022. Anomaly detection in asset degradation process using variational autoencoder and explanations. *Sensors* 22 (1), <http://dx.doi.org/10.3390/s22010291>, URL <https://www.mdpi.com/1424-8220/22/1/291>.
- Jang, K., Hong, S., Kim, M., Na, J., Moon, I., 2022. Adversarial autoencoder based feature learning for fault detection in industrial processes. *IEEE Trans. Ind. Inform.* 18, 827–834. <http://dx.doi.org/10.1109/TII.2021.3078414>.
- Kathlyn, T.K., Zabiri, H., Aldrich, C., Liu, X., Mohd Amiruddin, A.A.A., 2023. Fault detection and identification in an acid gas removal unit using deep autoencoders. *ACS Omega* 8 (22), 19273–19286. <https://dx.doi.org/10.1021/acsomega.2c08109>.
- Kingma, D.P., Ba, J., 2014. Adam: A method for stochastic optimization. *CoRR* URL <https://arxiv.org/abs/1412.6980>.
- Kingma, D.P., Welling, M., 2019. An introduction to variational autoencoders. *Found. Trends® Mach. Learn.* <http://dx.doi.org/10.1561/22000000056>, URL <http://arxiv.org/abs/1906.02691>.
- Lau, C., Ghosh, K., Hussain, M.A., Hassan, C.C., 2013. Fault diagnosis of Tennessee eastman process with multi-scale PCA and ANFIS. *Chemometr. Intell. Lab. Syst.* 120, 1–14.
- Liu, C., Wang, Y., Wang, K., Yuan, X., 2021. Deep learning with nonlocal and local structure preserving stacked autoencoder for soft sensor in industrial processes. *Eng. Appl. Artif. Intell.* 104, <http://dx.doi.org/10.1016/j.engappai.2021.104341>.
- Lomov, I., Lyubimov, M., Makarov, I., Zhukov, L.E., 2021. Fault detection in Tennessee eastman process with temporal deep learning models. *J. Ind. Inf. Integr.* 23, <http://dx.doi.org/10.1016/j.jii.2021.100216>.
- Lyman, P., Georgakis, C., 1995. Plant-wide control of the Tennessee Eastman problem. *Comput. Chem. Eng.* 19 (3), 321–331. [http://dx.doi.org/10.1016/0098-1354\(94\)00057-U](http://dx.doi.org/10.1016/0098-1354(94)00057-U), URL <https://linkinghub.elsevier.com/retrieve/pii/S009813549400057U>.
- Mansouri, M., Harkat, M.-F., Nounou, H.N., Nounou, M.N., 2020. Data-Driven and Model-Based Methods for Fault Detection and Diagnosis. Elsevier, <http://dx.doi.org/10.1016/c2018-0-04213-9>.
- Melo, A., Câmara, M.M., Clavijo, N., Pinto, J.C., 2022. Open benchmarks for assessment of process monitoring and fault diagnosis techniques: A review and critical analysis. *Comput. Chem. Eng.* 165, 107964. <http://dx.doi.org/10.1016/j.compchemeng.2022.107964>, URL <https://www.sciencedirect.com/science/article/pii/S0098135422003003>.
- Melo, A., Câmara, M.M., Pinto, J.C., 2024. Data-driven process monitoring and fault diagnosis: A comprehensive survey. *Processes* 12 (2), <http://dx.doi.org/10.3390/pr12020251>, URL <https://www.mdpi.com/2227-9717/12/2/251>.
- Miljković, D., 2011. Fault detection methods: A literature survey. In: 2011 Proceedings of the 34th International Convention MIPRO. pp. 750–755.
- Preferred Networks Inc., 2023. Optuna: Optimize Your Optimization. URL <https://optuna.org/>.
- Qian, J., Song, Z., Yao, Y., Zhu, Z., Zhang, X., 2022a. A review on autoencoder based representation learning for fault detection and diagnosis in industrial processes. *Chemometr. Intell. Lab. Syst.* 231, 104711.
- Qian, J., Song, Z., Yao, Y., Zhu, Z., Zhang, X., 2022b. A review on autoencoder based representation learning for fault detection and diagnosis in industrial processes. *Chemometr. Intell. Lab. Syst.* 231, 104711. <http://dx.doi.org/10.1016/j.chemolab.2022.104711>, URL <https://www.sciencedirect.com/science/article/pii/S0169743922002222>.
- Ran, Y., Zhou, X., Lin, P., Wen, Y., Deng, R., 2019. A survey of predictive maintenance: Systems, purposes and approaches. *IEEE Commun. Surv. Tutor.* 1–36, URL <http://arxiv.org/abs/1912.07383>.
- Rieth, C.A., Amsel, B.D., Tran, R., Cook, M.B., 2018. Issues and advances in anomaly detection evaluation for joint human-automated systems. In: Advances in Human Factors in Robots and Unmanned Systems. Vol. 595, Springer Verlag, pp. 52–63. [http://dx.doi.org/10.1007/978-3-319-60384-1\\_6](http://dx.doi.org/10.1007/978-3-319-60384-1_6).
- Tang, P., Peng, K., Dong, J., 2021. Nonlinear quality-related fault detection using combined deep variational information bottleneck and variational autoencoder. *ISA Trans.* 114, 444–454. <http://dx.doi.org/10.1016/j.isatra.2021.01.002>, URL <https://www.sciencedirect.com/science/article/pii/S0019057821000045>.
- Thorndike, R.L., 1953. Who belongs in the family? *Psychometrika* 18 (4), 267–276.
- Vidaurre, D., Bielza, C., Larrañaga, P., 2013. A survey of L1 regression. *Internat. Statist. Rev.* 81 (3), 361–387. <http://dx.doi.org/10.1111/insr.12023>, [arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1111/insr.12023](https://onlinelibrary.wiley.com/doi/pdf/10.1111/insr.12023). URL <https://onlinelibrary.wiley.com/doi/abs/10.1111/insr.12023>.
- Vincent, P., Larochelle, H., Lajoie, I., Bengio, Y., Manzagol, P.-A., 2010. Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion. *J. Mach. Learn. Res.* 11, 3371–3408.
- Wager, S., Wang, S., Liang, P.S., 2013. Dropout training as adaptive regularization. *Adv. Neural Inf. Process. Syst. (NIPS)* 26, [arXiv:1307.1493](https://arxiv.org/abs/1307.1493). URL <https://arxiv.org/abs/1307.1493>.
- Xavier, G.M., de Seixas, J.M., 2018. Fault detection and diagnosis in a chemical process using long short-term memory recurrent neural network. In: 2018 International Joint Conference on Neural Networks. IJCNN, IEEE, Rio de Janeiro, pp. 1–8. <http://dx.doi.org/10.1109/IJCNN.2018.8489385>, URL <https://ieeexplore.ieee.org/document/8489385/>.
- Xiao, Z., Kordon, A., Sen, S., 2023. Fault detection and diagnosis in Tennessee eastman process with deep autoencoder. *Annu. Conf. PHM Soc.* 15, <http://dx.doi.org/10.36001/phmconf.2023.v15i1.3578>.
- Yang, Z., Xu, B., Luo, W., Chen, F., 2022. Autoencoder-based representation learning and its application in intelligent fault diagnosis: A review. *Meas.: J. Int. Meas. Confed.* 189, <http://dx.doi.org/10.1016/j.measurement.2021.110460>.
- Yin, S., Ding, S.X., Haghani, A., Hao, H., Zhang, P., 2012. A comparison study of basic data-driven fault diagnosis and process monitoring methods on the benchmark Tennessee eastman process. *J. Process Control* 22, 1567–1581. <http://dx.doi.org/10.1016/j.jprocont.2012.06.009>.
- Yu, J., Zheng, X., Wang, S., 2019. A deep autoencoder feature learning method for process pattern recognition. *J. Process Control* 79, 1–15. <http://dx.doi.org/10.1016/j.jprocont.2019.05.002>.
- Zhang, C., Hu, D., Yang, T., 2022. Anomaly detection and diagnosis for wind turbines using long short-term memory-based stacked denoising autoencoders and XGBoost. *Reliab. Eng. Syst. Saf.* 222, 108445. <http://dx.doi.org/10.1016/j.res.2022.108445>, URL <https://www.sciencedirect.com/science/article/pii/S0951832022001107>.
- Zhang, Z., Jiang, T., Li, S., Yang, Y., 2018. Automated feature learning for nonlinear process monitoring—An approach using stacked denoising autoencoder and k-nearest neighbor rule. *J. Process Control* 64, 49–61.
- Zhang, J., Ren, H., Ren, H., Chai, Y., Liu, Z., Liang, X., 2023. Comprehensive review of safety studies in process industrial systems: Concepts, progress, and main research topics. *Processes* 11 (8), <http://dx.doi.org/10.3390/pr11082454>, URL <https://www.mdpi.com/2227-9717/11/8/2454>.
- Zhu, J., Jiang, M., Liu, Z., 2022. Fault detection and diagnosis in industrial processes with variational autoencoder: A comprehensive study. *Sensors* 22 (1), <http://dx.doi.org/10.3390/s22010227>, URL <https://www.mdpi.com/1424-8220/22/1/227>.
- Zonta, T., da Costa, C.A., da Rosa Righi, R., de Lima, M.J., da Trindade, E.S., Li, G.P., 2020. Predictive maintenance in the industry 4.0: A systematic literature review. *Comput. Ind. Eng.* 150, <http://dx.doi.org/10.1016/j.cie.2020.106889>.

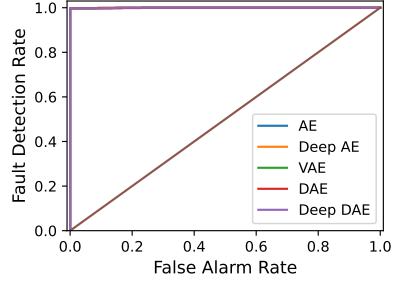
# Comparison of Autoencoder Architectures for Fault Detection in Industrial Processes

## Supplementary Material

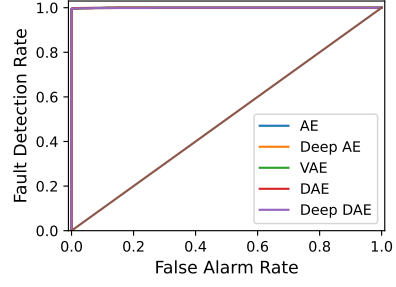
### Contents

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>1</b> | <b>ROC Curves for TEP Faults</b>                             | <b>2</b>  |
| <b>2</b> | <b>SPE Charts</b>                                            | <b>10</b> |
| 2.1      | Sparse Autoencoders . . . . .                                | 10        |
| 2.2      | Undercomplete Autoencoders . . . . .                         | 31        |
| <b>3</b> | <b>Fault Detection Rate for Fixed Threshold Autoencoders</b> | <b>52</b> |

# 1 ROC Curves for TEP Faults

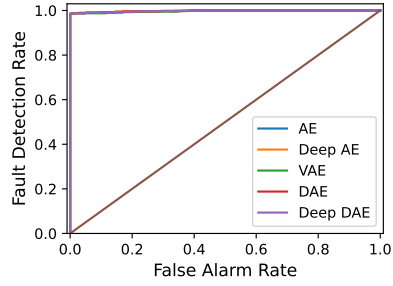


(a)

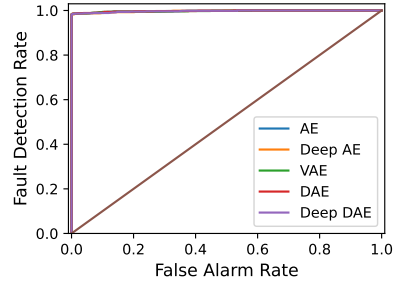


(b)

Figure S1: ROC curves for fault 1 obtained from (a) undercomplete and (b) sparse autoencoders

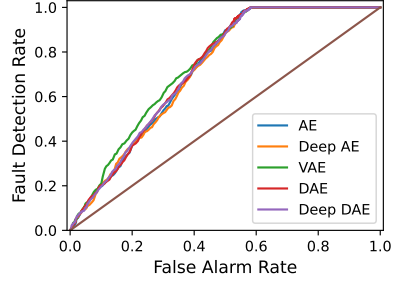


(a)

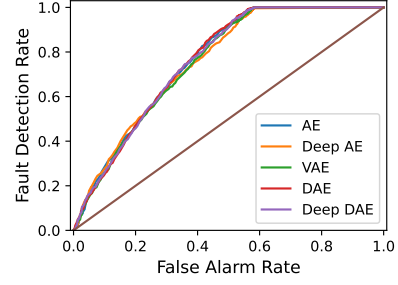


(b)

Figure S2: ROC curves for fault 2 obtained from (a) undercomplete and (b) sparse autoencoders

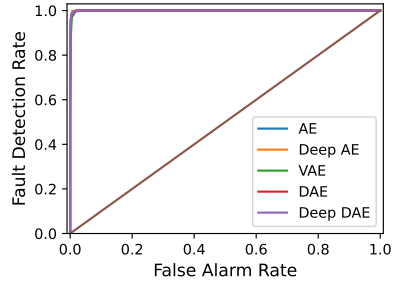


(a)

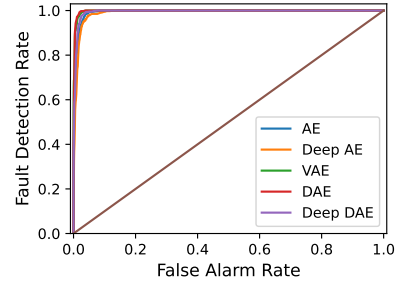


(b)

Figure S3: ROC curves for fault 3 obtained from (a) undercomplete and (b) sparse autoencoders

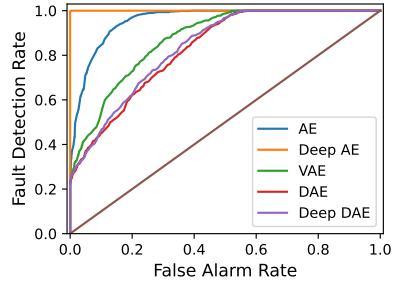


(a)

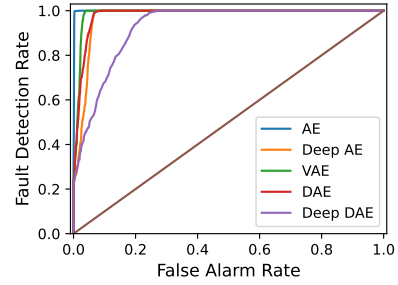


(b)

Figure S4: ROC curves for fault 4 obtained from (a) undercomplete and (b) sparse autoencoders

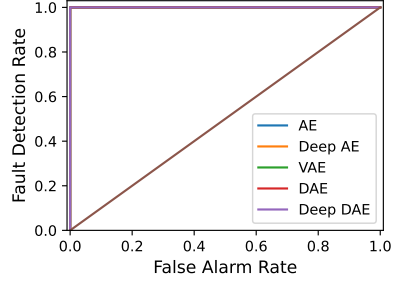


(a)

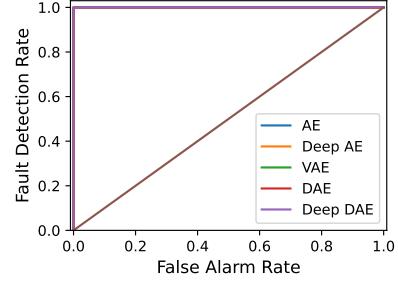


(b)

Figure S5: ROC curves for fault 5 obtained from (a) undercomplete and (b) sparse autoencoders

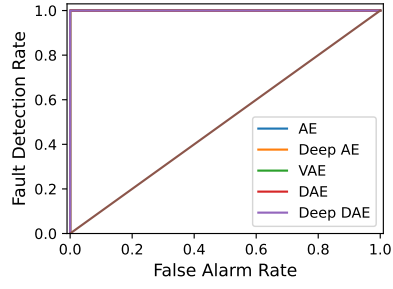


(a)

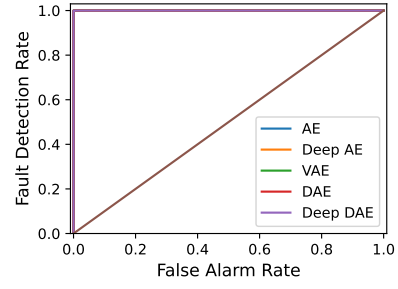


(b)

Figure S6: ROC curves for fault 6 obtained from (a) undercomplete and (b) sparse autoencoders

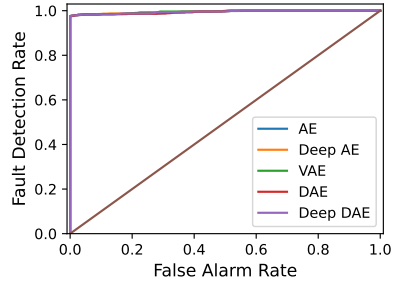


(a)

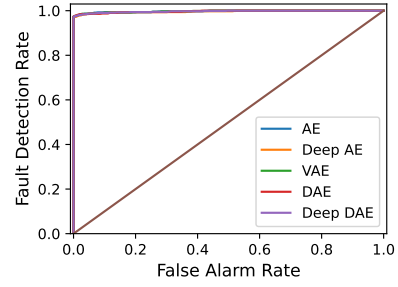


(b)

Figure S7: ROC curves for fault 7 obtained from (a) undercomplete and (b) sparse autoencoders



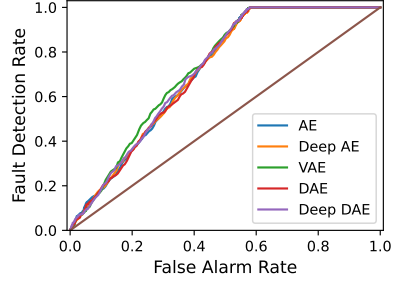
(a)



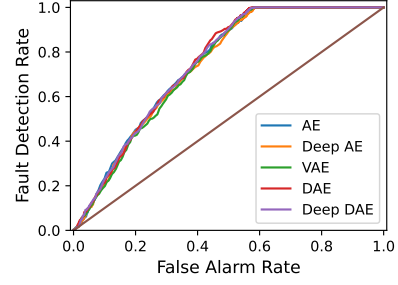
(b)

Figure S8: ROC curves for fault 8 obtained from (a) undercomplete and (b) sparse autoencoders



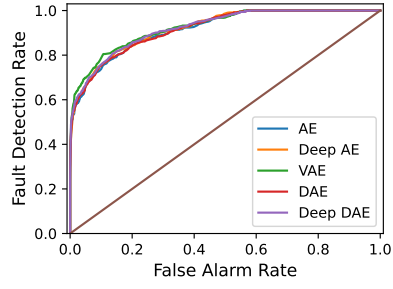


(a)

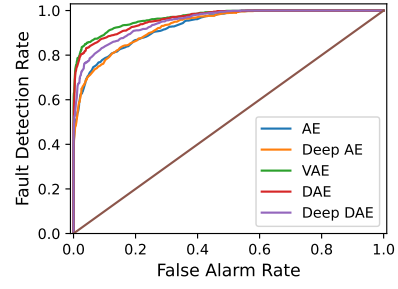


(b)

Figure S9: ROC curves for fault 9 obtained from (a) undercomplete and (b) sparse autoencoders

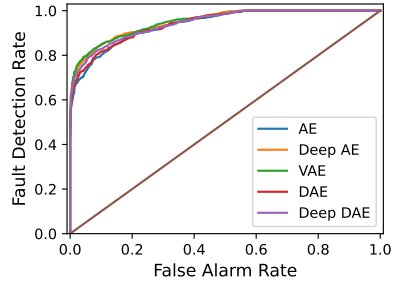


(a)

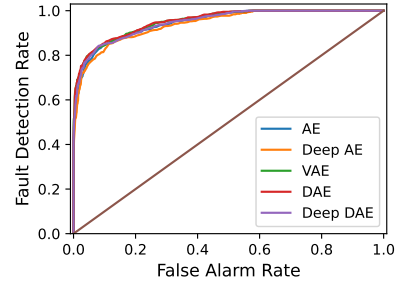


(b)

Figure S10: ROC curves for fault 10 obtained from (a) undercomplete and (b) sparse autoencoders

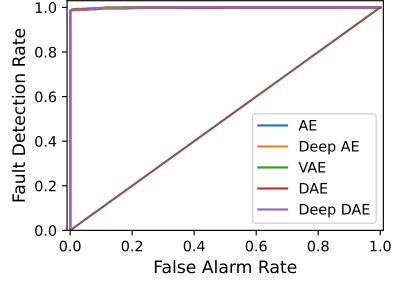


(a)

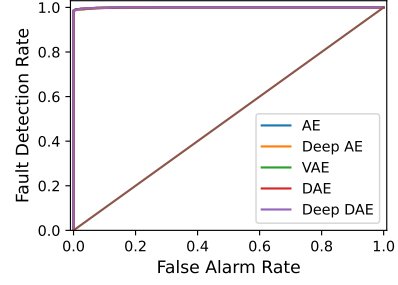


(b)

Figure S11: ROC curves for fault 11 obtained from (a) undercomplete and (b) sparse autoencoders

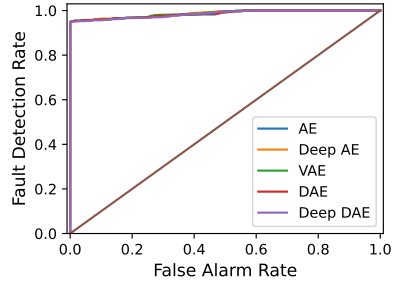


(a)

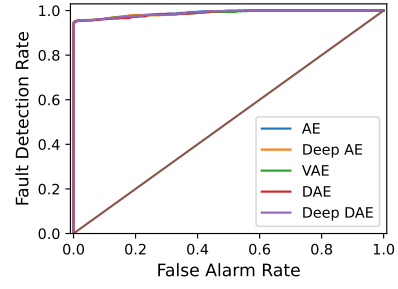


(b)

Figure S12: ROC curves for fault 12 obtained from (a) undercomplete and (b) sparse autoencoders

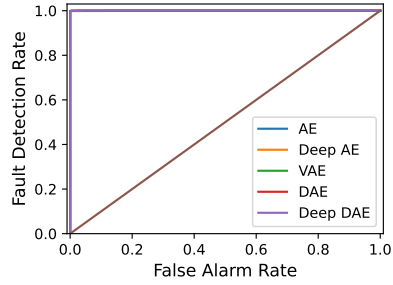


(a)

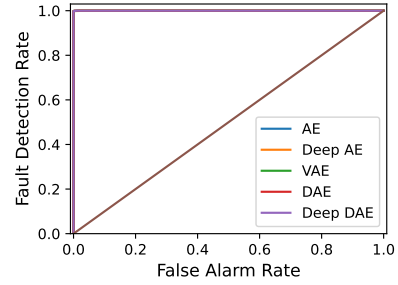


(b)

Figure S13: ROC curves for fault 13 obtained from (a) undercomplete and (b) sparse autoencoders

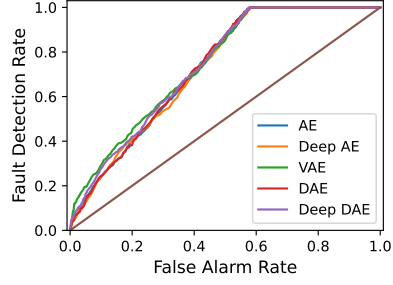


(a)

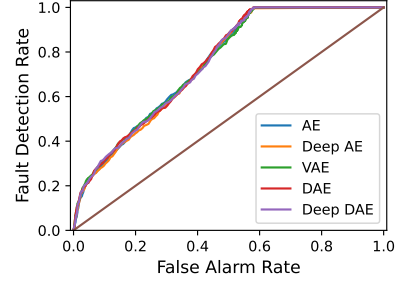


(b)

Figure S14: ROC curves for fault 14 obtained from (a) undercomplete and (b) sparse autoencoders

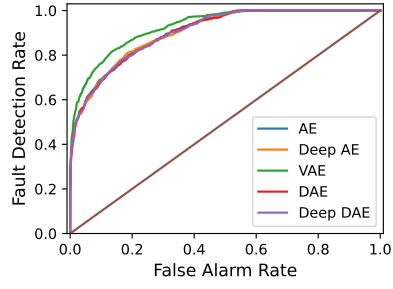


(a)

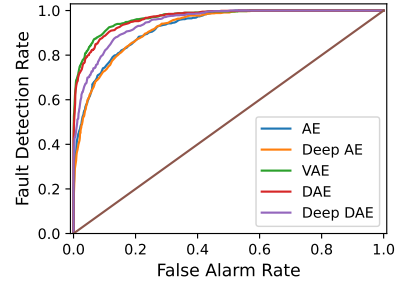


(b)

Figure S15: ROC curves for fault 15 obtained from (a) undercomplete and (b) sparse autoencoders

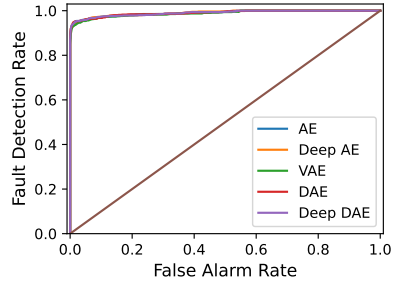


(a)

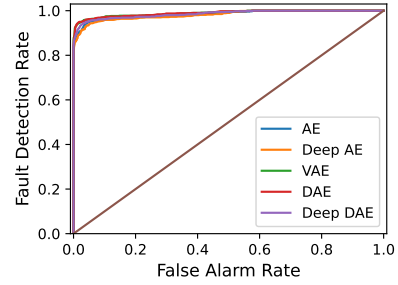


(b)

Figure S16: ROC curves for fault 16 obtained from (a) undercomplete and (b) sparse autoencoders

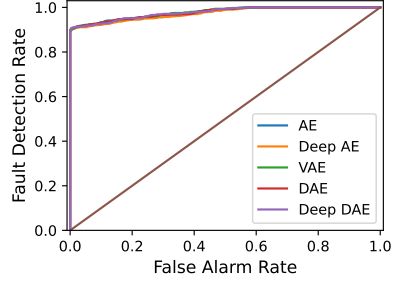


(a)

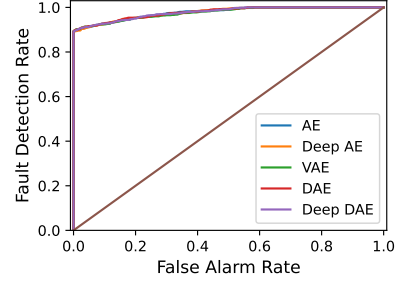


(b)

Figure S17: ROC curves for fault 17 obtained from (a) undercomplete and (b) sparse autoencoders

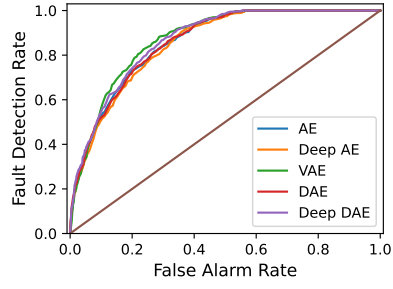


(a)

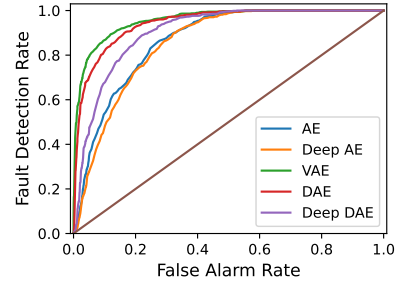


(b)

Figure S18: ROC curves for fault 18 obtained from (a) undercomplete and (b) sparse autoencoders

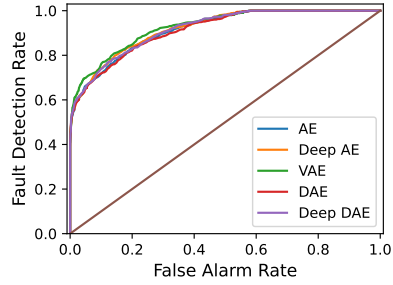


(a)

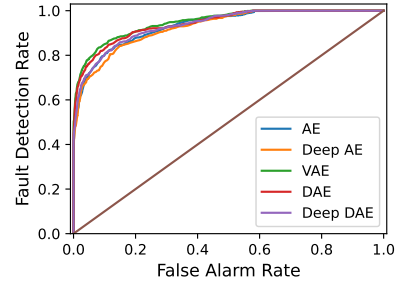


(b)

Figure S19: ROC curves for fault 19 obtained from (a) undercomplete and (b) sparse autoencoders

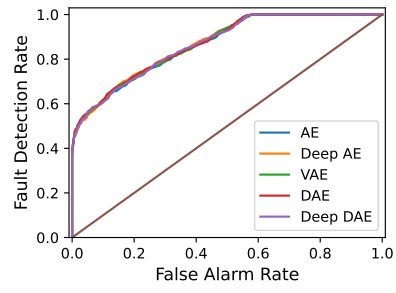


(a)

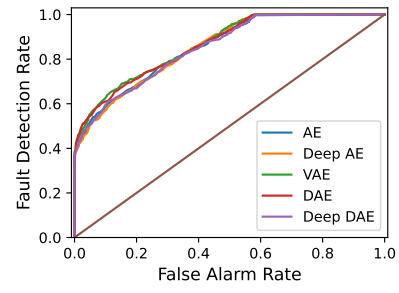


(b)

Figure S20: ROC curves for fault 20 obtained from (a) undercomplete and (b) sparse autoencoders



(a)



(b)

Figure S21: ROC curves for fault 21 obtained from (a) undercomplete and (b) sparse autoencoders

## 2 SPE Charts

### 2.1 Sparse Autoencoders

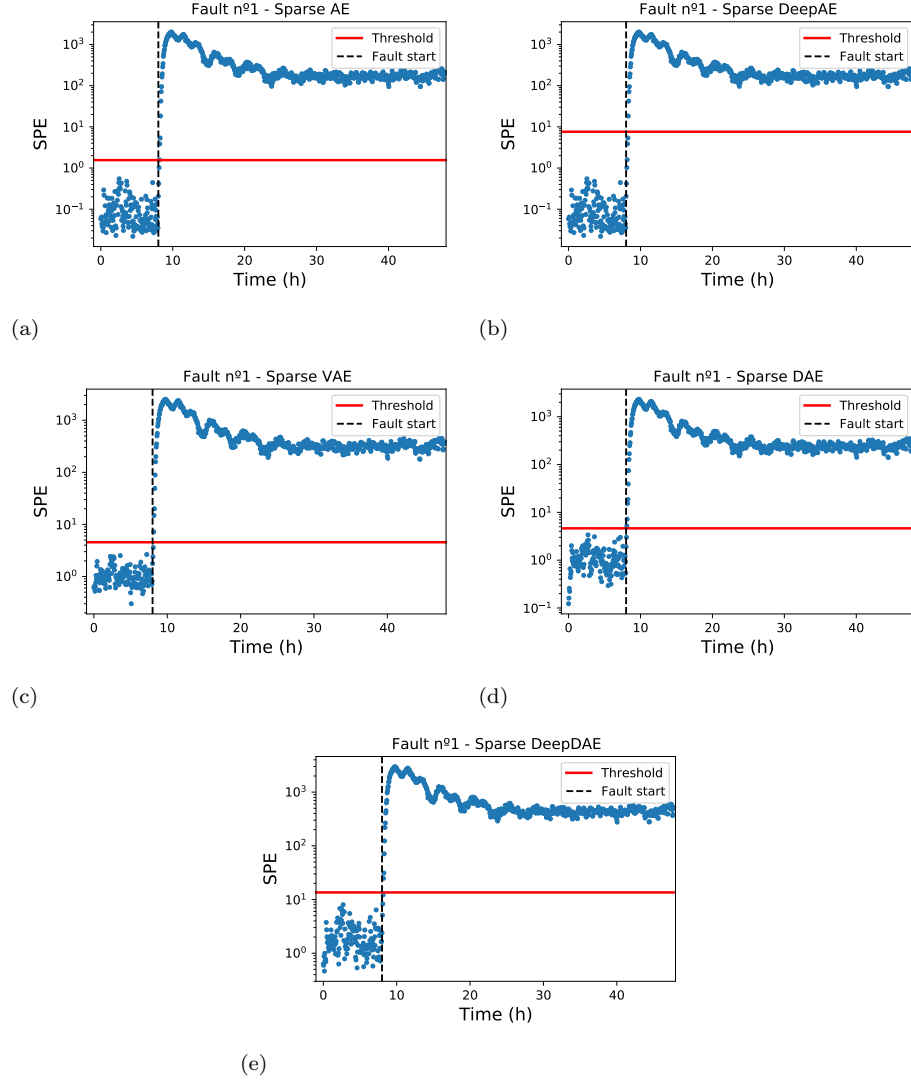


Figure S22: SPE Charts for sparse Autoencoders - fault 1: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder



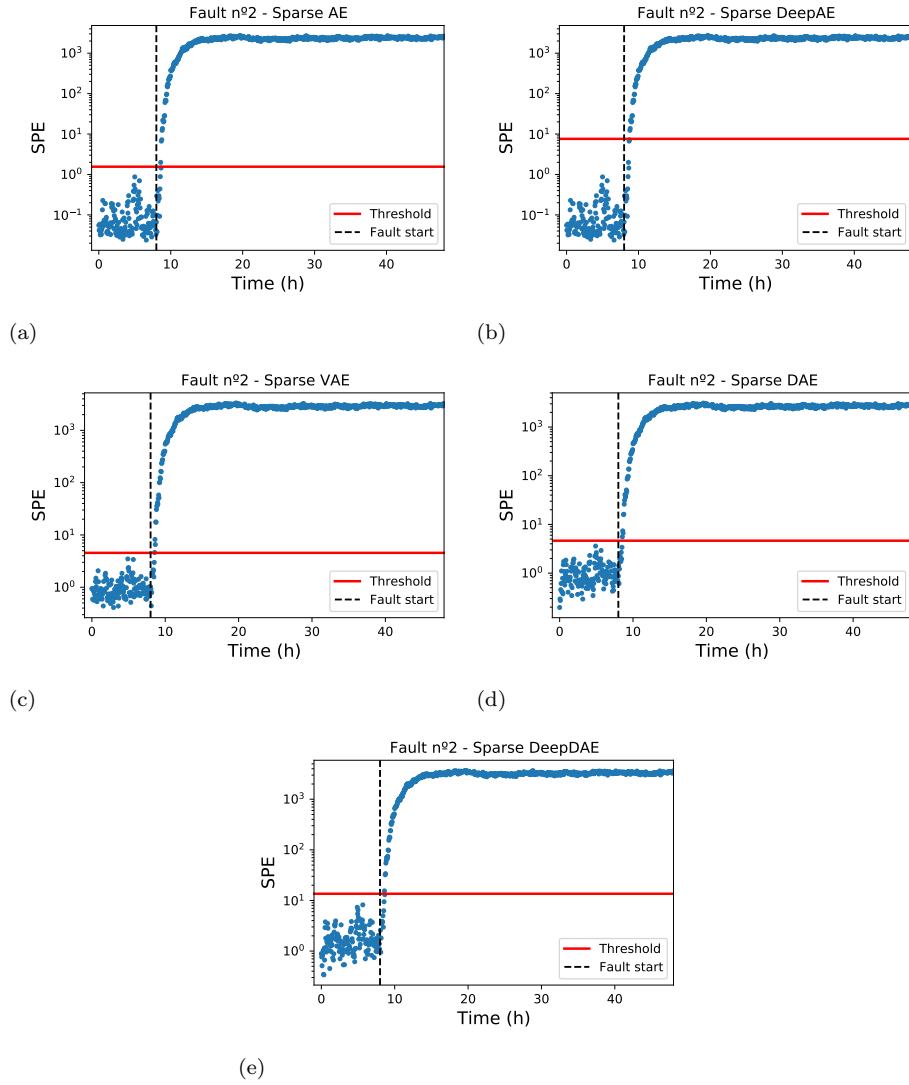


Figure S23: SPE Charts for sparse Autoencoders - fault 2: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

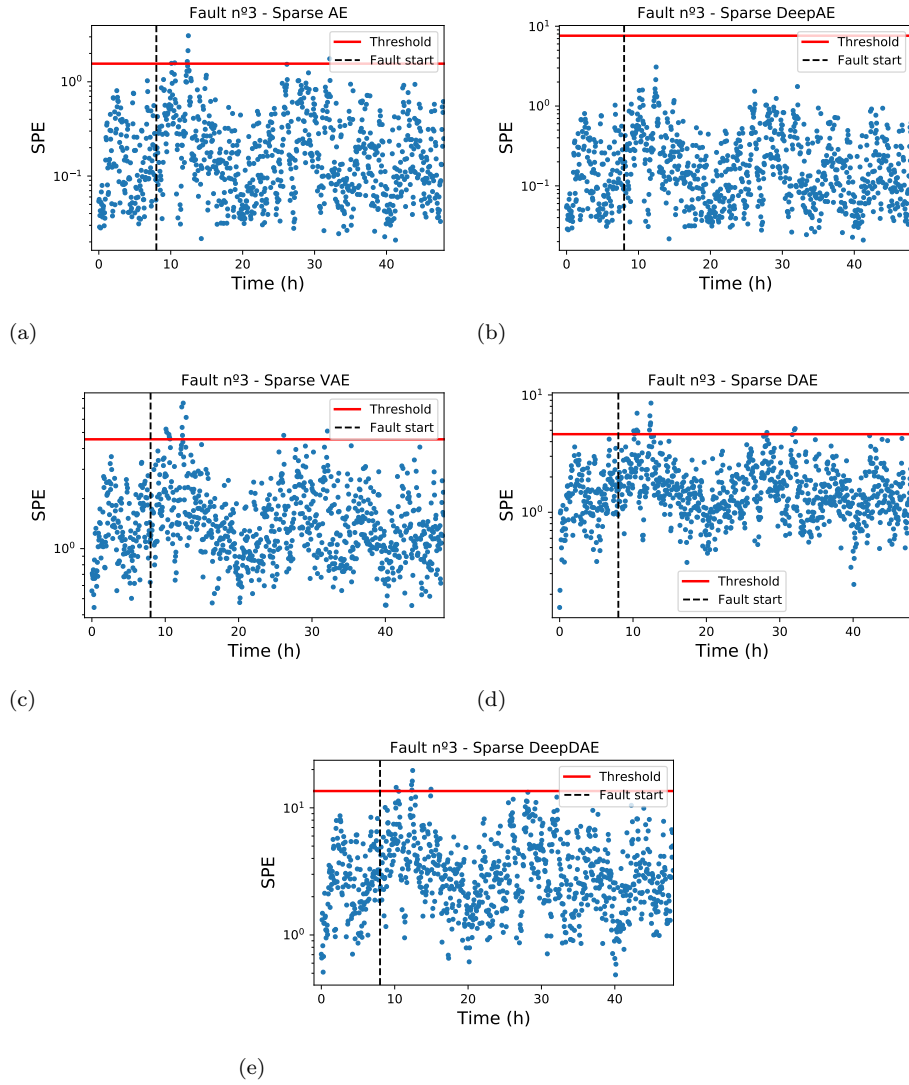


Figure S24: SPE Charts for sparse Autoencoders - fault 3: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

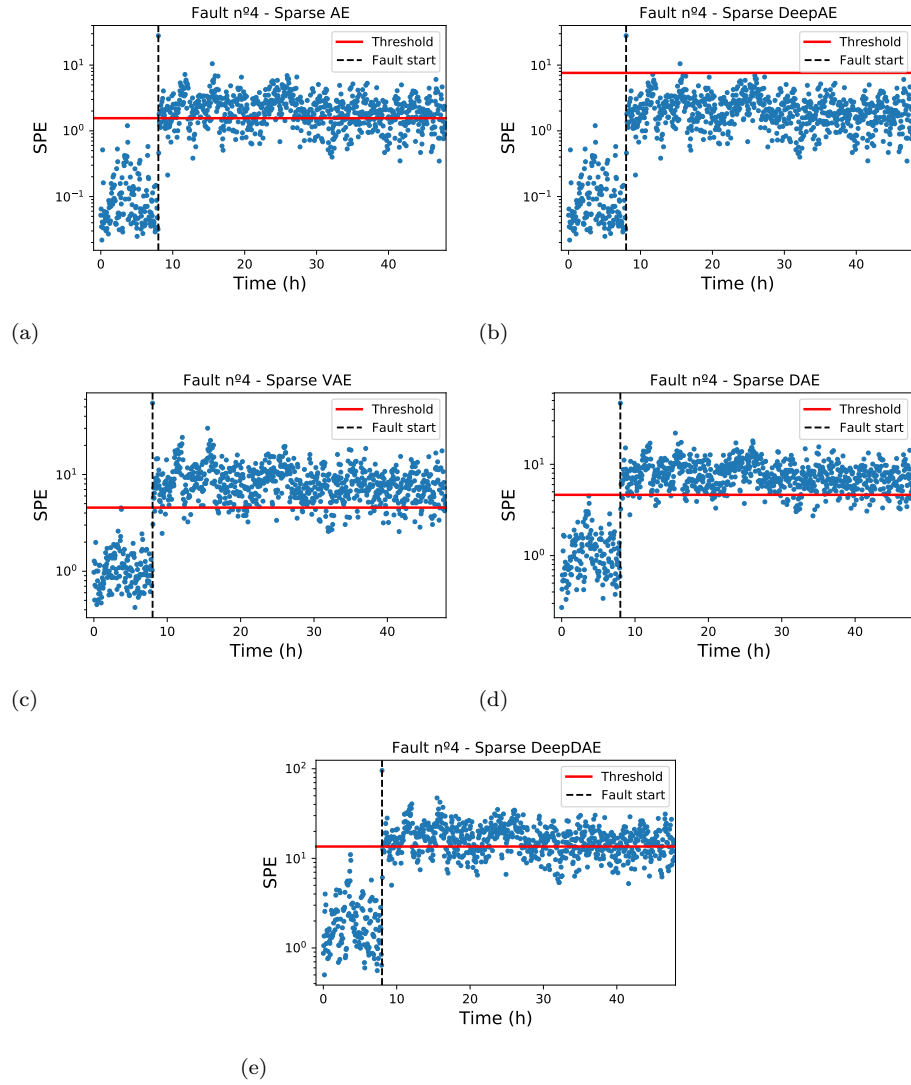


Figure S25: SPE Charts for sparse Autoencoders - fault 4: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

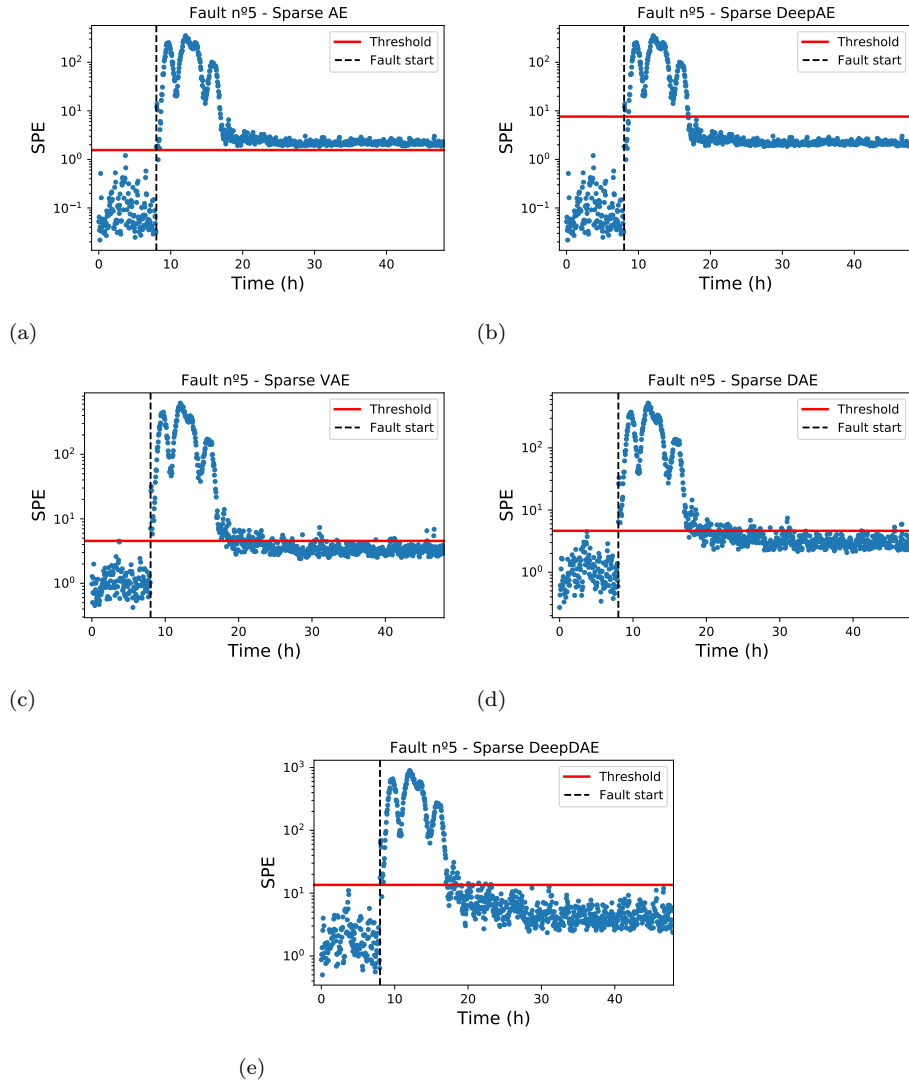


Figure S26: SPE Charts for sparse Autoencoders - fault 5: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

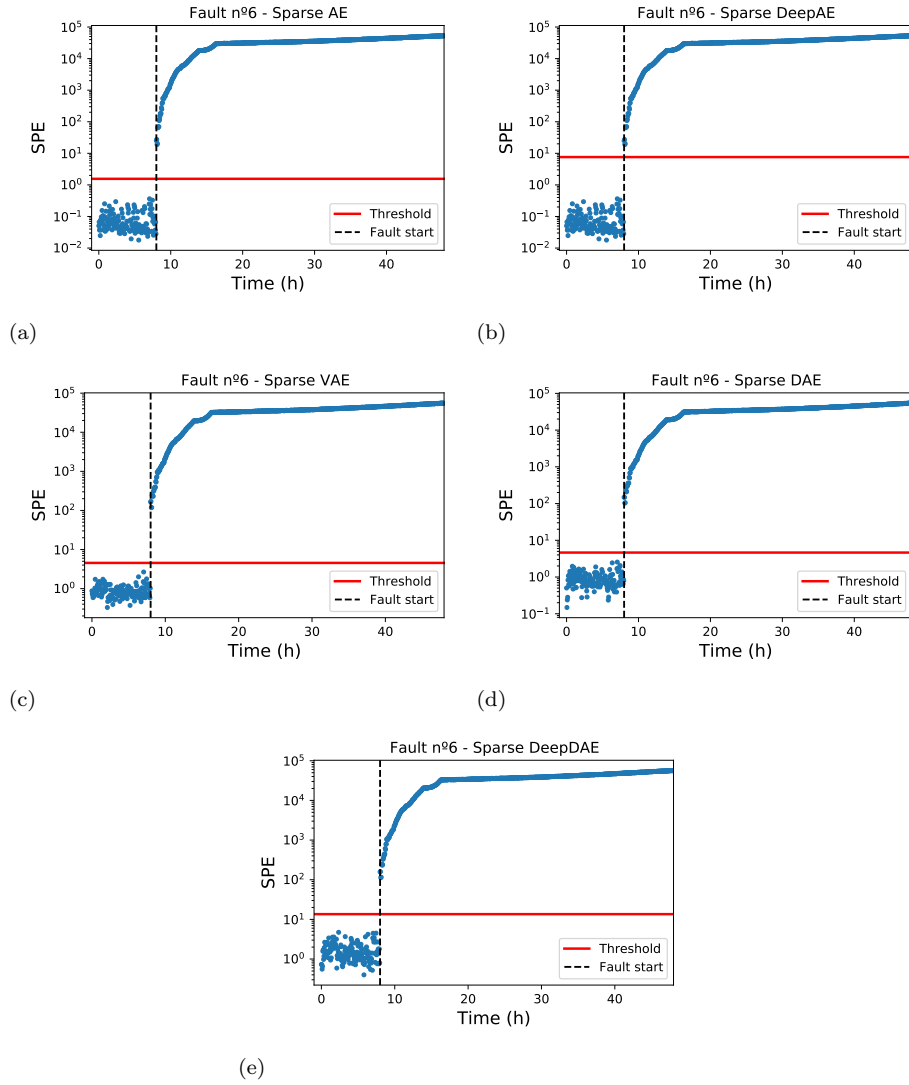


Figure S27: SPE Charts for sparse Autoencoders - fault 6: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

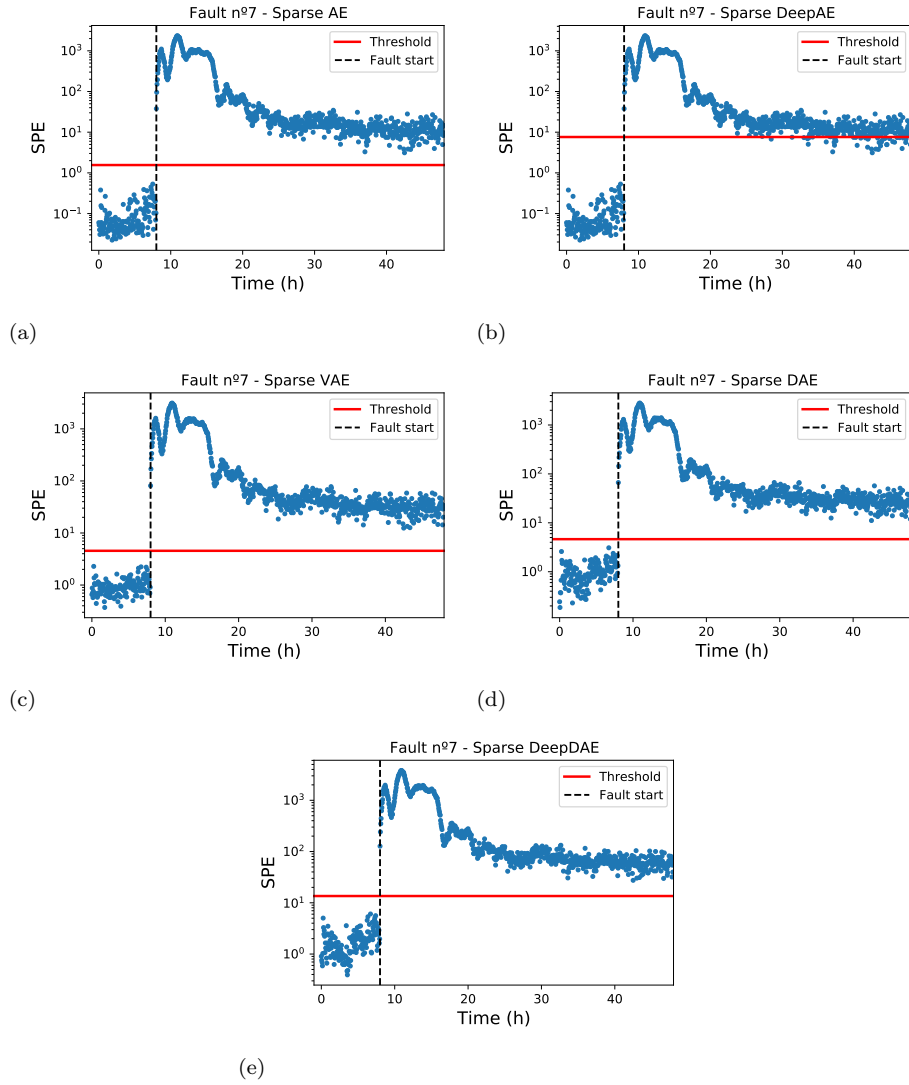


Figure S28: SPE Charts for sparse Autoencoders - fault 7: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

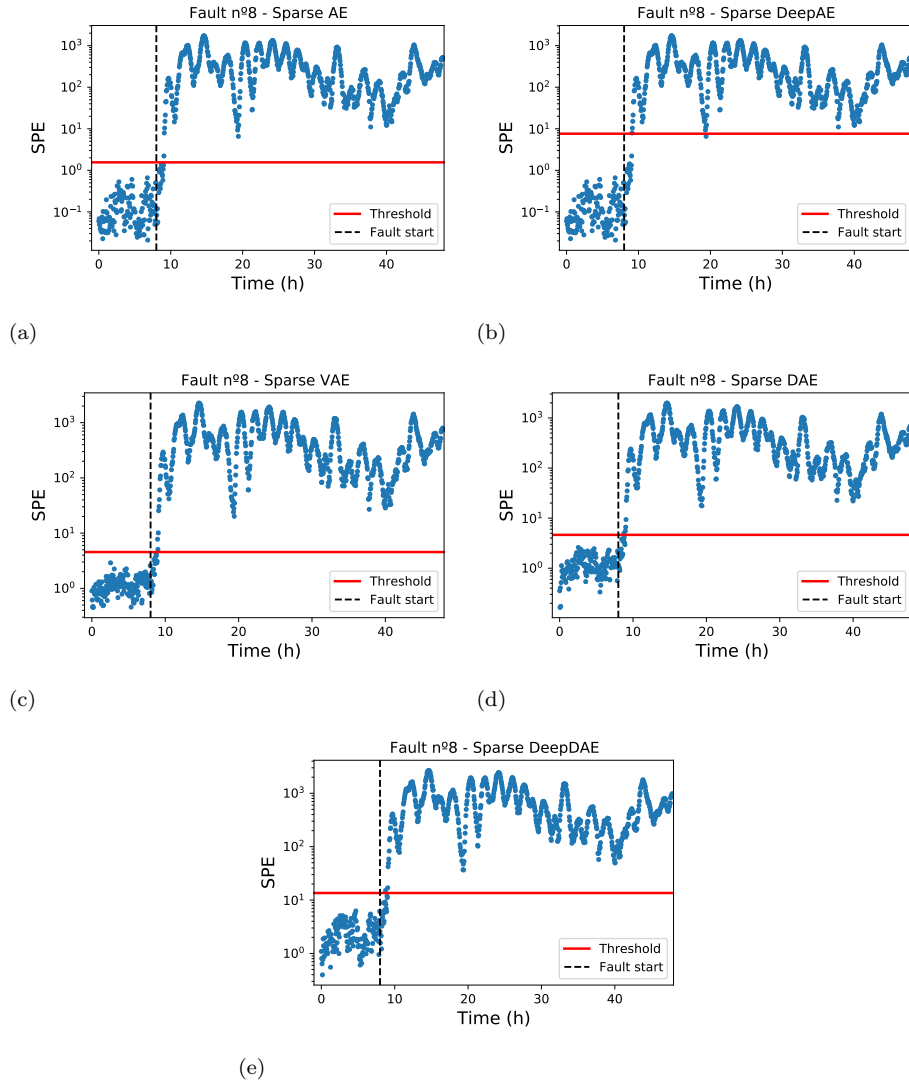


Figure S29: SPE Charts for sparse Autoencoders - fault 8: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder



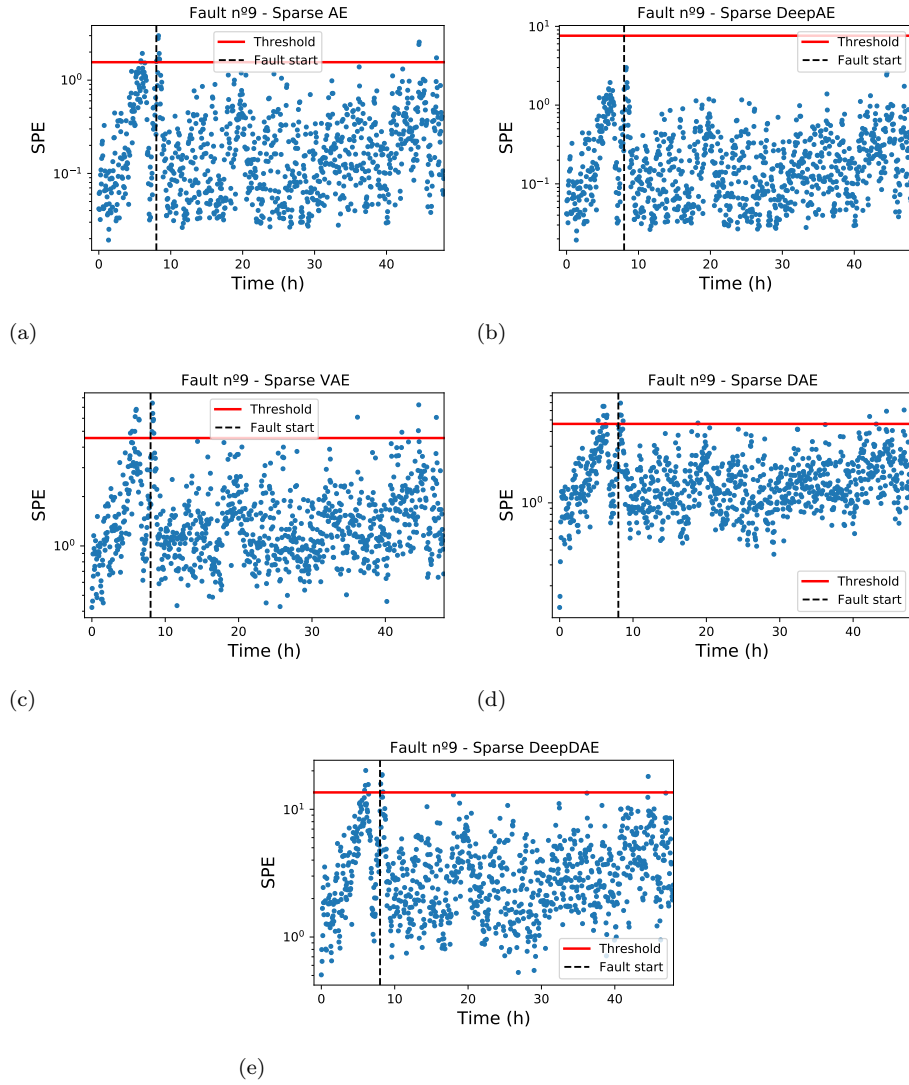


Figure S30: SPE Charts for sparse Autoencoders - fault 9: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

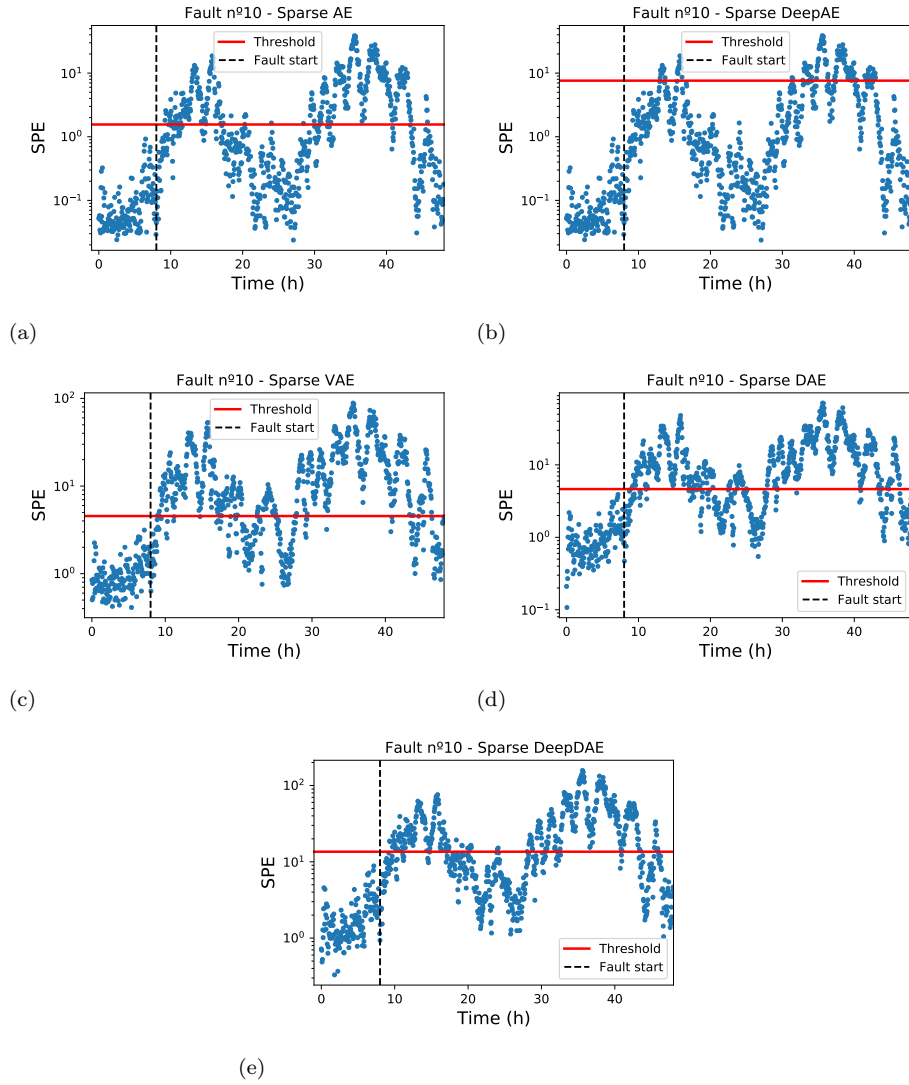


Figure S31: SPE Charts for sparse Autoencoders - fault 10: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

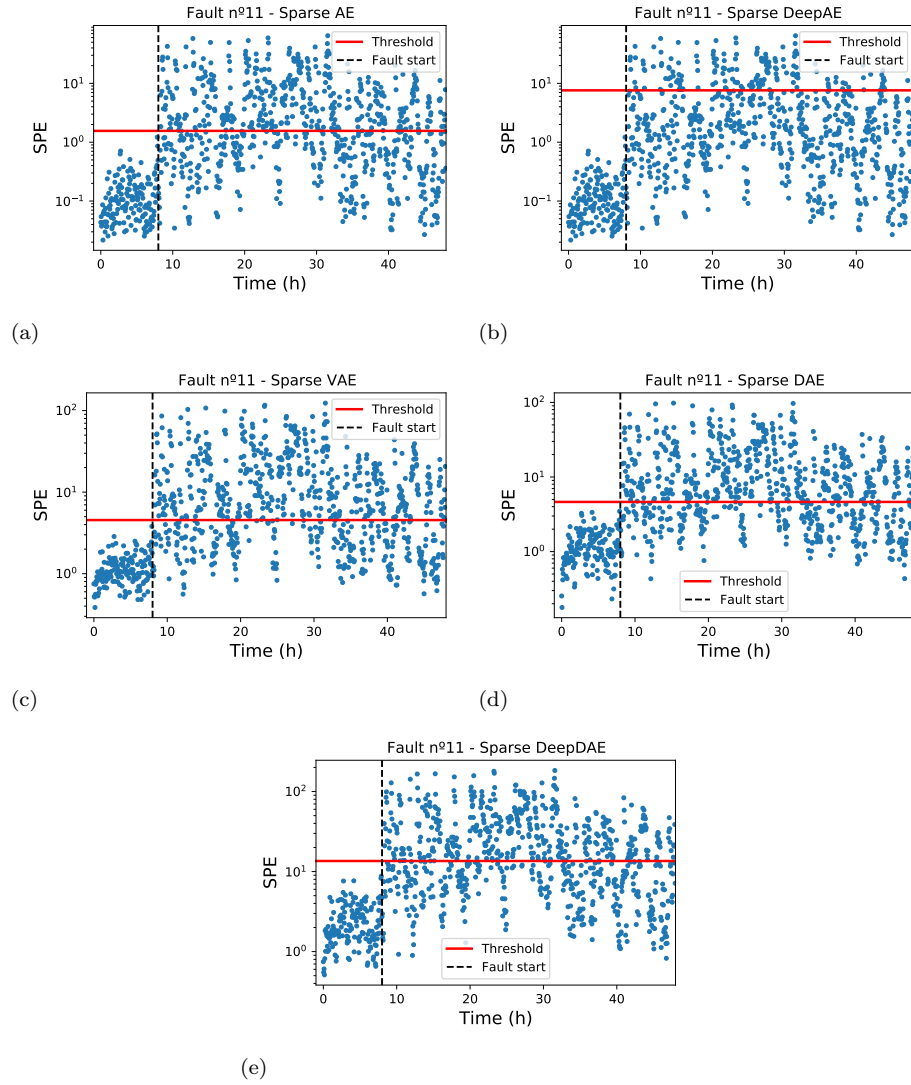


Figure S32: SPE Charts for sparse Autoencoders - fault 11: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

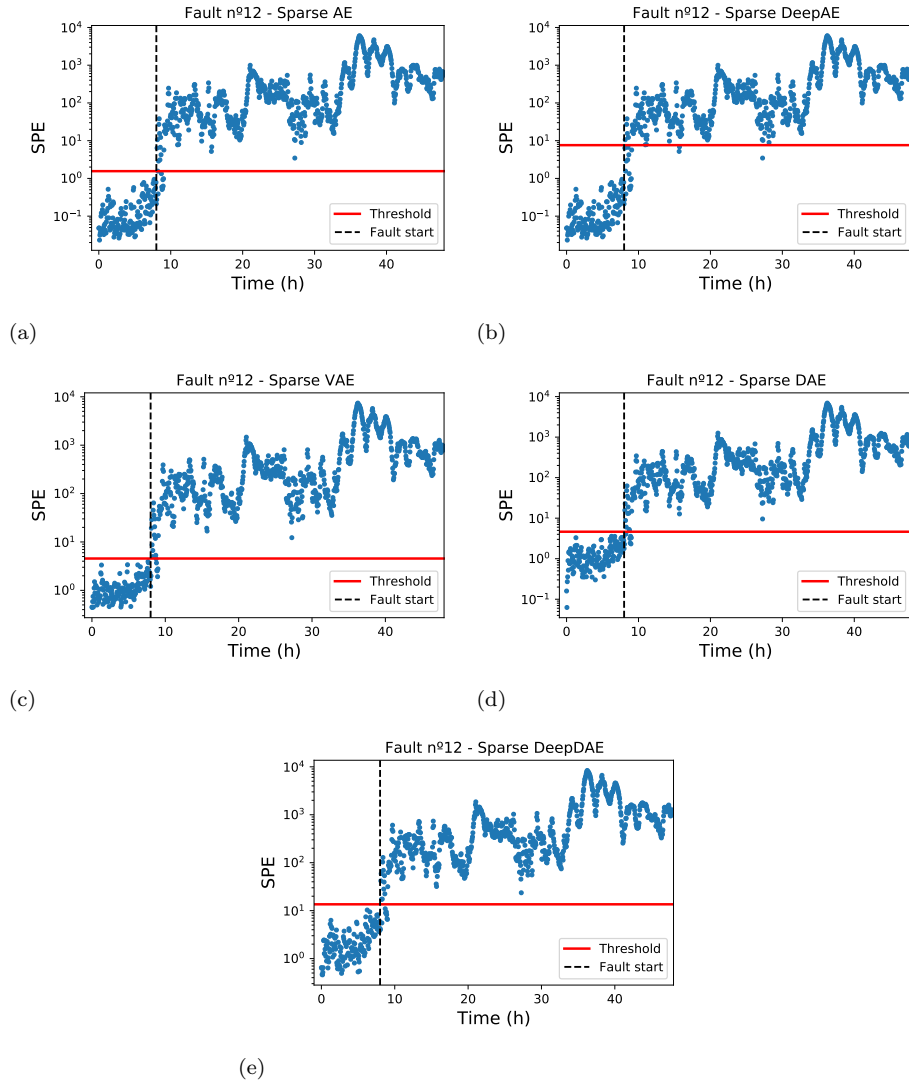


Figure S33: SPE Charts for sparse Autoencoders - fault 12: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

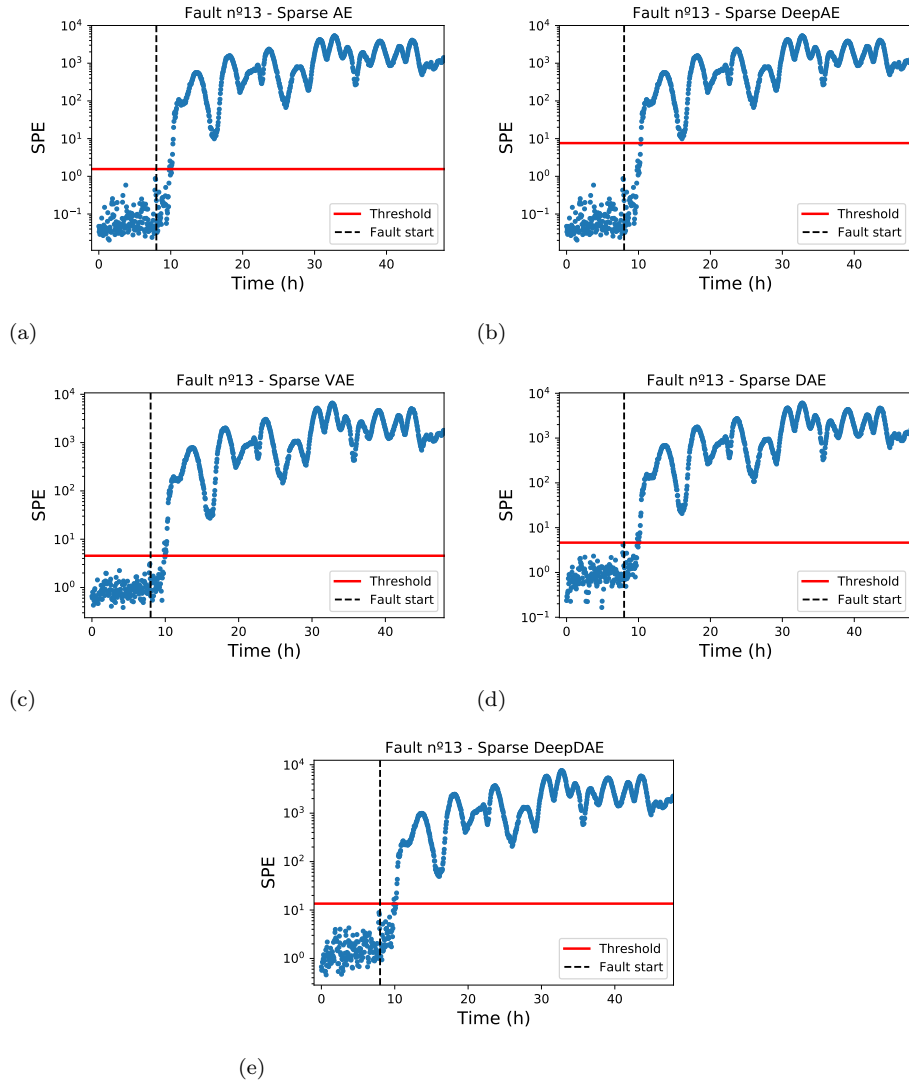


Figure S34: SPE Charts for sparse Autoencoders - fault 13: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

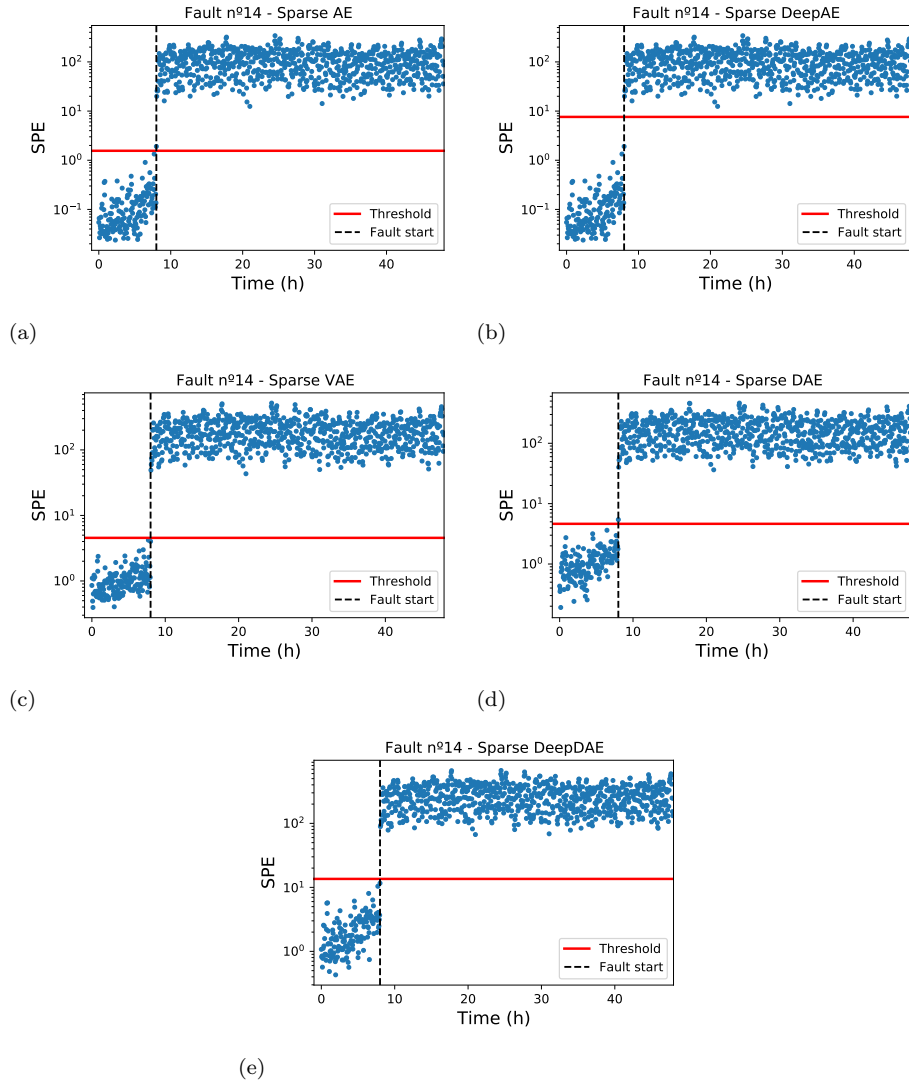


Figure S35: SPE Charts for sparse Autoencoders - fault 14: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

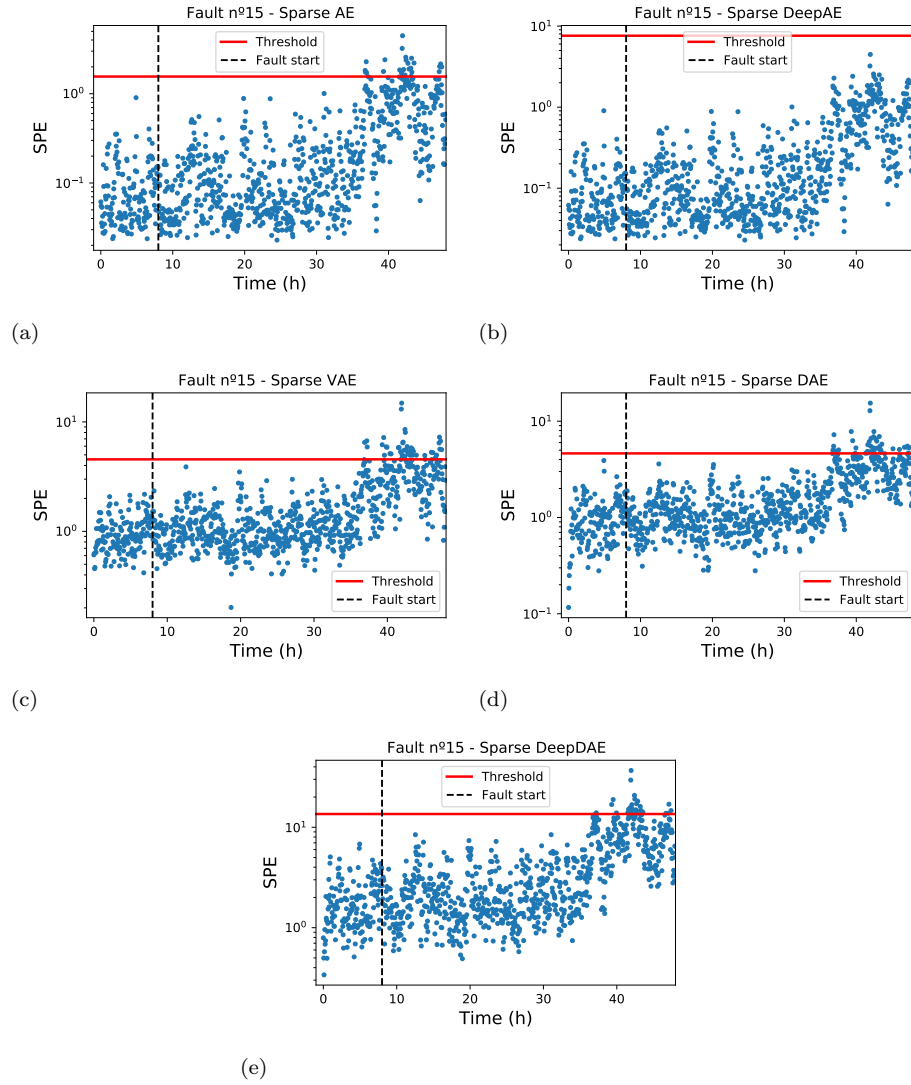


Figure S36: SPE Charts for sparse Autoencoders - fault 15: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

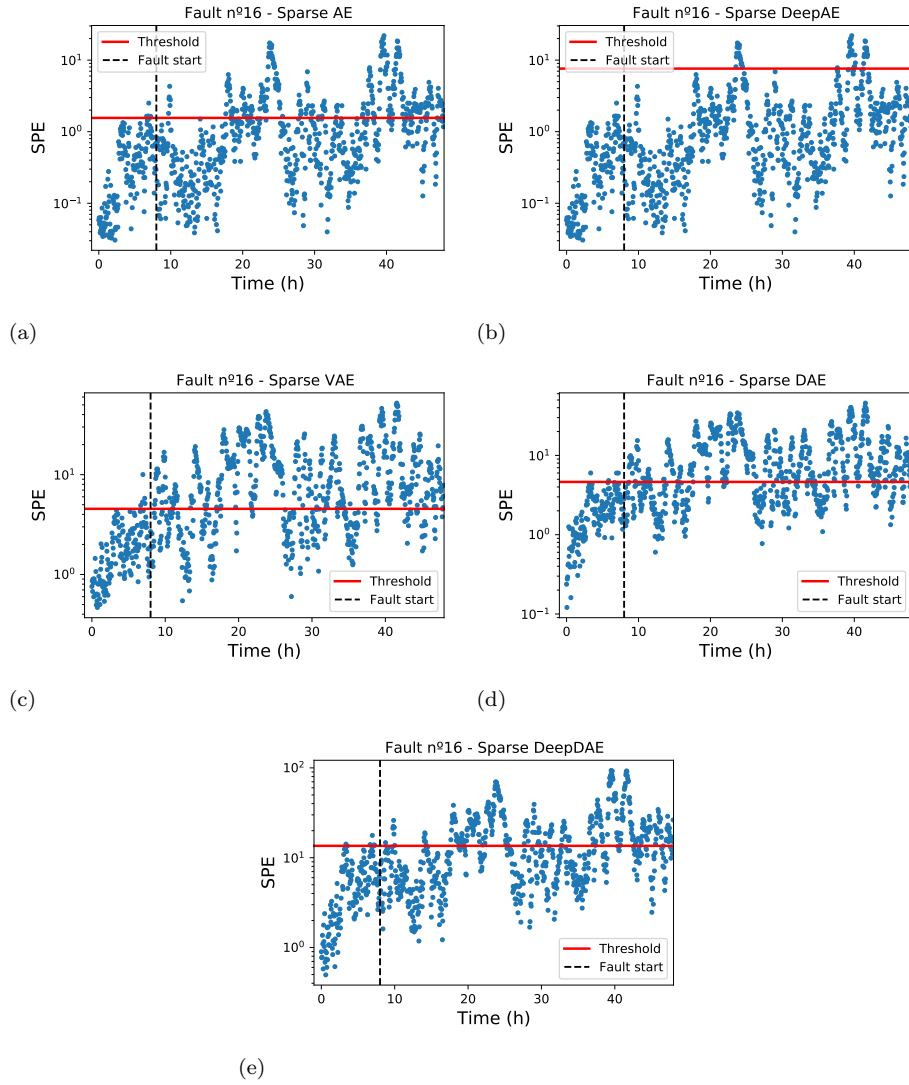


Figure S37: SPE Charts for sparse Autoencoders - fault 16: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder



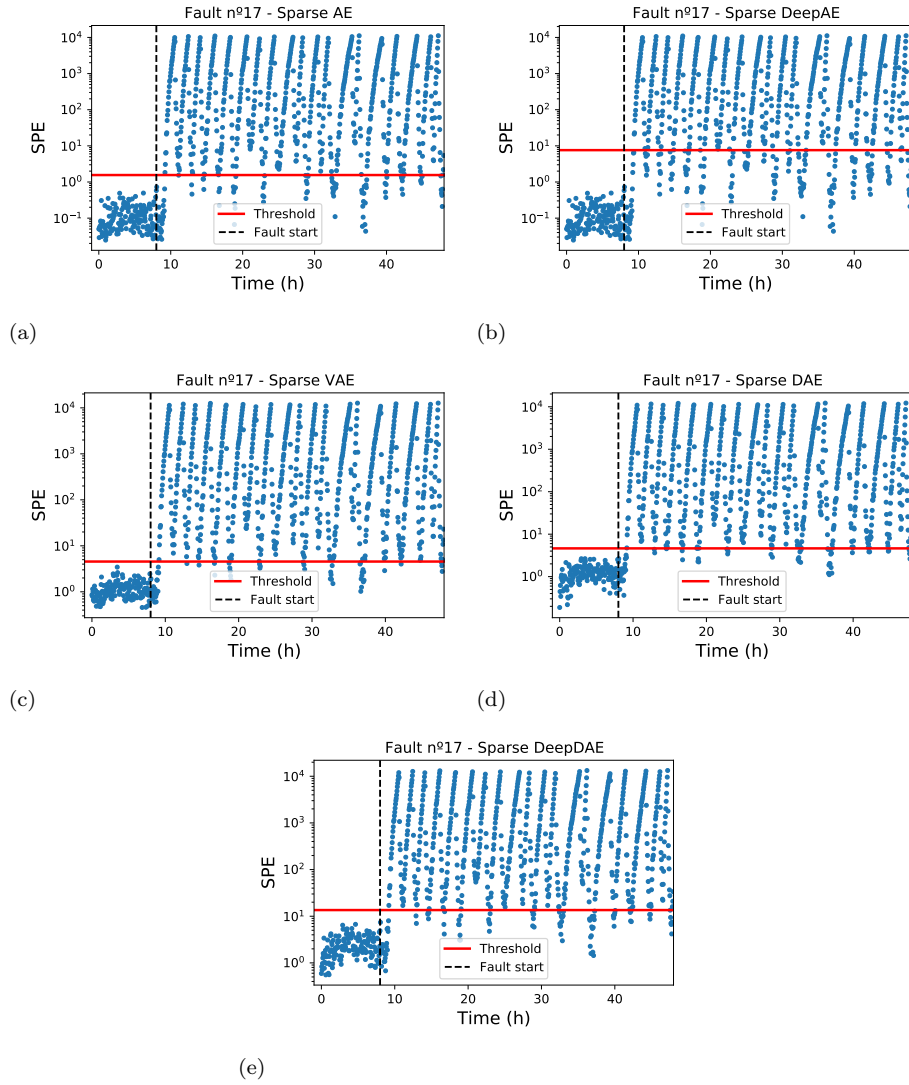


Figure S38: SPE Charts for sparse Autoencoders - fault 17: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

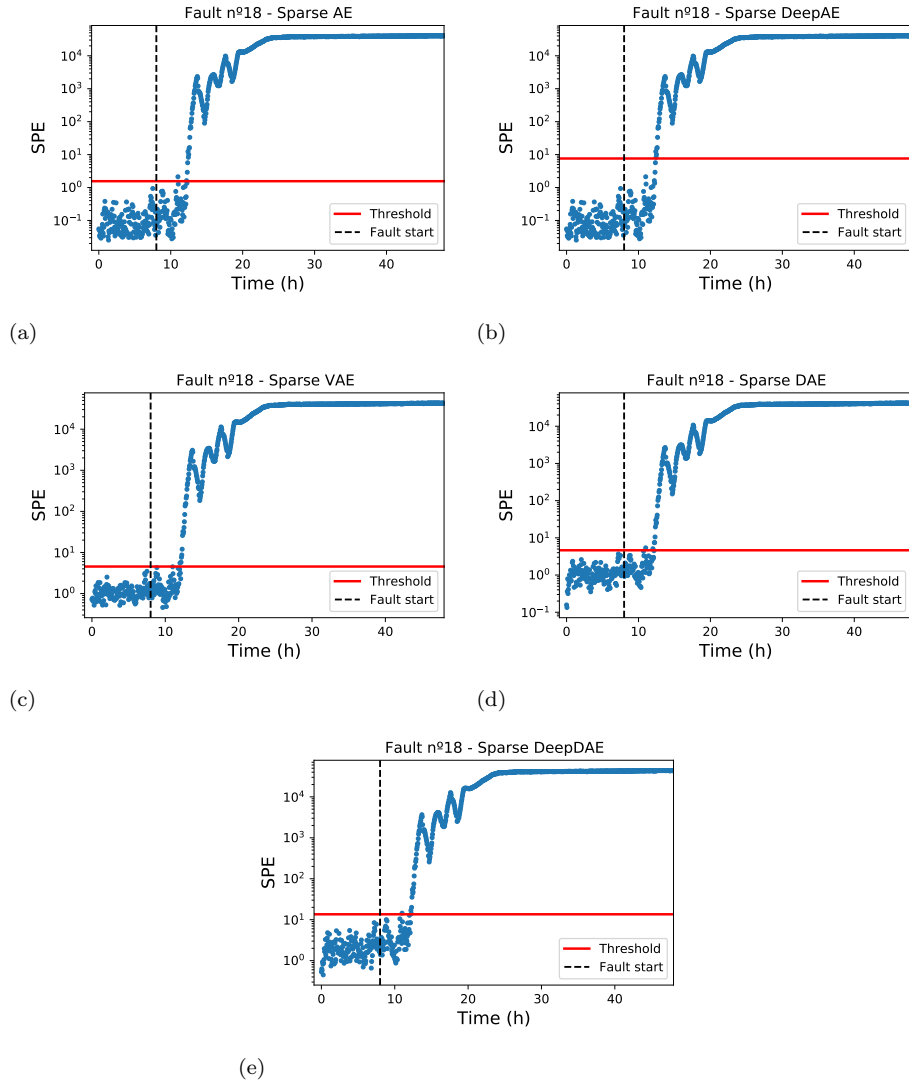


Figure S39: SPE Charts for sparse Autoencoders - fault 18: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

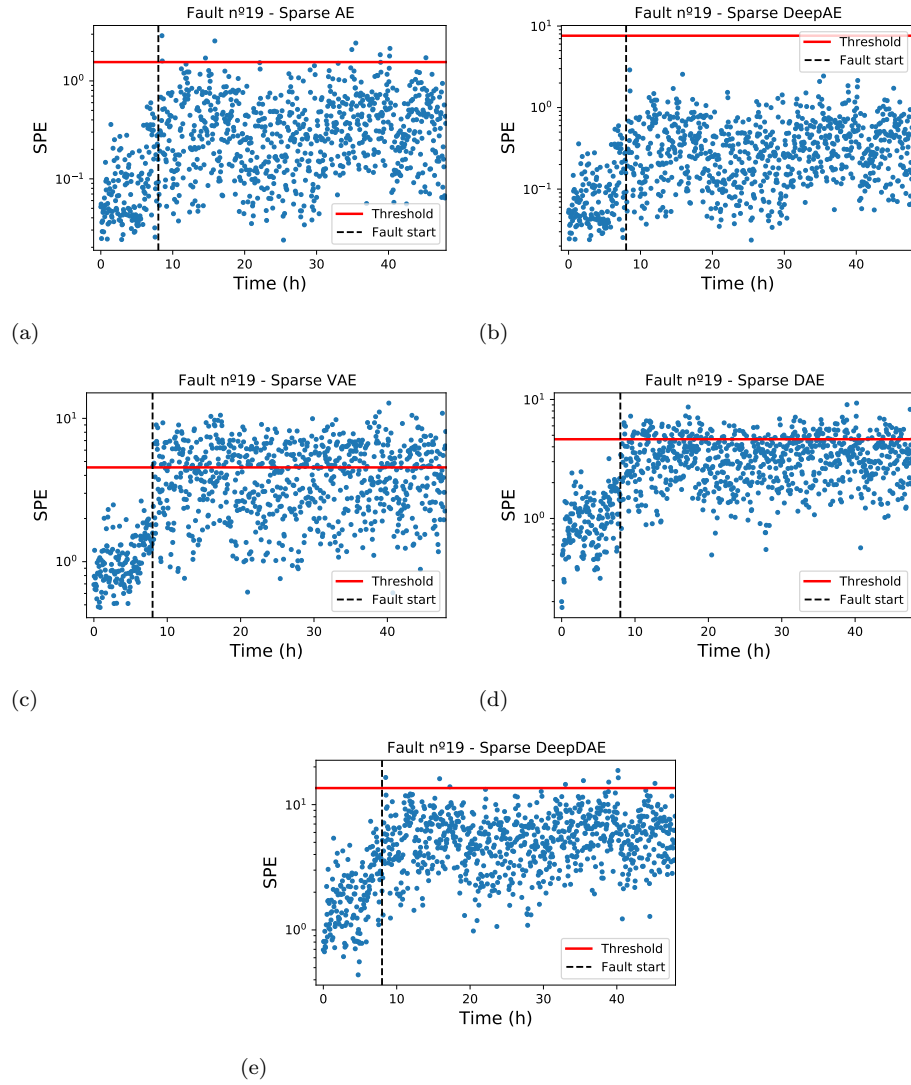


Figure S40: SPE Charts for sparse Autoencoders - fault 19: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

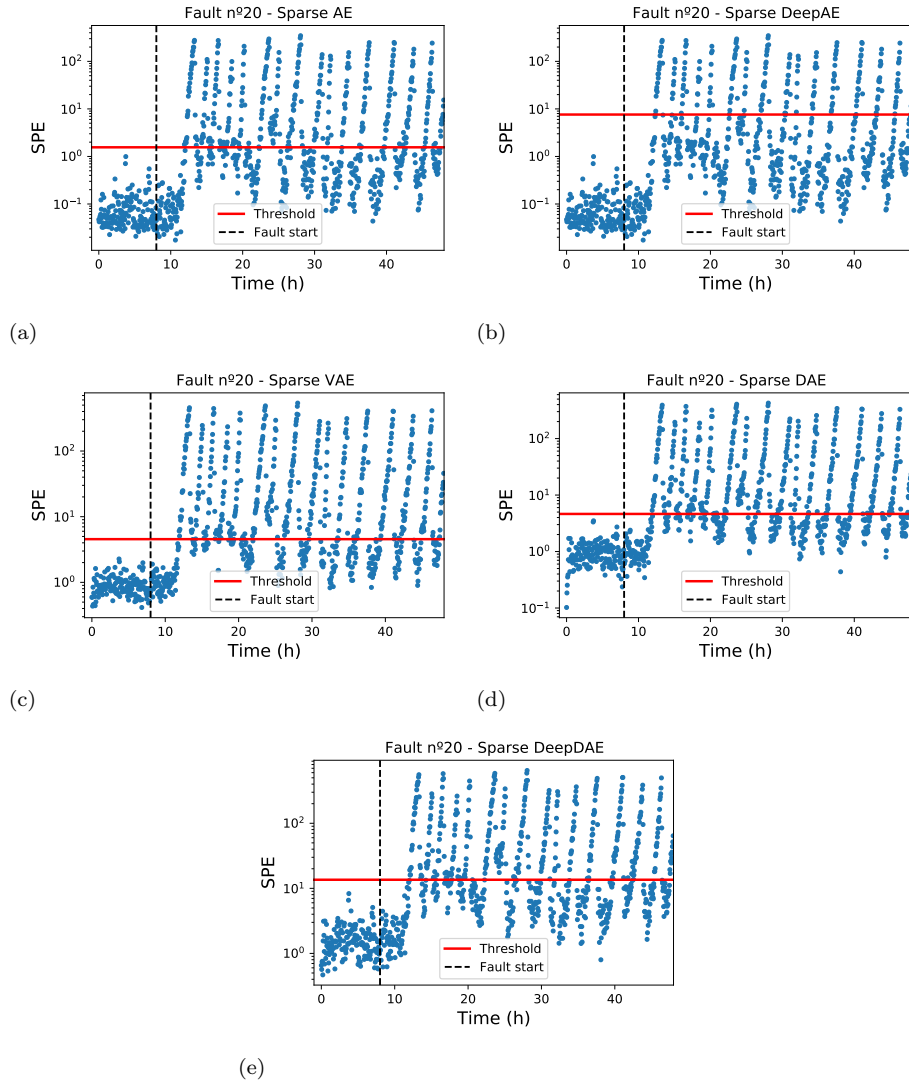


Figure S41: SPE Charts for sparse Autoencoders - fault 20: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

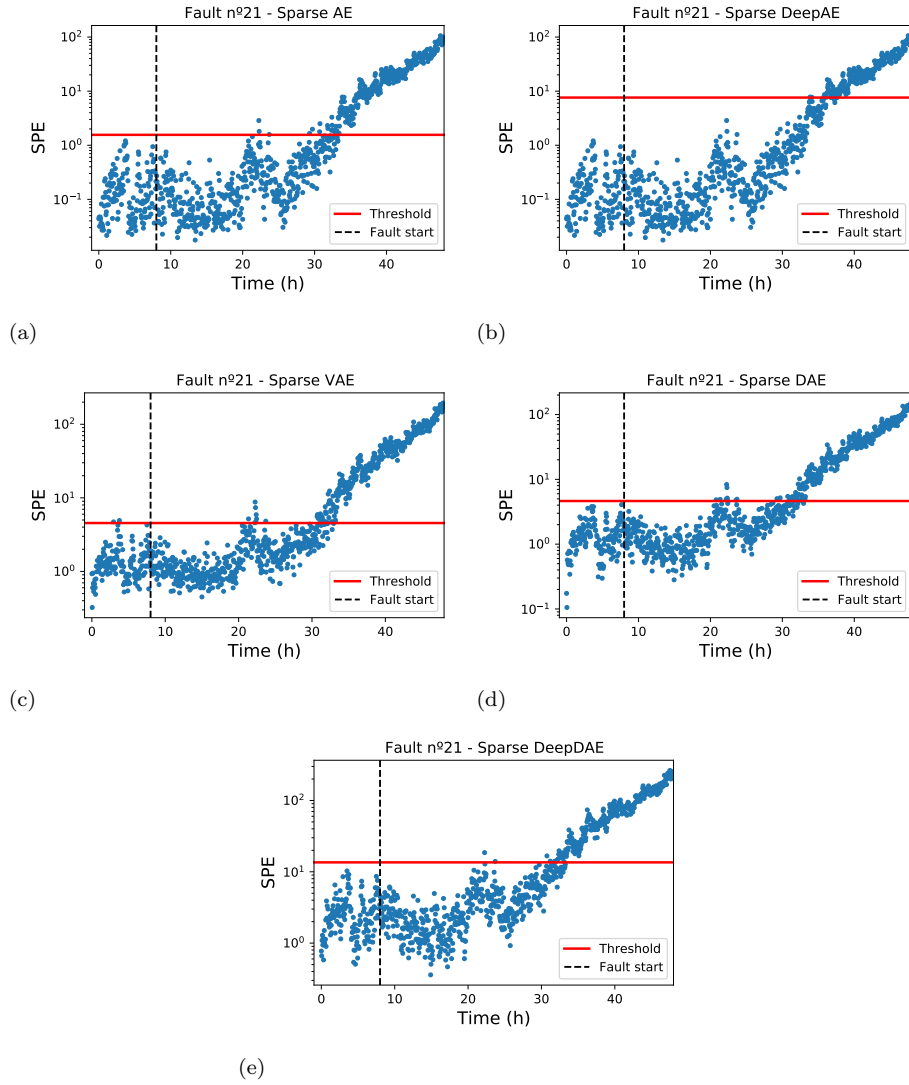


Figure S42: SPE Charts for sparse Autoencoders - fault 21: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

## 2.2 Undercomplete Autoencoders

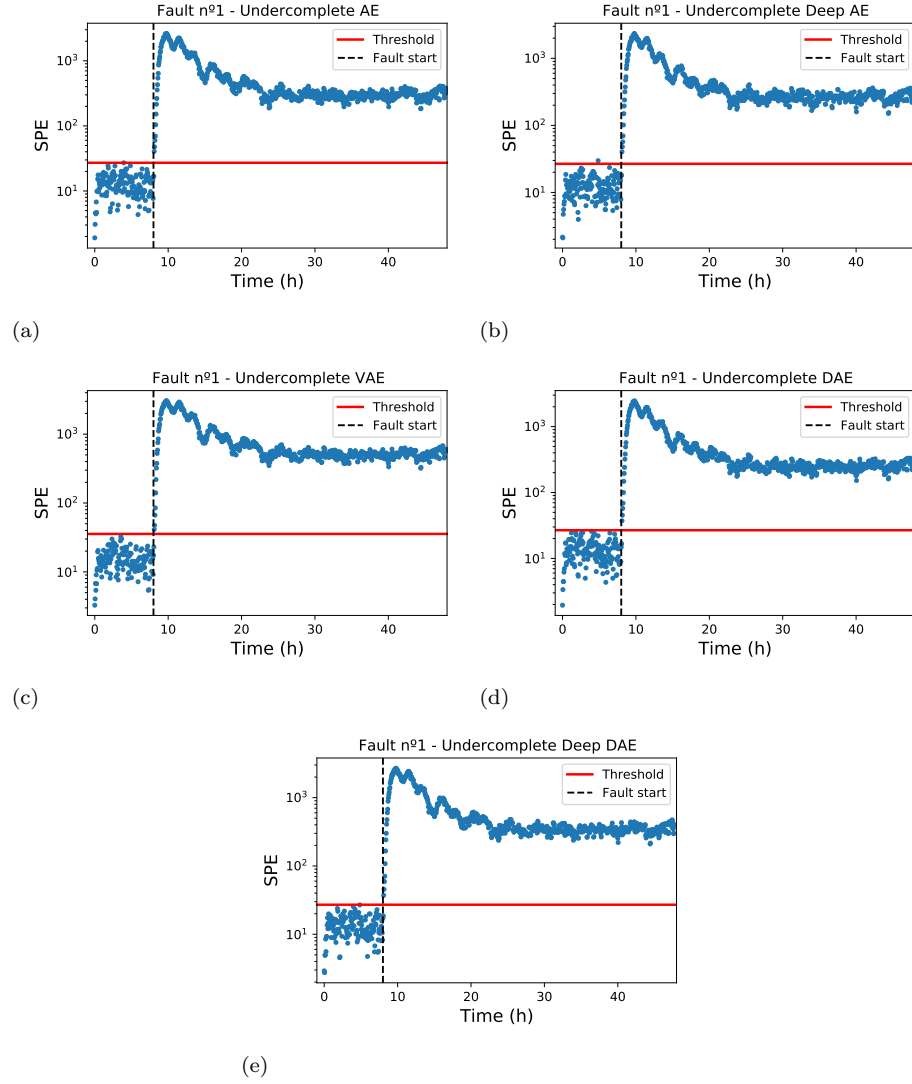


Figure S43: SPE Charts for undercomplete Autoencoders - fault 1: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

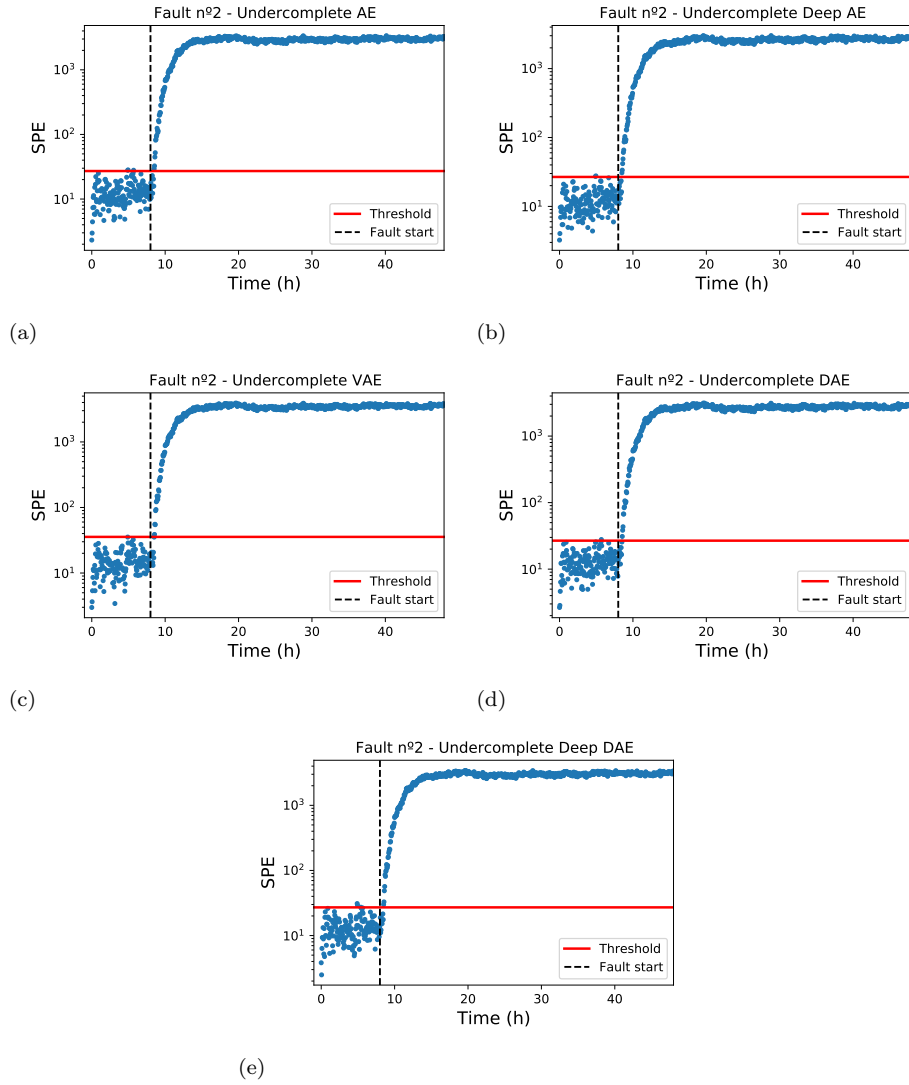


Figure S44: SPE Charts for undercomplete Autoencoders - fault 2: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

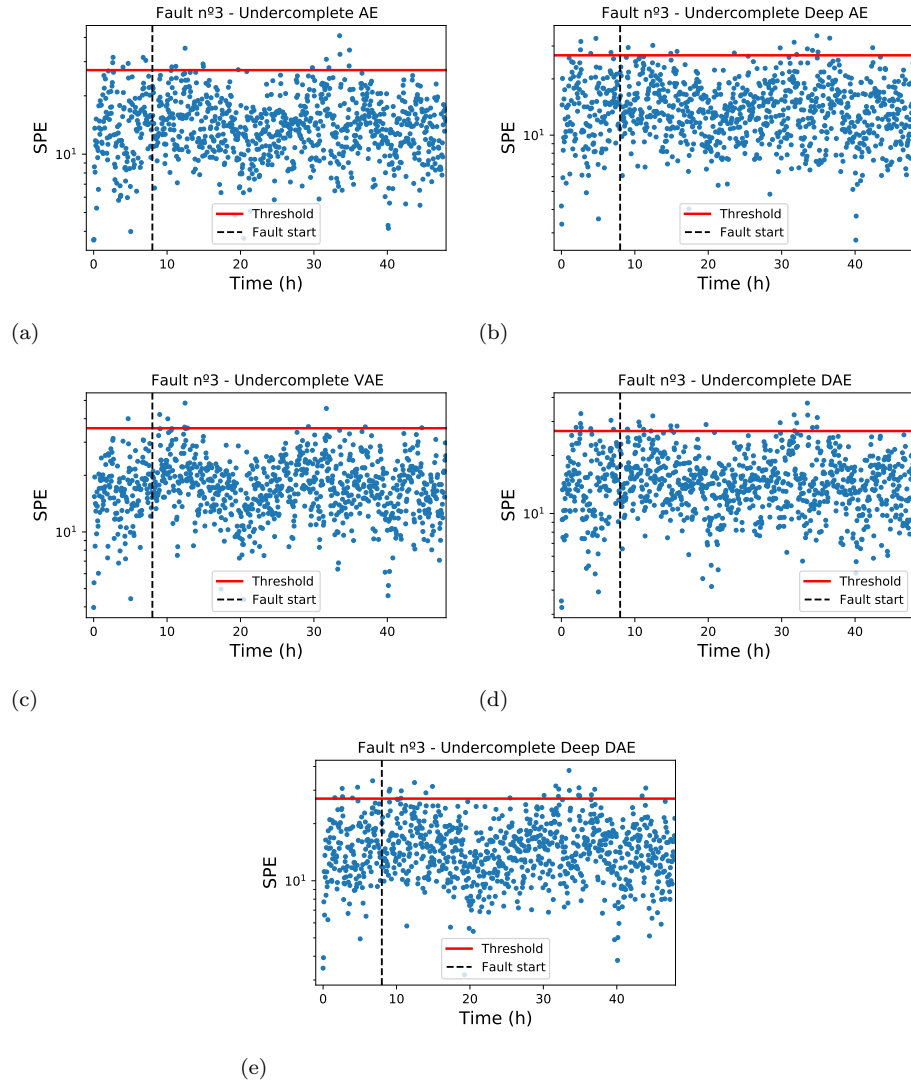


Figure S45: SPE Charts for undercomplete Autoencoders - fault 3: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder



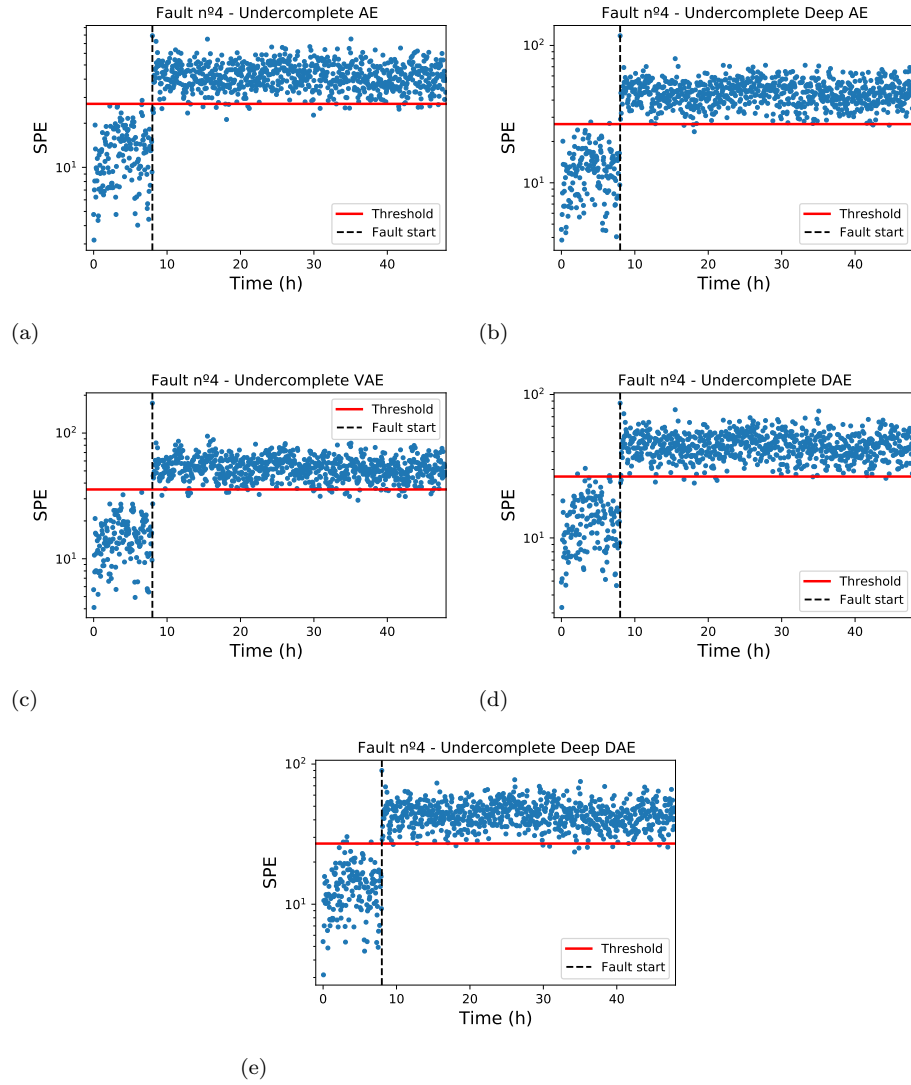


Figure S46: SPE Charts for undercomplete Autoencoders - fault 4: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

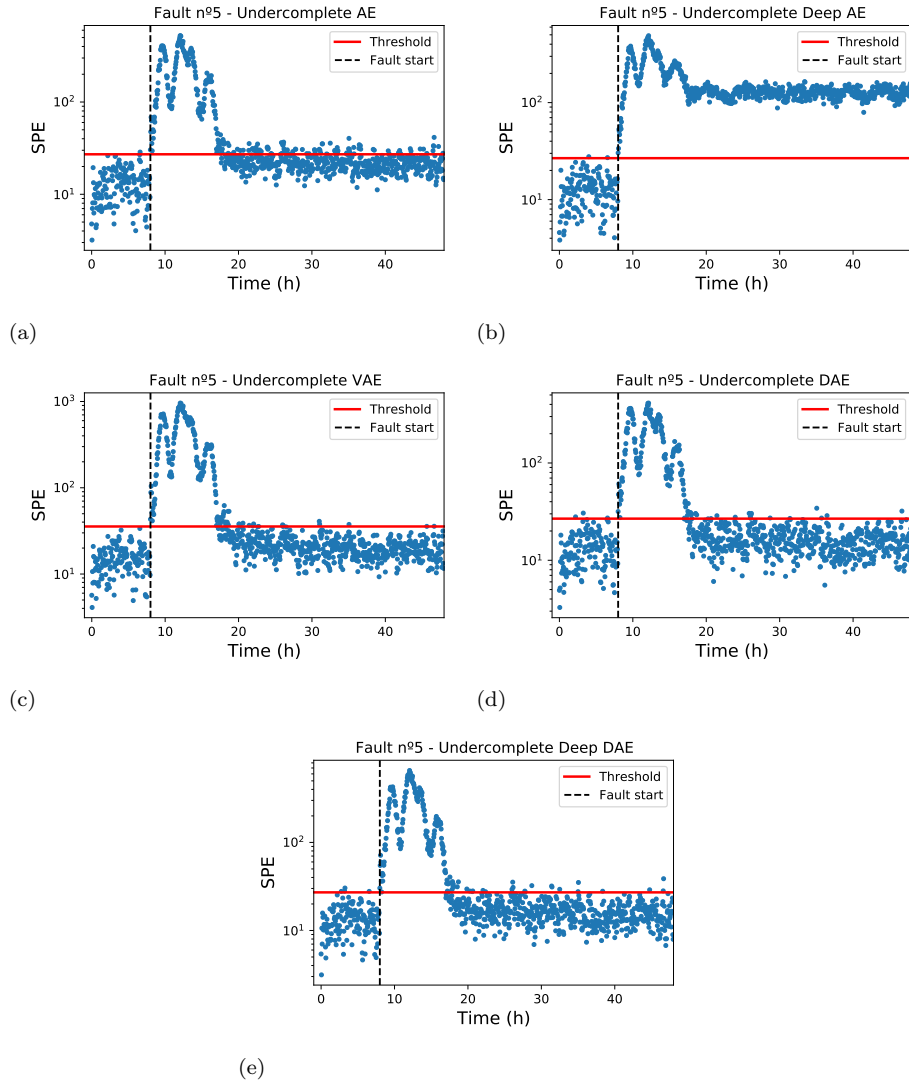


Figure S47: SPE Charts for undercomplete Autoencoders - fault 5: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

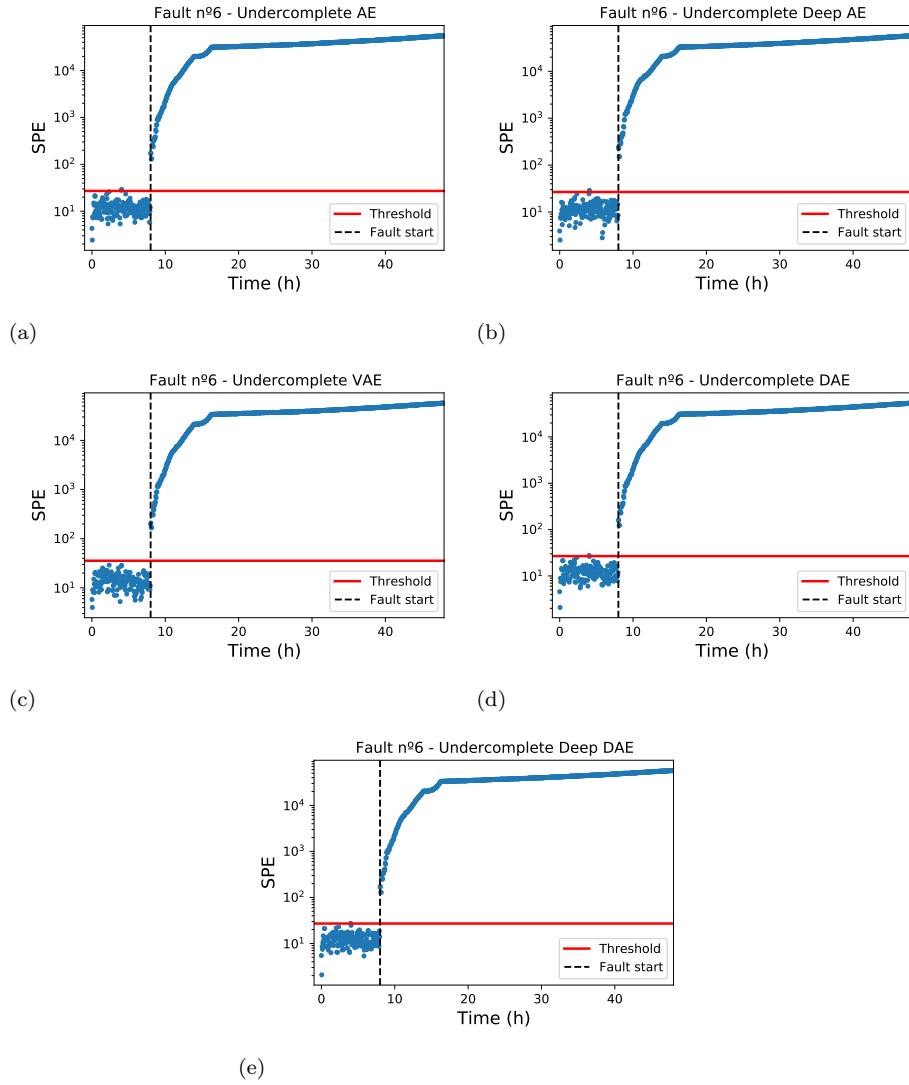


Figure S48: SPE Charts for undercomplete Autoencoders - fault 6: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

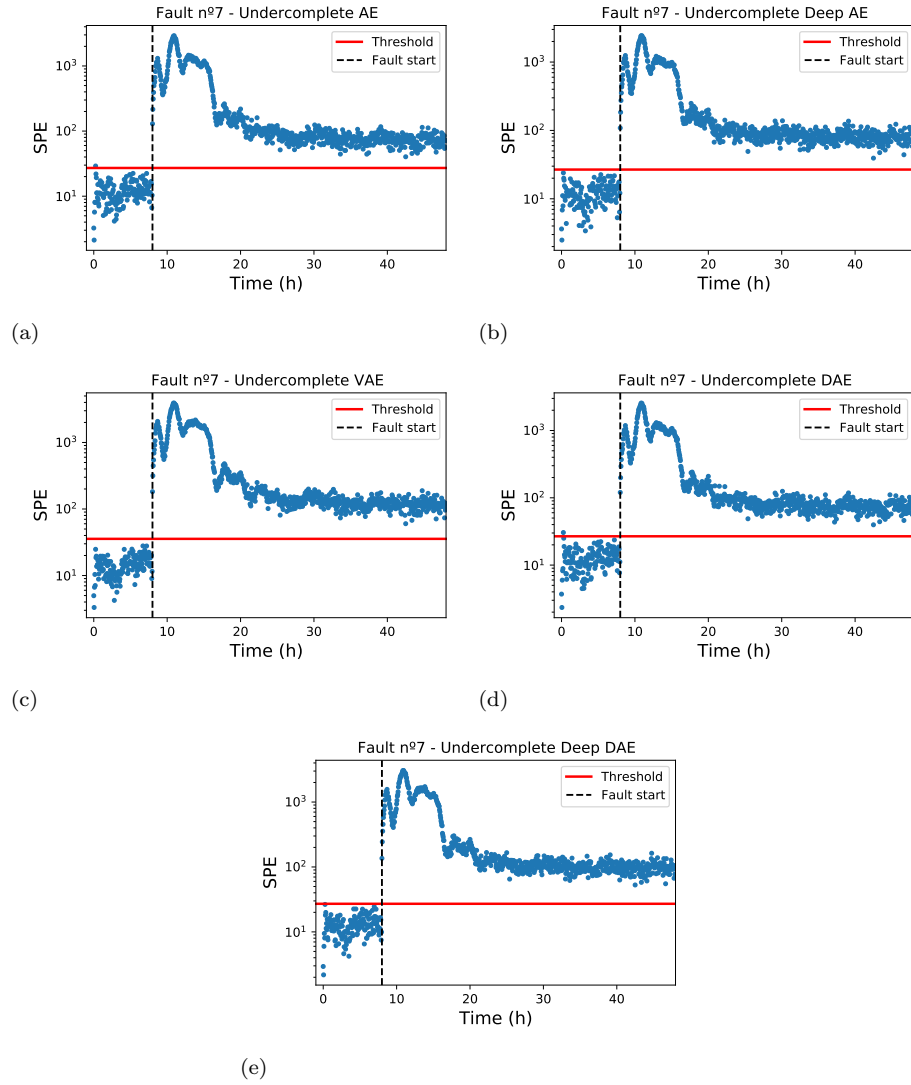


Figure S49: SPE Charts for undercomplete Autoencoders - fault 7: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

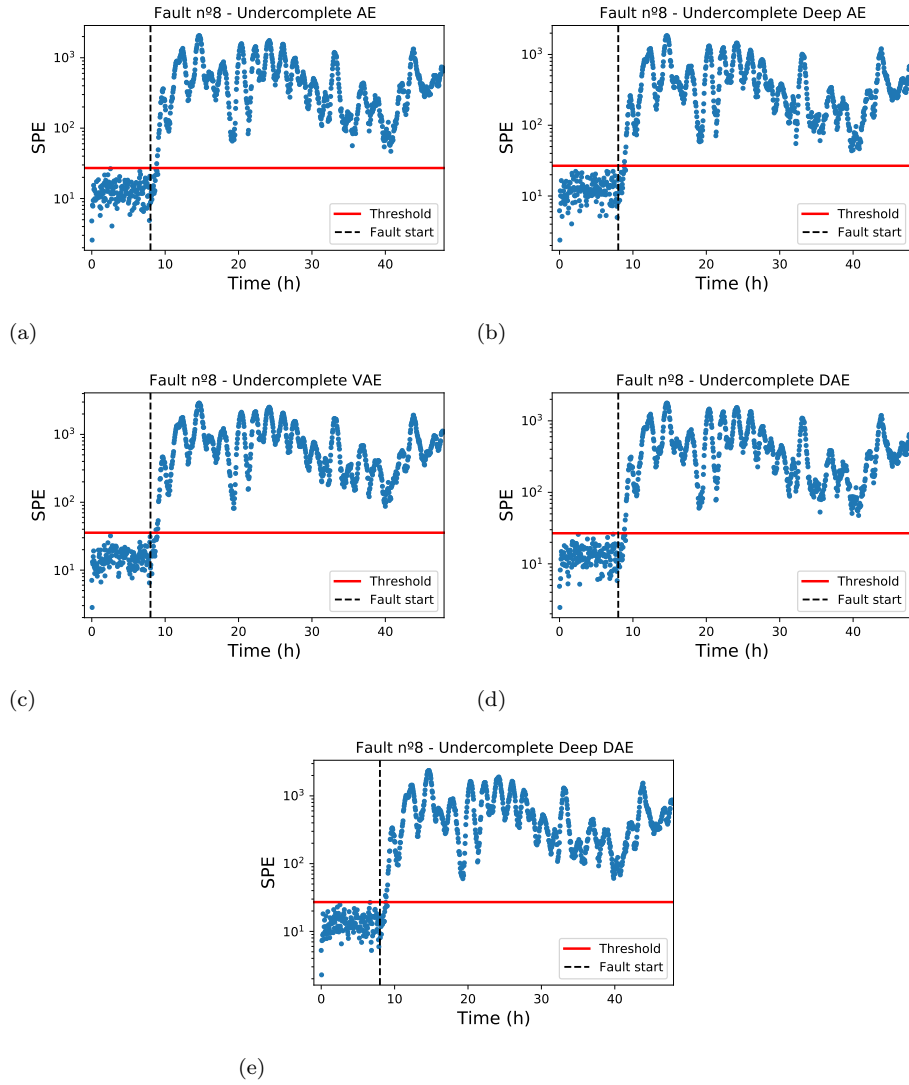


Figure S50: SPE Charts for undercomplete Autoencoders - fault 8: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

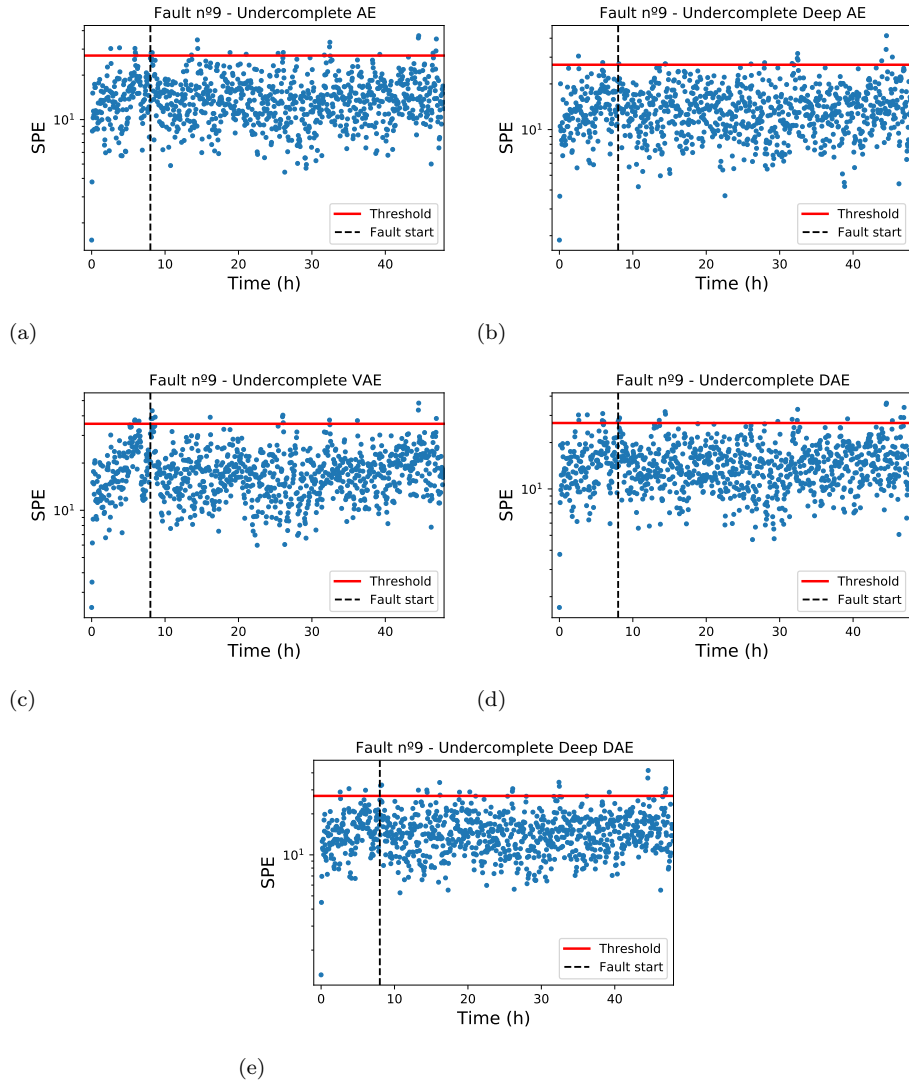


Figure S51: SPE Charts for undercomplete Autoencoders - fault 9: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

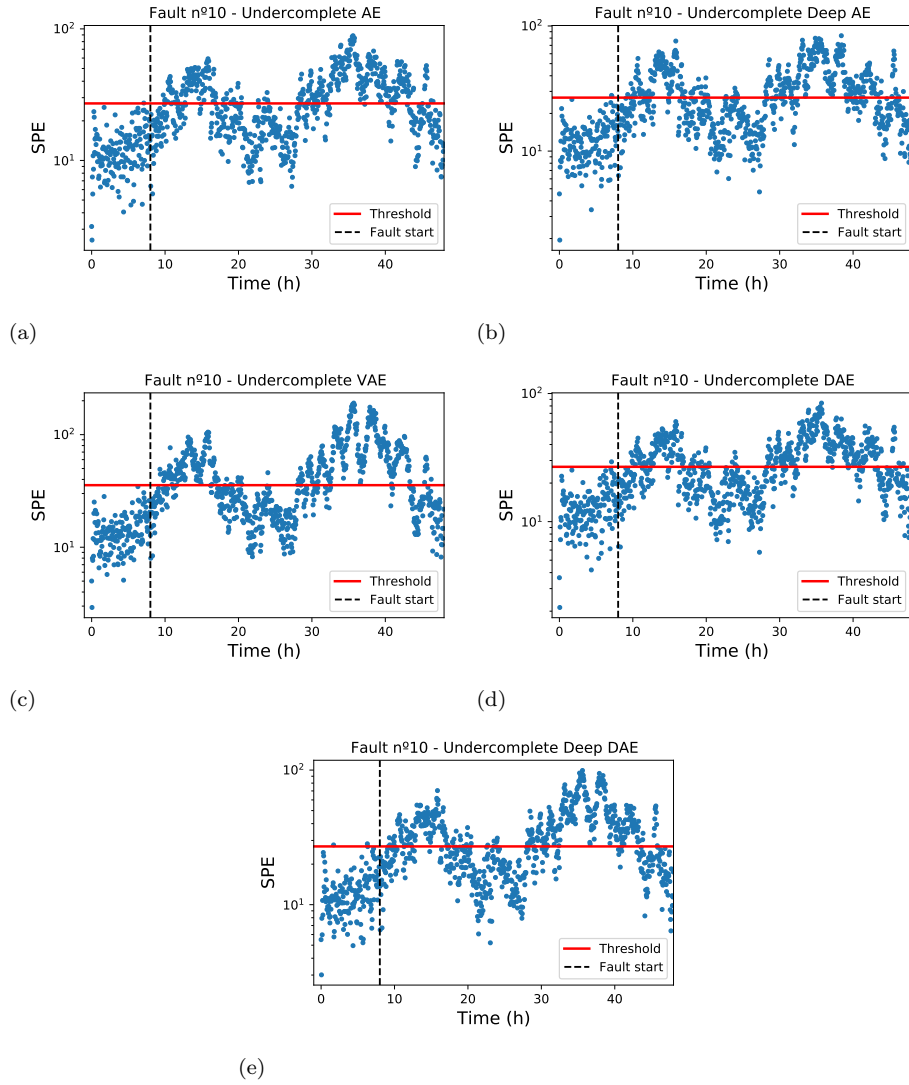


Figure S52: SPE Charts for undercomplete Autoencoders - fault 10: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

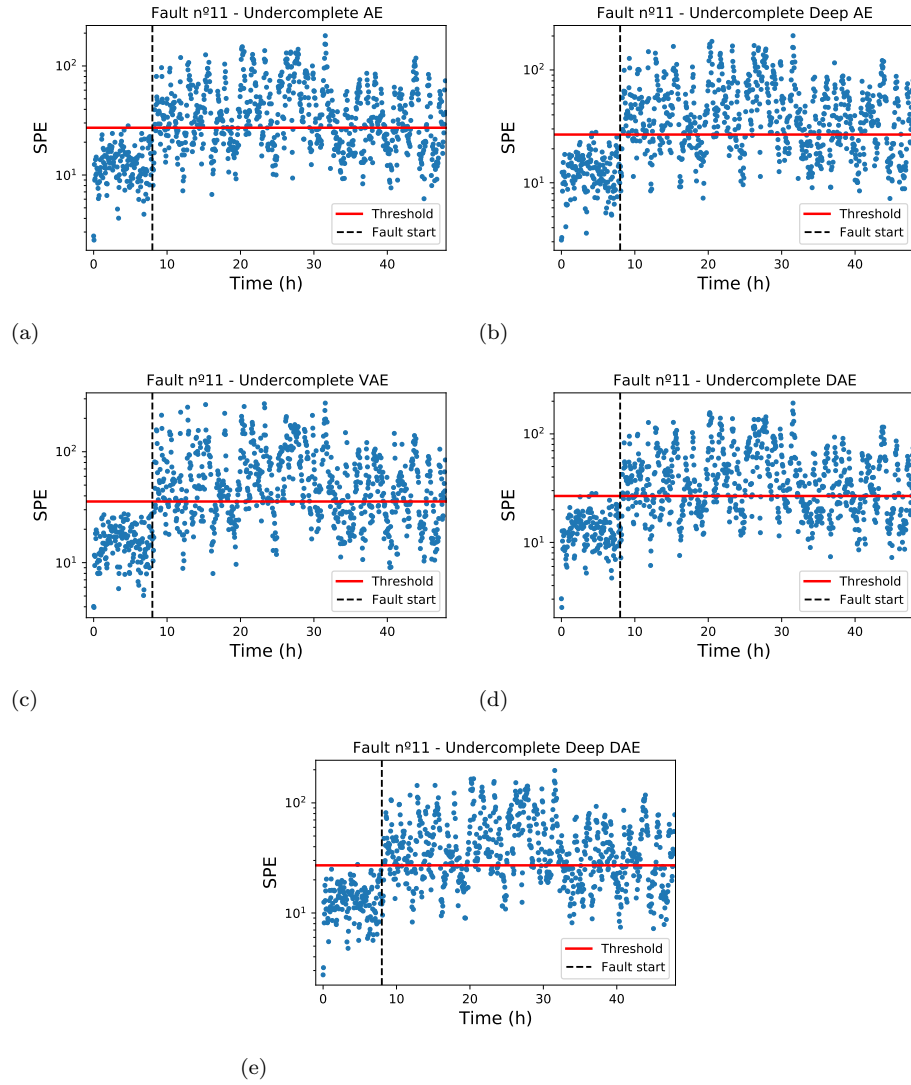


Figure S53: SPE Charts for undercomplete Autoencoders - fault 11: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder



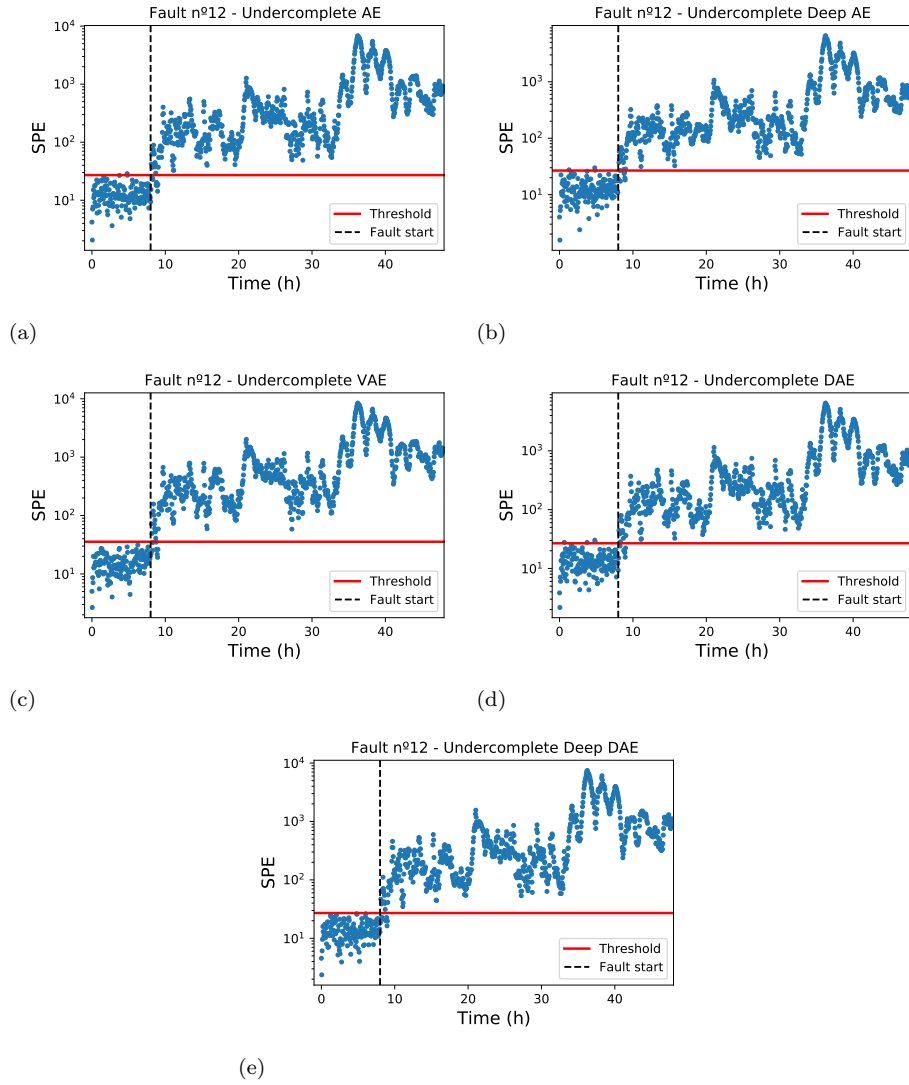


Figure S54: SPE Charts for undercomplete Autoencoders - fault 12: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

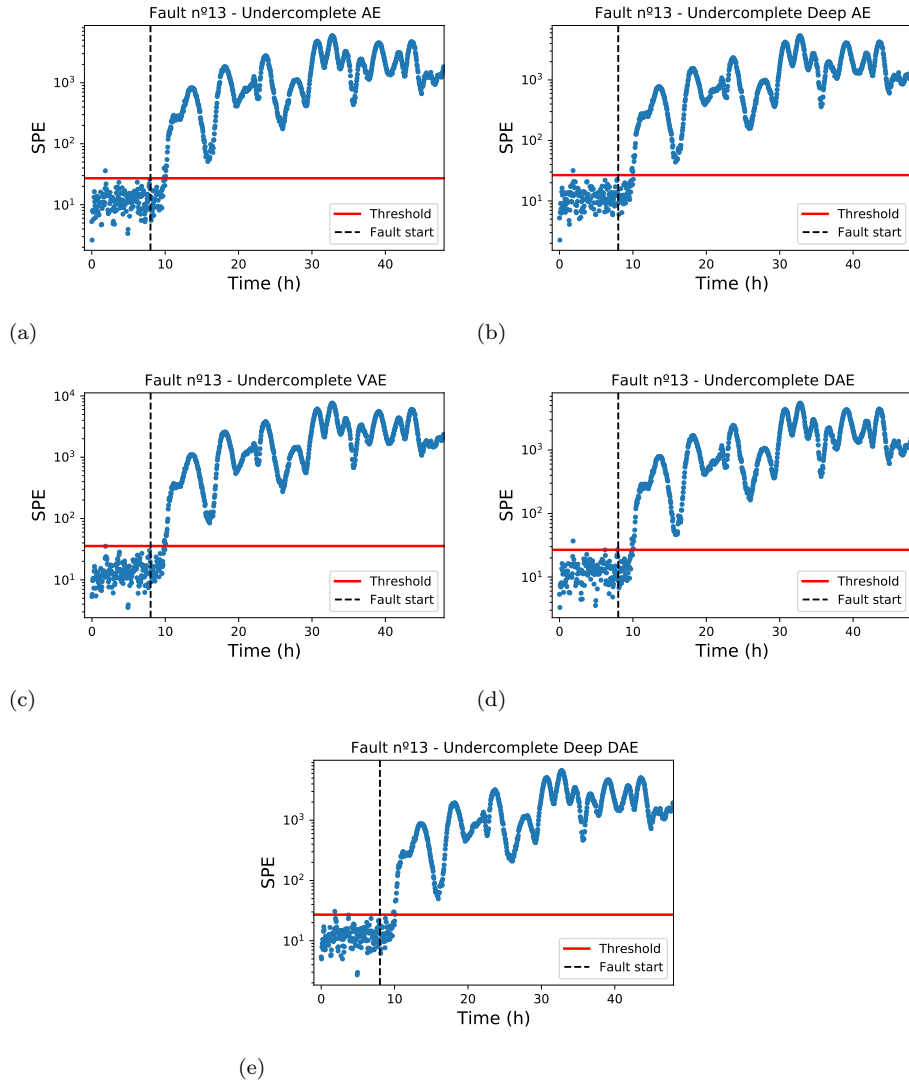


Figure S55: SPE Charts for undercomplete Autoencoders - fault 13: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

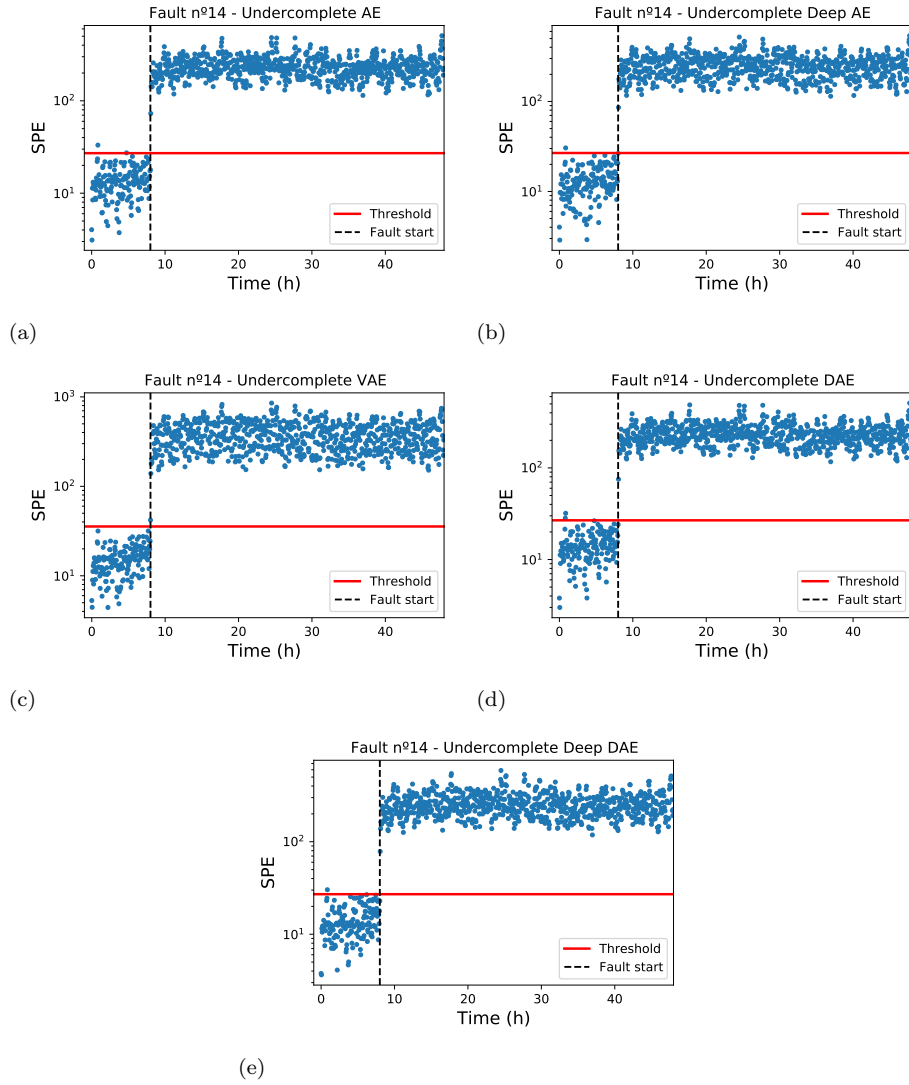


Figure S56: SPE Charts for undercomplete Autoencoders - fault 14: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

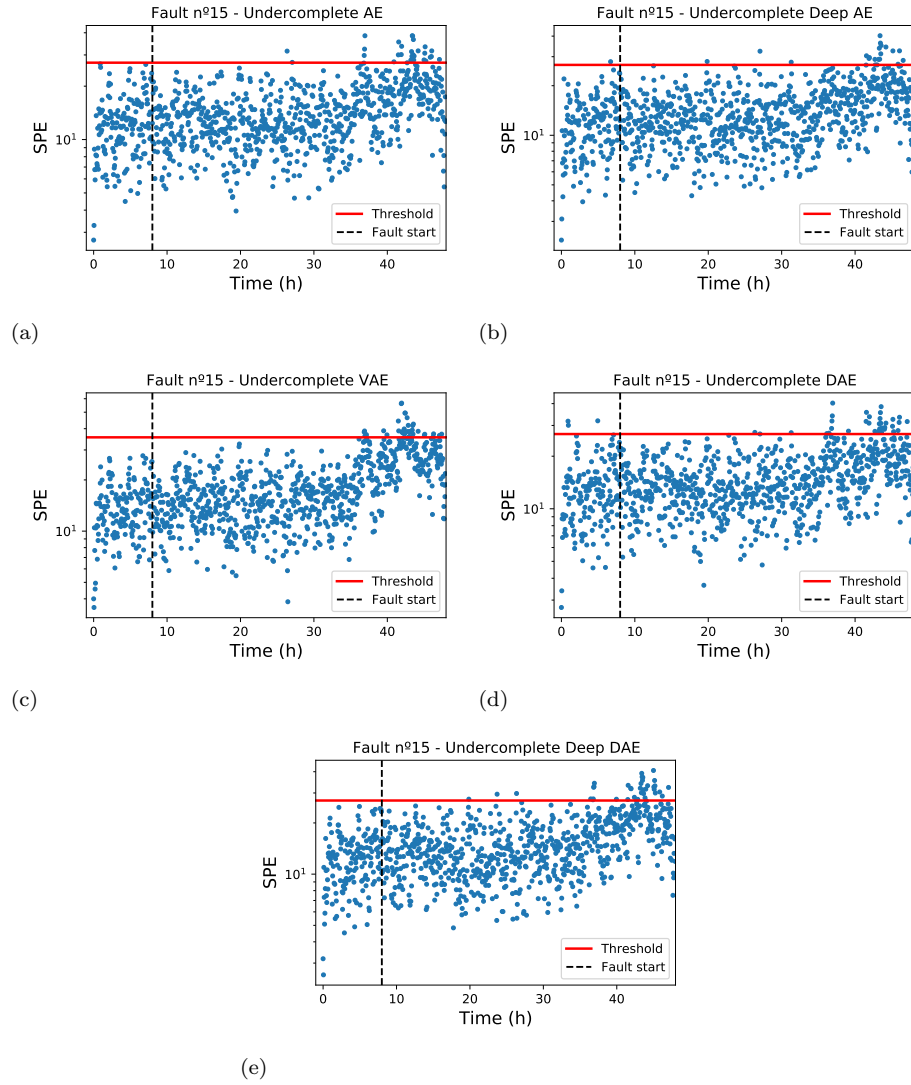


Figure S57: SPE Charts for undercomplete Autoencoders - fault 15: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

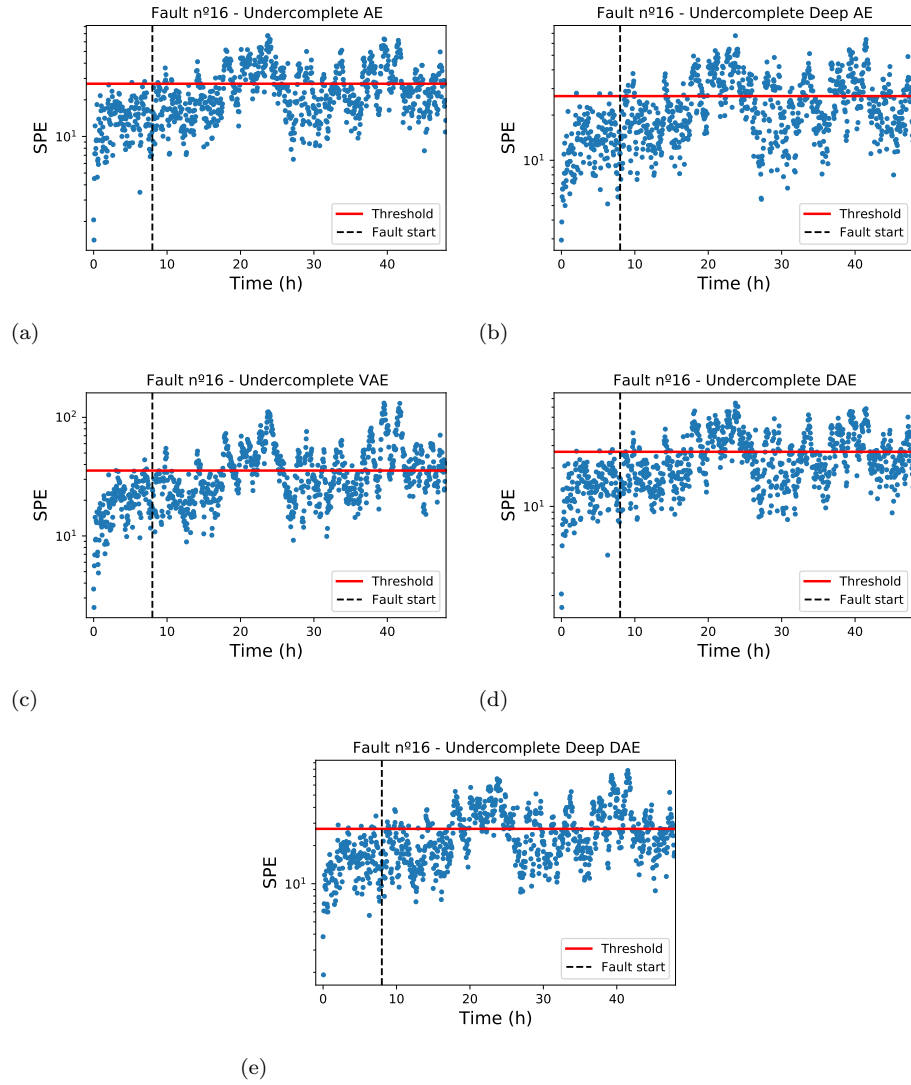


Figure S58: SPE Charts for undercomplete Autoencoders - fault 16: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

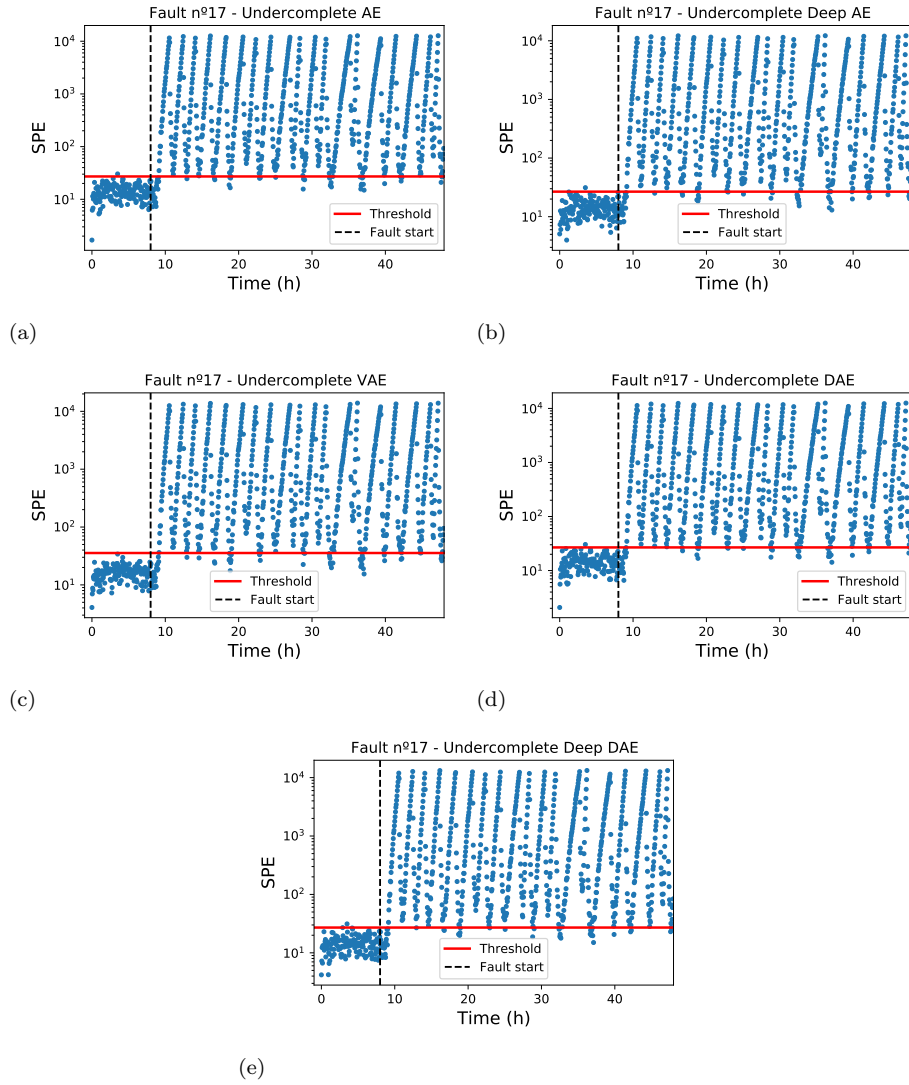


Figure S59: SPE Charts for undercomplete Autoencoders - fault 17: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

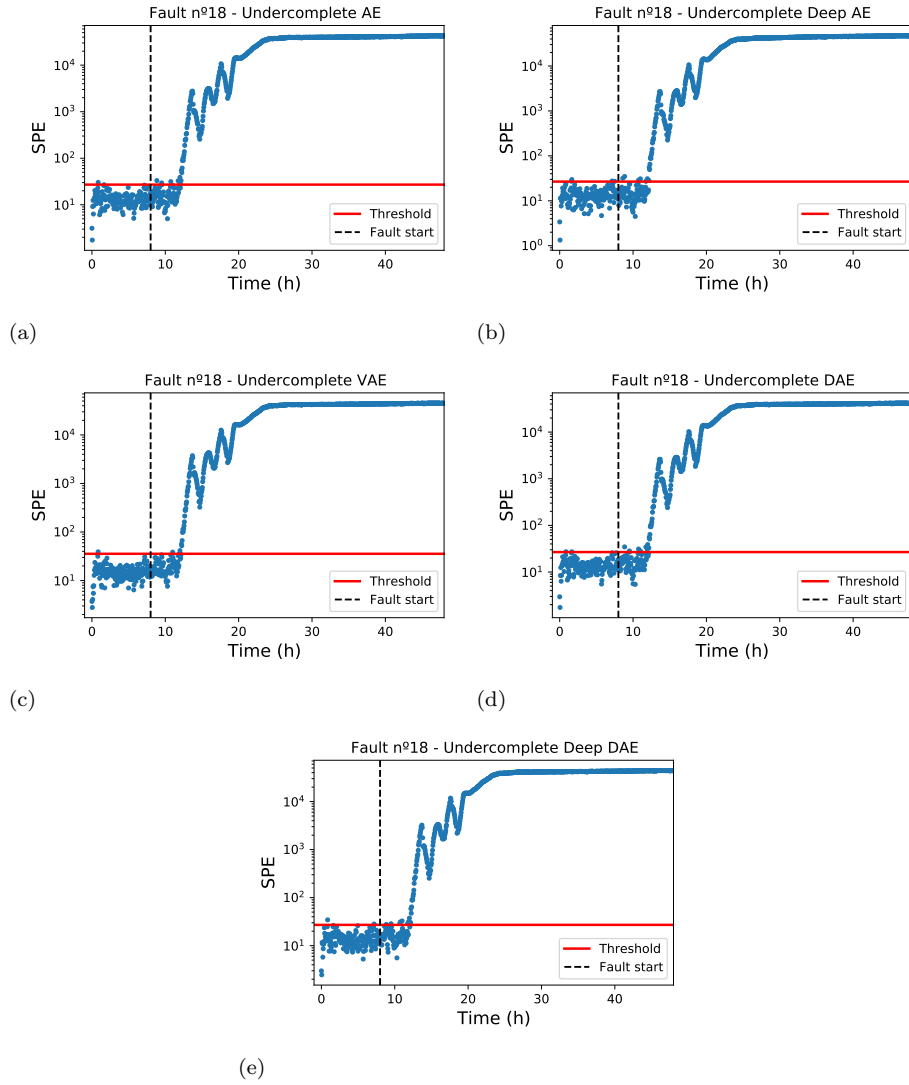


Figure S60: SPE Charts for undercomplete Autoencoders - fault 18: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

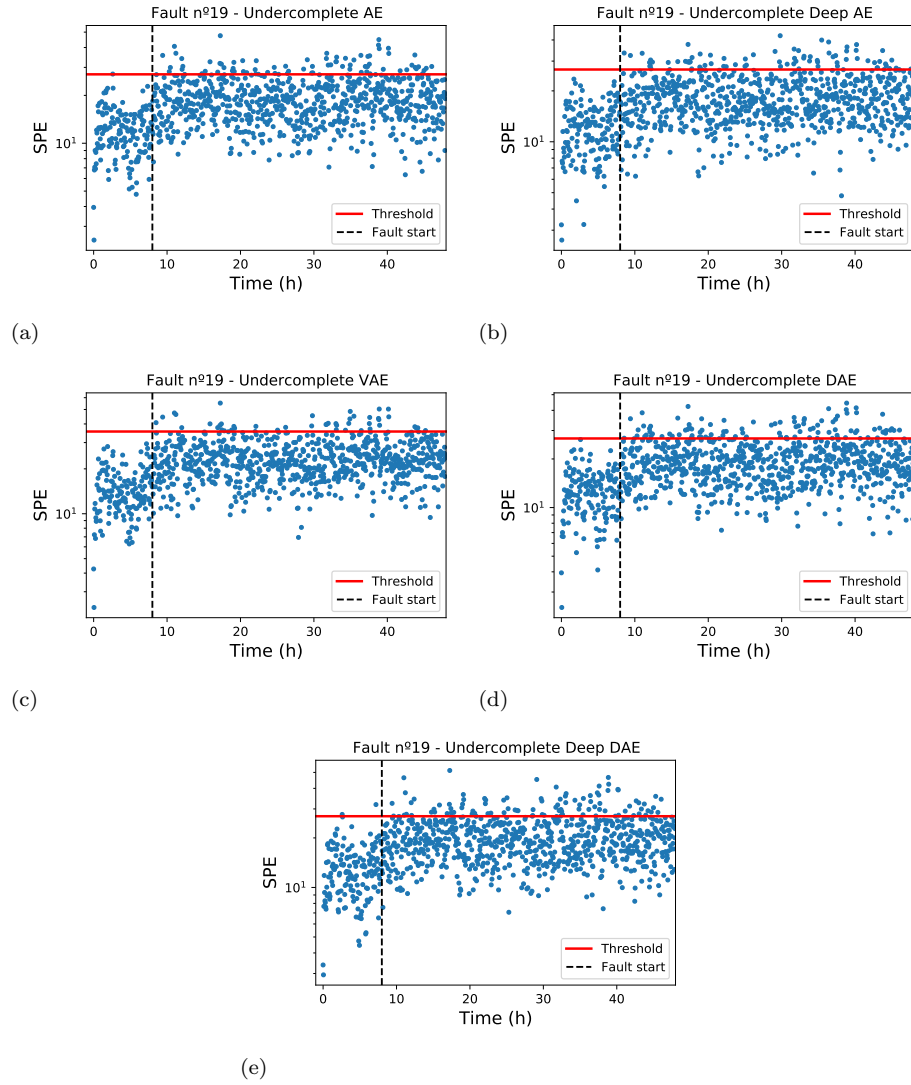


Figure S61: SPE Charts for undercomplete Autoencoders - fault 19: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder



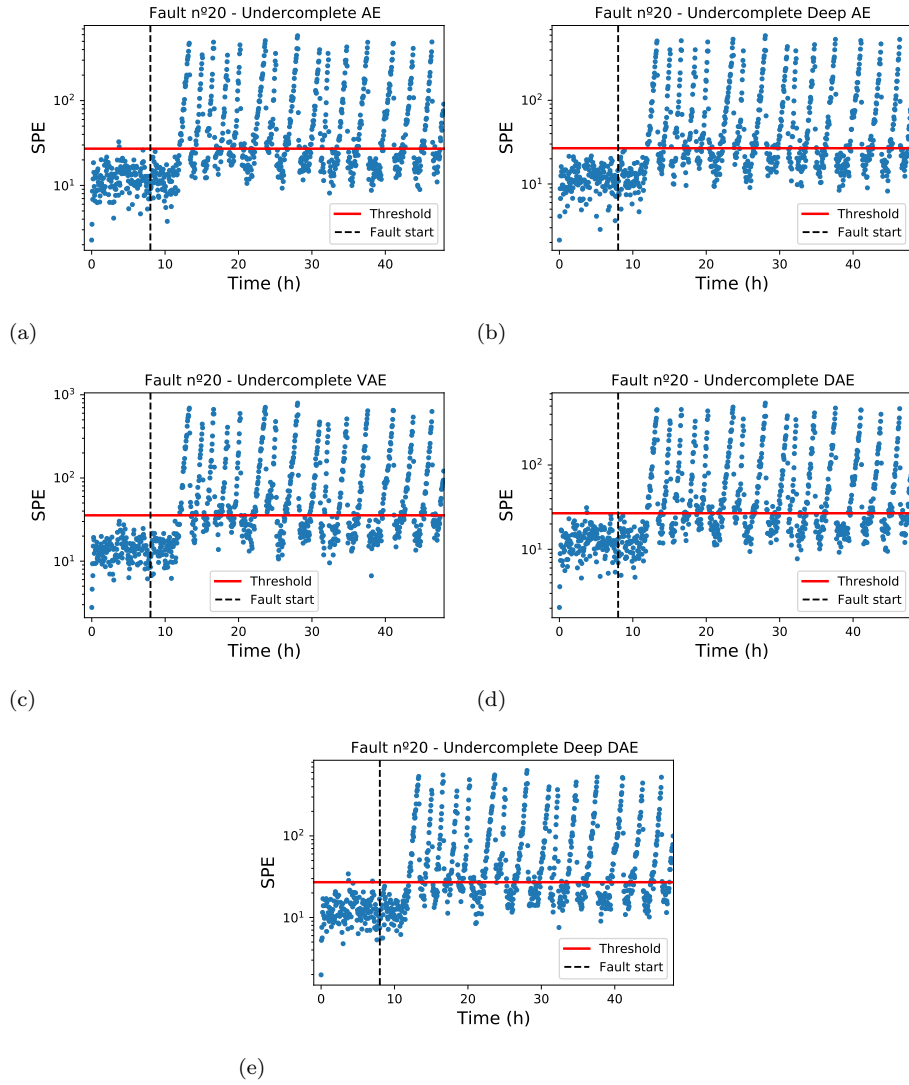


Figure S62: SPE Charts for undercomplete Autoencoders - fault 20: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

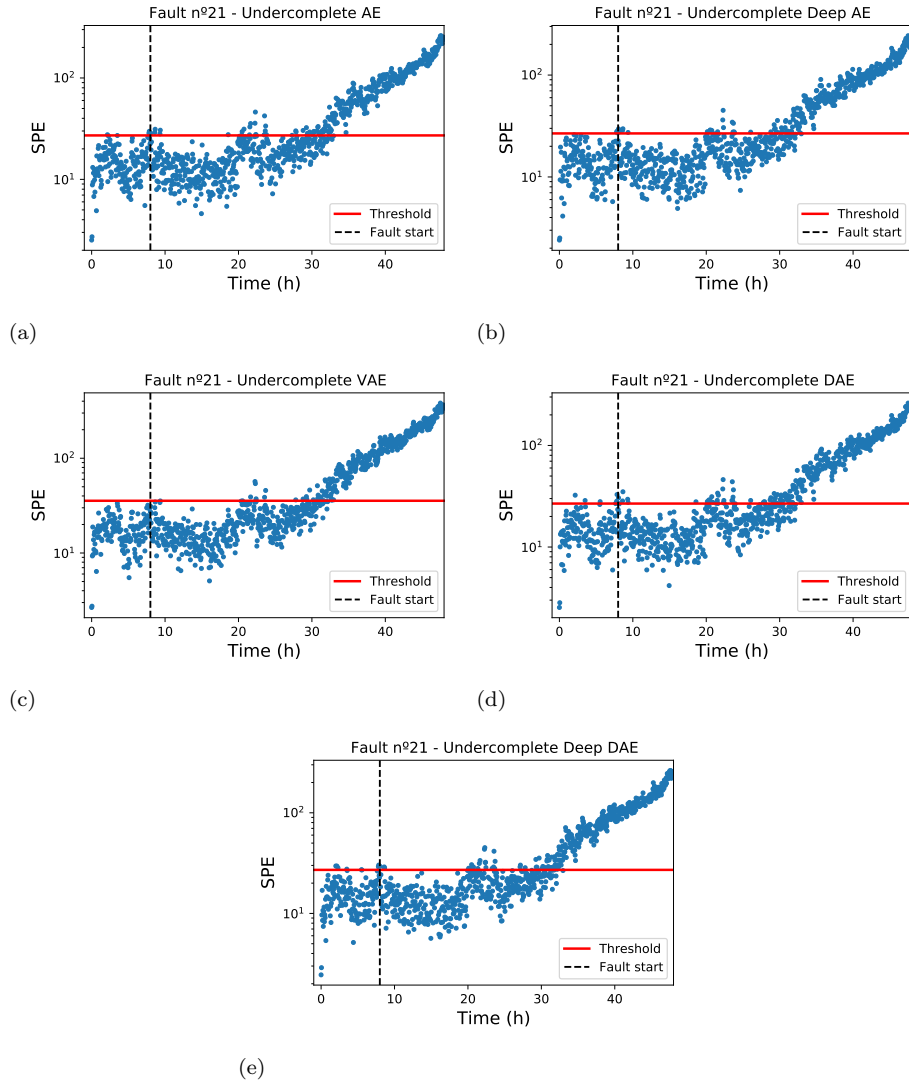


Figure S63: SPE Charts for undercomplete Autoencoders - fault 21: (a) Autoencoder; (b) Deep Autoencoder; (c) Variational Autoencoder; (d) Denoising Autoencoder; (e) Deep Denoising Autoencoder

### 3 Fault Detection Rate for Fixed Threshold Autoencoders

| Fault Number | Autoencoder |        | Deep Autoencoder |        | Variational Autoencoder |        | Denoising Autoencoder |        | Deep Denoising Autoencoder |        |
|--------------|-------------|--------|------------------|--------|-------------------------|--------|-----------------------|--------|----------------------------|--------|
|              | Underc.     | Sparse | Underc.          | Sparse | Underc.                 | Sparse | Underc.               | Sparse | Underc.                    | Sparse |
| 1            | 99.75       | 99.63  | 99.75            | 99.50  | 99.75                   | 99.63  | 99.75                 | 99.75  | 99.75                      | 99.50  |
| 2            | 98.75       | 98.38  | 98.75            | 98.25  | 98.63                   | 98.63  | 98.63                 | 98.63  | 98.75                      | 98.38  |
| 3            | 1.63        | 0.75   | 1.75             | 1.00   | 1.13                    | 1.50   | 2.25                  | 1.63   | <b>2.63</b>                | 0.75   |
| 4            | 96.88       | 66.75  | 99.25            | 55.38  | 96.50                   | 90.50  | 98.75                 | 91.13  | 98.63                      | 65.00  |
| 5            | 36.25       | 99.75  | 99.88            | 25.87  | 26.88                   | 34.50  | 25.75                 | 32.88  | 26.88                      | 25.00  |
| 6            | 99.88       | 99.88  | 99.88            | 99.88  | 99.88                   | 99.88  | 99.88                 | 99.88  | 99.88                      | 99.88  |
| 7            | 99.88       | 99.88  | 99.88            | 99.88  | 99.88                   | 99.88  | 99.88                 | 99.88  | 99.88                      | 99.88  |
| 8            | 97.63       | 97.38  | 97.63            | 97.25  | 97.75                   | 97.63  | 97.63                 | 97.75  | 97.75                      | 97.50  |
| 9            | 2.25        | 1.13   | 1.63             | 1.00   | 1.63                    | 1.38   | 2.00                  | 1.00   | 2.25                       | 0.63   |
| 10           | 48.75       | 48.63  | 48.75            | 47.75  | 52.50                   | 72.88  | 50.63                 | 70.25  | 52.13                      | 54.25  |
| 11           | 59.13       | 54.13  | 66.88            | 51.13  | 63.75                   | 63.38  | 62.13                 | 62.75  | 63.50                      | 54.75  |
| 12           | 98.75       | 98.75  | 99.00            | 98.88  | 98.75                   | 99.00  | 98.88                 | 99.00  | 98.75                      | 98.75  |
| 13           | 95.13       | 95.00  | 95.13            | 94.75  | 95.25                   | 95.25  | 95.25                 | 95.13  | 95.13                      | 94.63  |
| 14           | 99.88       | 99.88  | 99.88            | 99.88  | 99.88                   | 99.88  | 99.88                 | 99.88  | 99.88                      | 99.88  |
| 15           | 3.50        | 3.13   | 2.50             | 5.38   | 4.50                    | 5.75   | 4.25                  | 5.50   | 4.75                       | 3.50   |
| 16           | 38.38       | 36.13  | 38.50            | 36.25  | 44.50                   | 68.00  | 40.50                 | 64.88  | 42.13                      | 45.00  |
| 17           | 92.63       | 86.88  | 92.88            | 86.13  | 92.00                   | 92.50  | 93.63                 | 91.88  | 93.38                      | 87.88  |
| 18           | 90.50       | 89.50  | 90.63            | 89.25  | 89.75                   | 89.75  | 90.75                 | 89.75  | 90.88                      | 89.63  |
| 19           | 9.63        | 1.25   | 9.50             | 0.75   | 5.63                    | 43.25  | 12.25                 | 20.50  | 12.00                      | 1.13   |
| 20           | 52.50       | 49.13  | 52.00            | 47.75  | 53.50                   | 59.50  | 53.00                 | 57.63  | 54.00                      | 49.13  |
| 21           | 45.13       | 38.88  | 45.00            | 38.75  | 43.25                   | 42.25  | 47.88                 | 42.50  | 45.88                      | 38.88  |